408 详解 · 2012 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2012-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

>

说明：Q9 的 A、B、C 三图键位细节在源材料中未齐，本书只按借位规则与 D 图特征讲，不编造其余三图的键位；数值型干扰项未齐的题按规则与通用陷阱讲，不编造具体数值。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 递归阶乘算法的时间复杂度（正确答案：B）

题干与选项：

求整数 n(n≥0)阶乘的算法如下，其时间复杂度是( )

```
int fact(int n )
{ if (n<=1) return 1;
return n*fact(n-1);
}
```

- A. O(log2 n)
- B. O(n)
- C. O(nlog2n)
- D. O(n^2)

入手点： 先写出递归式：算 n! 要先算 (n-1)!，再乘一次 n；数一数一共递归多少层，每层干多少活。

专有名词从零解释：

- 先想“递归”：函数自己调用自己，问题规模一次比一次小，直到小到不用再分。
- 再想“阶乘”：n!＝n×(n-1)×……×2×1。
- 再想“时间复杂度”：衡量工作量随问题规模怎么涨的粗略尺子，只看涨势，不抠常数。

考查知识点：

- 先想递归式：T(n)＝T(n-1)＋1（n＞1），意思是算 n! 的工作量＝算 (n-1)! 的工作量＋一次乘法。
- 再想一共多少层：从 n 一路减到 1，一共 n 层，每层 1 次乘法，总共 n 次乘法，量级是 O(n)，就是 B。
- 拿生活例子验算：n＝5 时，算 5! 要先算 4!，算 4! 要先算 3!，一路算到 1!，再一层一层乘回来，一共 5 次乘法。
- 边界情况：n＝1 时不用递归，直接返回 1，工作量是常数；n＝0 时 0!＝1，同样是常数。
- 易混对子：不要把递归层数和每层的工作量乘起来——每层只做 1 次乘法，不是每层都做 n 次；不要把阶乘的值 n! 当成工作量——值长得快，工作量只是线性涨。

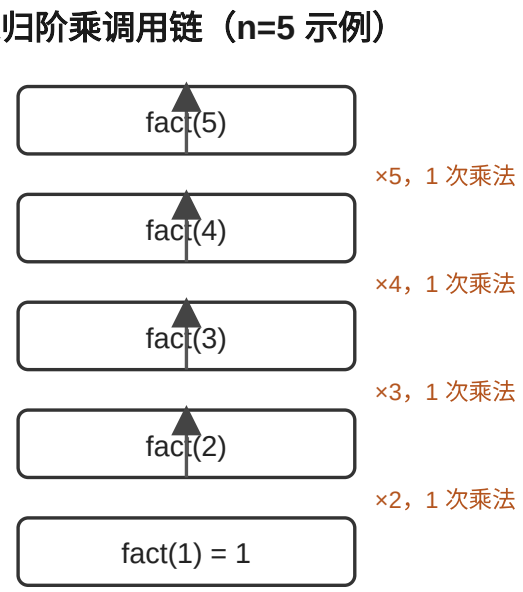
正确选项为什么对： 递归式是 T(n)＝T(n-1)＋1，展开后是 n 项常数相加，量级 O(n)，就是 B，选 B。

错误选项为什么错＋举例：

- 量级算大了（比如算成平方级）：把每层的工作量想成了 n 次。教训：每层只做 1 次乘法，层数是 n，相加不是相乘。
- 量级算小了（比如算成常数级）：只看到最后一层返回 1，漏了前面 n-1 层。教训：递归式要从 n 展开到 1，每层都算。
- 把阶乘的值当成工作量：n! 这个数本身很大，但算它的步数只是 n 步。教训：分清“算出来的数有多大”和“算它要多少步”。

笔记本一句话： 第 1 题＋递归阶乘：T(n)＝T(n-1)＋1，n 层每层 1 次乘法，O(n)。

【图像】



递归阶乘 n 层调用链、每层 1 次乘法的示意图

【定位】 数据结构 · 第1章 绪论 · 递归算法的时间复杂度

【易混对比】

- "O(n)"和"O(n^2)"的区别：O(n) 是 n 层、每层 1 次乘法，把 n 个 1 加起来；O(n^2) 是每层干 n 件事再乘以 n 层；阶乘递归每层只有 1 次乘法，层数和每层工作量是相加，不是相乘。
- "T(n)=T(n-1)+1"和"T(n)=T(n-1)+n"的区别：前者展开是 n 个常数相加，量级 O(n)；后者每层多加一个 n，展开是平方量级；阶乘递归每层只做一次乘法，对应前者。

T(n)=T(n-1)+1 展开是 n 个 1 相加，量级 O(n)，选 B。

【坑位】

分清"阶乘的值 n! 有多大"和"算它要多少步"，别把数大当成步数多。

误选 D（O(n^2)）：把每层的工作量想成了 n 次。教训：每层只做 1 次乘法，n 层是把 n 个 1 相加，不是相乘。误选 A（O(log n)）：把问题规模的缩小速度当成了层数——每次只减 1，层数是 n，不是折半。误选 C（O(n log n)）：log 的出现没有任何依据，是臆想的叠乘。

【套路】

- 第一步，写递归式：算 n! 先算 (n-1)! 再乘一次 n，T(n)=T(n-1)+1。
- 第二步，数层数和每层工作量：从 n 减到 1，一共 n 层，每层 1 次乘法。
- 第三步，展开写量级：n 个 1 相加是 n，量级 O(n)，对上选项 B。

写递归式，数清层数和每层工作量，展开后看量级。

【类比】

像下楼梯：从 n 楼一层一层下到 1 楼，每下一层踩一步（一次乘法），一共 n 步。n! 这个数像雪球越滚越大，但下楼的步数还是 n 步。术语贴回去：楼层数是"递归层数"，踩一步是"一次乘法"，总步数是"时间复杂度 O(n)"。

【口诀】 阶乘递归问量级，层数 n 来每层一；层层相加不是乘，O(n) 选 B 别迟疑。

【考频】 「时间复杂度」在 2009–2022 年解析中出现 12 次 · 高频

第 2 题 · 中缀转后缀时运算符栈的最大深度（正确答案：A）

题干与选项：

已知操作符包括‘+’、‘-’、‘*’、‘/’ 、‘(’ 和 ‘)’。将中缀表达式 a+b-a*((c+d)/e-f)+g 转换

为等价的后缀表达式 ab+acd+e/f-*-g+时，用栈来存放暂时还不能确定运算次序的操

作符，若栈初始时为空，则转换过程中同时保存栈中的操作符的最大个数是( )

- A. 5
- B. 7
- C. 8
- D. 11

入手点： 不用背结论，按中缀转后缀的规则一步步模拟：运算符进栈、出栈，记下栈里最多同时有几个。

专有名词从零解释：

- 先想“中缀表达式”：平时写算式的方法，运算符写在两个数中间，比如 a＋b。
- 再想“后缀表达式”：运算符写在两个数后面，比如 ab＋，算的时候不用括号、不用记优先级。
- 再想“运算符栈”：转化时临时放运算符的一摞盘子，只能从顶上放、只能从顶上拿，后放进去的先拿出来。

考查知识点：

- 先想转化规则：数字直接输出；运算符进栈前，先把栈顶那些优先级不比它低的运算符弹出来输出；左括号直接进栈，右括号把栈里一直弹到左括号为止。
- 再想栈深怎么涨：高优先级的运算符（比如乘除）来了，栈顶低优先级的不会被弹，栈就越摞越高；低优先级的运算符（比如加减）来了，栈顶一串高优先级的都要先弹出来，栈就矮下去。
- 再想本题的模拟结果：一步步模拟下来，栈里最多同时有 5 个运算符，就是 A。
- 拿小例子验算：a＋b×c 转后缀时，＋先进栈，×后进栈（×优先级高，不弹＋），栈里有 2 个；如果后面再来一个低优先级的＋，栈顶的×、＋都要先弹出来，栈先矮下去再进新的＋。
- 边界情况：表达式里没有运算符，栈深是 0；全是同级运算符，栈深最多是 1（来一个弹一个）。
- 易混对子：不要把操作数也算进栈——栈里只放运算符，数字是直接输出的；不要只数某一个时刻的栈深——要数整个过程里出现过的最大值。

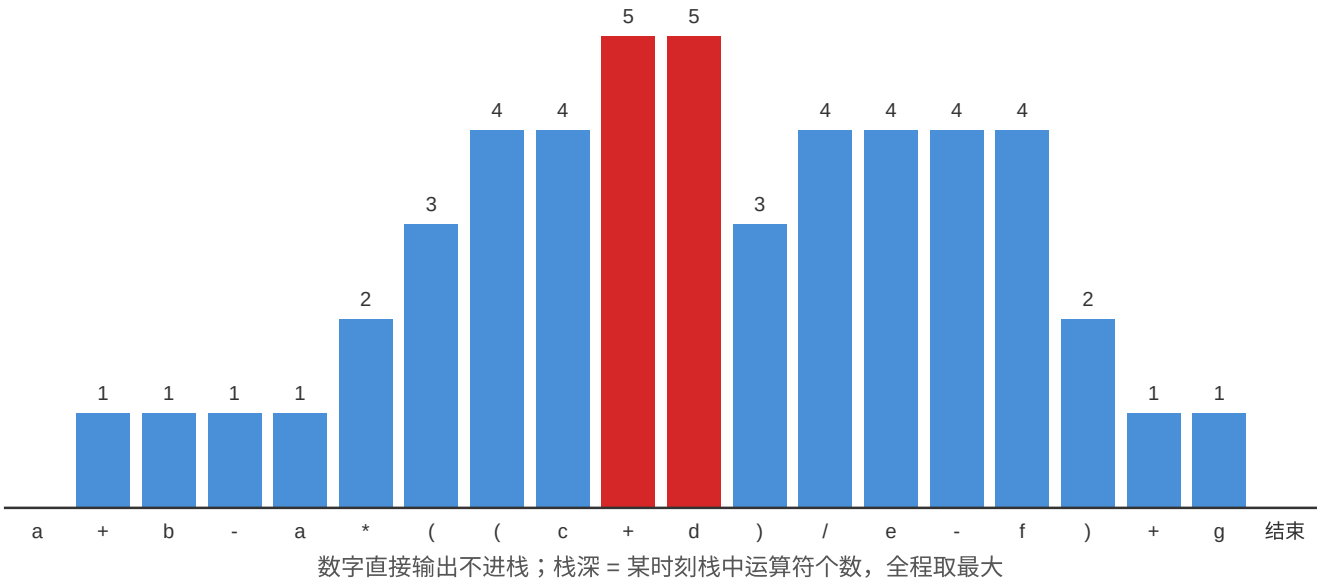
正确选项为什么对： 按规则全程模拟，栈里最多同时有 5 个运算符，就是 A，选 A。

错误选项为什么错＋举例：

- 栈深数少了：只数了某一段的栈深，漏了栈摞得最高的那一刻。教训：最大值要看全程，不能只看一段。
- 栈深数多了：把操作数也算进了栈，或者把已经弹出去的运算符还留在栈里。教训：数字直接输出不进栈，弹出去的就不算了。
- 栈深算错是因为优先级规则用反：低优先级来了不弹栈顶，高优先级来了反而弹栈。教训：规则是“新来的优先级低，栈顶优先级不低的先弹”。

笔记本一句话： 第 2 题＋中缀转后缀：一步步模拟，栈里最多同时有 5 个运算符。

【图像】



中缀转后缀过程中运算符栈深度的变化柱状图，最高点为 5

【定位】 数据结构 · 第3章 栈和队列 · 中缀转后缀的栈应用

【易混对比】

- "栈深"和"运算符总数"的区别：栈深是某一时刻栈里同时存在的运算符个数，不是表达式里运算符的总数；本题运算符很多，栈深最大只有 5。
- "栈"和"输出"的区别：数字直接输出不进栈，栈里只放运算符；右括号来了要把栈一直弹到左括号为止。
- "进栈弹栈规则"和"谁先进谁后出"的区别：新运算符进栈前，先把栈顶优先级不低于它的弹出去输出，这是栈能缩回去的原因；光记后进先出算不出最大深度。

按规则全程模拟，栈里最多同时有 5 个运算符，选 A。

【坑位】

数栈深只数运算符，数字直接输出不进栈，别把操作数也算进去。

数大了（B 7、C 8、D 11）：往往是把操作数算进了栈，或者把已经弹出去的运算符还留在栈里。教训：数字直接输出，弹出去的就不算了。数小了：只看了某一段的栈深，漏了栈摞得最高的那一刻——出现在处理 -((c+d)/e-f) 这一段，栈里同时有 -、*、(、(、+ 共 5 个。优先级用反（低优先级来了不弹栈顶）也会算错：规则是"新来的优先级低，栈顶优先级不低的先弹"。

【套路】

- 第一步，数字直接输出：操作数不进栈。
- 第二步，运算符进栈前先弹：把栈顶优先级不低于新运算符的先弹出去输出，再进栈。
- 第三步，括号特殊处理：左括号直接进栈；右括号把栈一直弹到左括号为止。
- 第四步，全程记最大值：每一步记下栈深，取整个过程的最大值。

数字直出，运算符进栈先弹顶，括号照规则办，全程取最大栈深。

【类比】

像摞盘子：乘除是小盘子，压在加减大盘子上不会被弹走，栈就越摞越高；新的加减大盘子来了，要把上面的小盘子先拿走再放，栈先矮下去。左括号像隔板，直接放进去；右括号像"把隔板以上的盘子全拿走"。术语贴回去：盘子是"运算符"，摞的层数是"栈深"，拿走盘子是"弹栈输出"。

【口诀】 数字直出不进栈，新符进栈先弹顶；左括直进右括清，全程最大是五层。

【考频】 「中缀转后缀」在 2009–2022 年解析中出现 3 次 · 中频

第 3 题 · 已知前序与后序序列时二叉树根结点孩子关系的判断（正确答案：A）

题干与选项：

若一颗二叉树的前序遍历序列为 a, e, b, d, c，后序遍历序列为 b, c, d, e, a，则根节点的孩

子节点( )

- A. 只有 e
- B. 有 e、b
- C. 有 e、c
- D. 无法确定

入手点： 先定根，再看根后面那个结点在两个序列里的位置——它在前序最前、在后序最后，就是根的孩子、其余结点的祖先。

专有名词从零解释：

- 先想“前序遍历”：先看根，再看左子树，再看右子树；所以前序的第一个一定是根。
- 再想“后序遍历”：先看左子树，再看右子树，最后看根；所以后序的最后一个一定是根。
- 再想“孩子、祖先”：孩子是根直接连着的下一层结点；祖先是往上数若干层的结点，孩子也是祖先的一种特殊情况。

考查知识点：

- 先想定根：前序是 a、e、b、d、c，后序是 b、c、d、e、a；前序第一个是 a，后序最后一个也是 a，所以 a 是根。
- 再想去掉根之后：前序剩下 e、b、d、c，后序剩下 b、c、d、e；e 在前序里排第一、在后序里排最后，说明 e 是 a 的孩子，并且 b、c、d 都在 e 的子树里（e 是它们的祖先）。
- 再想根的孩子有几个：a 的孩子只有 e 一个，就是 A。
- 拿生活例子验算：家谱里 a 是爷爷，e 是爸爸，b、c、d 是孩子；前序是“爷爷、爸爸、孩子们”，后序是“孩子们、爸爸、爷爷”——爸爸在前序最前、在后序最后，正好对上。
- 边界情况：只有前序和后序，树的形状本来不唯一；但本题里 e 的位置把关系钉死了，根的孩子只能是 e。
- 易混对子：不要只凭前序就定孩子——前序第二个是 e，只能说明 e 在根的左子树方向，不能说明 e 就是根的孩子；要结合后序一起看。

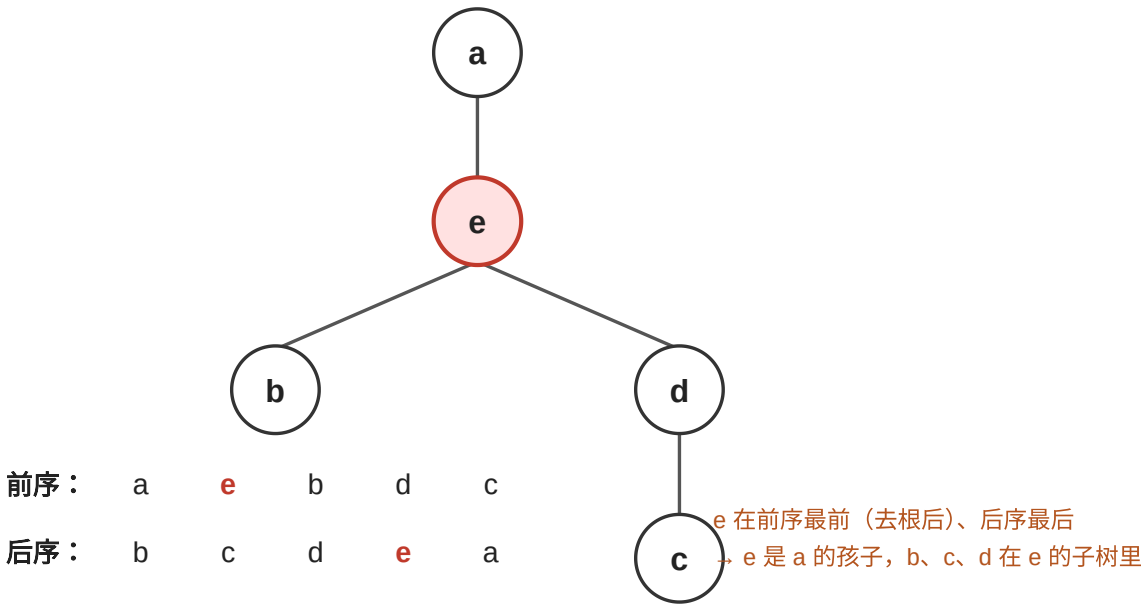
正确选项为什么对： a 是根，e 是 a 的孩子、又是 b、c、d 的祖先，根的孩子只有 e，就是 A，选 A。

错误选项为什么错＋举例：

- 把 b 也当成 a 的孩子：b 在 e 的子树里，不是 a 的直接孩子。教训：前序里排在 e 后面，不等于和 e 平级。
- 把 e 排除在 a 的孩子之外：e 在前序最前、在后序最后，这个位置只能是根的孩子。教训：两个序列要一起用，单看一个序列定不准。

笔记本一句话： 第 3 题＋前序后序定关系：前序首是根，后序尾是根；e 前序最前、后序最后，就是根的孩子、其余结点的祖先。

【图像】



由前序 aebdc 和后序 bcdea 还原的树形，e 在前序最前、后序最后

【定位】 数据结构 · 第5章 树与二叉树 · 由遍历序列确定结点关系

【易混对比】

- "前序第二个"和"根的孩子"的区别：前序第二个 e 只说明 e 在根的左子树方向，不等于 e 就是根的直接孩子；要结合后序一起看。
- "孩子"和"子孙"的区别：孩子是直接相连的下一层；b、c、d 在 e 的子树里，是 e 的子孙、a 的孙辈，不是 a 的孩子。
- "前序+后序"和"前序+中序"的区别：中序能唯一划分左右子树，前序+后序一般定不出唯一树形；但本题 e 在前序最前、后序最后，位置把关系钉死了，根的孩子只能是 e。

a 是根，e 在前序排第一、在后序排最后，是 a 的唯一孩子，选 A。

【坑位】

别只看前序第二个就定孩子，两个序列要一起用。

误选 B（有 e、b）：把 e 的子孙 b 当成了 a 的孩子。教训：前序里排在 e 后面，不等于和 e 平级，b 在 e 的子树里。误选 C（有 e、c）：同样把子孙当孩子，c 也是 e 的子孙。误选 D（无法确定）：被"前序+后序定不出唯一树"吓退了——树形确实不唯一，但 e 的位置把根的孩子关系钉死了，a 的孩子只能是 e。

【套路】

- 第一步，定根：前序首、后序尾都是 a，a 是根。
- 第二步，看 e 的位置：去掉根后，e 在前序最前、在后序最后，e 是 a 的孩子，且 b、c、d 都在 e 的子树里。
- 第三步，数根的直接孩子：a 的直接孩子只有 e 一个。

前序首、后序尾定根；看"去掉根之后"谁在前序最前、后序最后，它就是根的孩子。

【类比】

像家谱点名：前序是"爷爷、爸爸、孩子们"（先报长辈），后序是"孩子们、爸爸、爷爷"（先报晚辈）；爸爸在前序最前、在后序最后，就是爷爷的直接孩子、孩子们的祖先。术语贴回去：爷爷是"根 a"，爸爸是"e"，孩子们是"b、c、d"，报名的顺序是"遍历序列"。

【口诀】 前序首来后序尾，根先钉死不能退；e 居前后两端头，唯一孩子跑不掉。

【考频】 「前序后序」在 2009–2022 年解析中出现 13 次 · 高频

第 4 题 · 各非叶结点平衡因子均为 1 的平衡二叉树高度为 6 时的最少结点数（正确答案：B）

题干与选项：

若平衡二叉树的高度为 6，且所有非叶节点的平衡因子均为 1，则该平衡二叉树的节点总

数为( )

- A. 10
- B. 20
- C. 32
- D. 33

入手点： 平衡因子是 1，意思是左子树比右子树高 1 层；结点最少，就是每棵子树都按最省结点的方式长。

专有名词从零解释：

- 先想“平衡因子”：左子树高度减去右子树高度；平衡二叉树里每个结点的平衡因子只能是 -1、0、1。
- 再想“高度”：从根到最深叶子的层数；高度为 6，就是最深的叶子在第 6 层。
- 再想“最少结点数”：在满足“每个非叶结点平衡因子都是 1、高度是 6”的前提下，结点能省就省。

考查知识点：

- 先想递推关系：记高度为 N 的这种树最少结点数为 C_N；根的左子树高度是 N-1（因为平衡因子是 1，左比右高 1 层），右子树高度是 N-2；两边都按最省的方式长，再加上根自己，所以 C_N＝C_{N-1}＋C_{N-2}＋1。
- 再想 base：高度为 1 的树只有根，C_1＝1；高度为 2 的树是根加一个左孩子，C_2＝2；高度为 3 的树是 C_2＋C_1＋1＝4。
- 再想一路算上去：C_4＝4＋2＋1＝7；C_5＝7＋4＋1＝12；C_6＝12＋7＋1＝20，就是 B。
- 拿小例子验算：高度为 3 时，根的左子树是 2 层（2 个结点）、右子树是 1 层（1 个结点），加上根一共 4 个，递推是对的。
- 边界情况：平衡因子是 1，不是 0——如果平衡因子是 0，两边一样高，结点数会更多；本题问最少，所以每层都按“左高右矮”的最省方式长。
- 易混对子：不要把递推式写成 C_N＝C_{N-1}＋1（漏了右子树）——右子树虽然矮一层，但它也有结点；不要把前五层排满、第六层只放一个结点的方法当成答案——那棵树 32 个结点，但它的非叶结点平衡因子不都是 1，不合题意。

正确选项为什么对： 按 C_N＝C_{N-1}＋C_{N-2}＋1 算到 C_6＝20，就是 B，选 B。

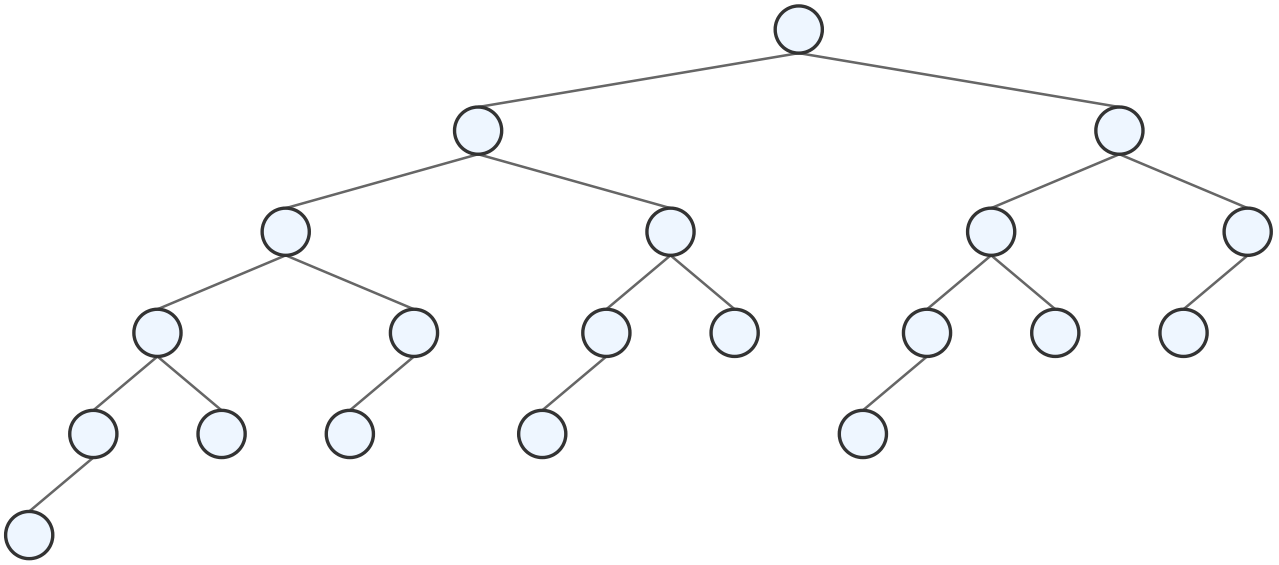
错误选项为什么错＋举例：

- 个数算少了（比如算成 10）：漏了右子树那一支，或者递推时少加了根。教训：根的左右两边都要算，再加根自己。
- 个数算成 32：把前五层排满、第六层只放一个结点。教训：那棵树的非叶结点平衡因子不都是 1，不合“每个非叶结点平衡因子都是 1”这个条件。
- 递推关系列错：把左右子树高度弄反（右比左高），或者把平衡因子 1 理解成右比左高。教训：平衡因子＝左高减右高，等于 1 就是左比右高 1 层。

笔记本一句话： 第 4 题＋平衡因子全为 1：C_N＝C_{N-1}＋C_{N-2}＋1，C_6＝20。

【图像】

平衡因子全为 1、高度为 6 的最少结点树（20 个结点）



C(N) = C(N-1) + C(N-2) + 1，C(6) = 12 + 7 + 1 = 20

平衡因子全为 1、高度为 6 的最少结点平衡二叉树（20 个结点）

【定位】 数据结构 · 第8章 查找 · 平衡二叉树的最少结点数

【易混对比】

- "平衡因子=1"和"平衡因子=0"的区别：=1 是左子树比右子树高 1 层（最省结点的长法），=0 是两边一样高（结点更多）；问最少，所以全按 1 长。
- "最少结点数"和"高度 6 的满树结点数"的区别：满树是结点最多的长法（63 个），最少是每层左高右矮的递推长法（20 个）。
- "C_N=C_{N-1}+C_{N-2}+1"和"C_N=C_{N-1}+1"的区别：后者漏了右子树——右子树虽矮一层，但它也有结点。

C_N=C_{N-1}+C_{N-2}+1，C_1=1、C_2=2，一路算到 C_6=20，选 B。

【坑位】

右子树矮一层不等于没有结点，递推式里右子树那项不能丢。

个数算少（A 10）：漏了右子树那一支，或者递推时少加了根。教训：根的左右两边都要算，再加根自己。个数算成 32（C）：前五层排满、第六层只放一个结点——那棵树 32 个结点，但它的非叶结点平衡因子不全是 1，不合题意。D 33：把递推写成"满树加 1"之类的臆想，没有依据。平衡因子别弄反：平衡因子=左高减右高，=1 就是左比右高 1 层。

【套路】

- 第一步，写递推式：平衡因子为 1，左子树高 N-1、右子树高 N-2，两边都按最省长，再加根自己，C_N=C_{N-1}+C_{N-2}+1。
- 第二步，定 base：C_1=1（只有根），C_2=2（根加一个左孩子）。
- 第三步，逐项算到 6：C_3=2+1+1=4，C_4=4+2+1=7，C_5=7+4+1=12，C_6=12+7+1=20。

平衡因子定左右高度差，写出递推加 base，一路算到目标高度。

【类比】

像盖楼：要求每层楼左半边比右半边高一层，右半边虽矮但也要盖；要盖 6 层高、用的砖最少，就每边都按最省砖的方案盖，最后加上楼顶那块砖。术语贴回去：楼层是"高度"，砖是"结点"，左半边是"左子树"，右半边是"右子树"，最省砖方案是"递推 C_N"。

【口诀】 平衡因子全是一，左高右矮省到底；前后两项加一根，C 六等于二十整。

【考频】 「平衡二叉树」在 2009–2022 年解析中出现 8 次 · 中频

第 5 题 · 邻接表存储下图的广度优先遍历时间复杂度（正确答案：C）

题干与选项：

对有 n 个节点、e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法时间复杂

度( )

- A. O(n)
- B. O(e)
- C. O(n+e)
- D. O(n*e)

入手点： 广度优先遍历干两件事：每个顶点恰好入队一次，每条边在扫描邻接表时至少被看一次；两部分的量级加起来就是答案。

专有名词从零解释：

- 先想“邻接表”：每个顶点后面挂一条链，链上是它的所有邻居；找一个顶点的邻居，只要顺着它的链走一遍。
- 再想“广度优先遍历”：先看起点，再看起点的所有邻居，再看邻居的邻居，一层一层往外扩；用一个队列记着“下一批要看谁”。
- 再想“n、e”：n 是顶点个数，e 是边的条数。

考查知识点：

- 先想顶点部分：每个顶点恰好入队一次、出队一次，这部分是 O(n)。
- 再想边部分：一个顶点出队时，要顺着它的邻接表把邻居都看一遍；整张图走下来，每条边至少被看一次，这部分是 O(e)。
- 再想合起来：O(n)＋O(e)＝O(n＋e)，就是 C。
- 拿生活例子验算：n 个人排队进场，每个人进一次场（n 的部分）；每个人进场时要和它的朋友打招呼，每对朋友关系至少被数一次（e 的部分）。
- 边界情况：无向图里每条边会被看两次（从两头各看一次），但大 O 不计常数 2，量级还是 O(n＋e)；图里没有边时，量级退化成 O(n)。
- 易混对子：不要写成 $O(n^2)$——那是邻接矩阵的量级（矩阵要一行一行扫）；不要只写 O(e)——顶点入队出队的工作不能漏；不要写成 O(n×e)——不是每个顶点都扫全部边，每个顶点只扫自己的邻接表。

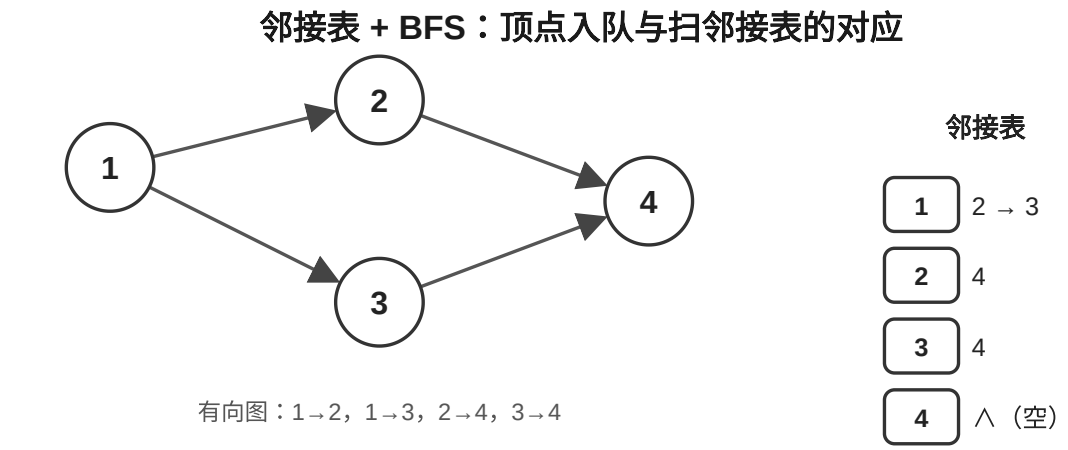
正确选项为什么对： 顶点入队 O(n)＋每边至少看一次 O(e)，合起来 O(n＋e)，就是 C，选 C。

错误选项为什么错＋举例：

- 写成邻接矩阵的量级：邻接矩阵要扫整个 n×n 的表。教训：题目给的是邻接表，不是矩阵，量级不一样。
- 只写边的部分：漏了每个顶点入队出队的工作。教训：遍历的代价是顶点部分加边部分。
- 写成乘积：把“每个顶点”想成了“扫全部边”。教训：每个顶点只扫自己的邻接表，不是全表。

笔记本一句话： 第 5 题＋邻接表 BFS：顶点入队 O(n)＋每边看一次 O(e)，合起来 O(n＋e)。

【图像】



有向图邻接表与 BFS 队列的对应关系示意图

【定位】 数据结构 · 第6章 图 · 广度优先遍历的时间复杂度

【易混对比】

- "邻接表"和"邻接矩阵"的区别：邻接表每个顶点只扫自己的邻接链，边部分是 O(e)；邻接矩阵要扫 n×n 整张表，是 $O(n^2)$。
- "O(n+e)"和"O(n×e)"的区别：加法是顶点部分和边部分各干各的；乘法是每个顶点都扫全部边——邻接表不是这么干的。
- "O(e)"和"O(n+e)"的区别：后者多了一项——每个顶点入队出队的一遍工作，顶点部分 O(n) 不能丢。

每个顶点入队一次 O(n)，每条边至少被看一次 O(e)，合起来 O(n+e)，选 C。

【坑位】

看到"图遍历"先看清存储结构：邻接表是 O(n+e)，邻接矩阵才是 $O(n^2)$。

误选 A（O(n)）：漏了扫边的工作，只数了顶点入队出队。教训：遍历的代价是顶点部分加边部分。误选 B（O(e)）：漏了顶点部分。误选 D（O(n×e)）：把"每个顶点"想成了"扫全部边"——每个顶点只扫自己的邻接表，不是全表。

【套路】

- 第一步，数顶点部分：每个顶点恰好入队一次、出队一次，O(n)。
- 第二步，数边部分：顶点出队时顺着它的邻接表看邻居，整张图走下来每条边至少被看一次，O(e)。
- 第三步，相加：O(n)+O(e)=O(n+e)。

顶点入队数一遍，邻接表扫边数一遍，两项相加。

【类比】

像点名加查通讯录：n 个人每人点一次名（顶点部分），每人再翻自己的通讯录、把每个朋友看一遍（边部分）；每对朋友关系至少被数一次。术语贴回去：点名是"入队出队"，通讯录是"邻接表"，朋友是"边"，一层一层往外扩

是"广度优先"。

【口诀】 顶点入队 n 一遍，邻接表扫边 e 一遍；两项相加不是乘，O(n+e) 选 C。

【考频】 「广度优先遍历」在 2009–2022 年解析中出现 4 次 · 中频

第 6 题 · 邻接矩阵对角线以下全为零的有向图是否存在拓扑序列及其唯一性（正确答案：C）

题干与选项：

若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的

结构是( )

- A. 存在，且唯一
- B. 存在，且不唯一
- C. 存在，可能不唯一
- D. 无法确定是否存在

入手点： 对角线以下全是 0，意思是边只能从小编号指向大编号；编号小的在前、编号大的在后，环就转不起来。

专有名词从零解释：

- 先想“邻接矩阵”：一张 n×n 的表，第 i 行第 j 列是 1，表示有一条从顶点 i 指向顶点 j 的边。
- 再想“对角线以下全为零”：行号大于列号的位置全是 0，也就是说，边只能从小编号指向大编号（i<j 时才可能有边）。
- 再想“拓扑序列”：把顶点排成一列，让每条边都从排在前面的顶点指向排在后面的顶点；有环的图排不出来。
- 再想“有向无环图”：没有环的有向图，一定能排拓扑序列。

考查知识点：

- 先想为什么没有环：边只能从小编号指向大编号，沿着边走，编号只会越走越大，永远回不到出发点，所以成不了环。
- 再想为什么一定存在拓扑序列：没有环的有向图一定有拓扑序列；最直接的一个就是按编号从小到大排（1、2、3、……），每条边都是小编号指向大编号，正好合要求。
- 再想为什么不唯一：反例矩阵 [[0,1,1],[0,0,0],[0,0,0]]，意思是顶点 1 指向 2、指向 3，2 和 3 之间没有边；拓扑序列可以是 1、2、3，也可以是 1、3、2，至少有两个，所以不唯一，就是 C。
- 拿生活例子验算：三门课，1 是先修课，2 和 3 都只要求先修 1，2 和 3 之间没有先后；排课可以是 1、2、3，也可以是 1、3、2。
- 边界情况：图里一条边都没有时，对角线以下当然全是 0，任何排列都是拓扑序列，更不唯一。
- 易混对子：不要把“对角线以下全为零”读成“没有边”——对角线以上可以有边；不要以为“能排出来”就是“只能这样排”——存在和唯一是两回事。

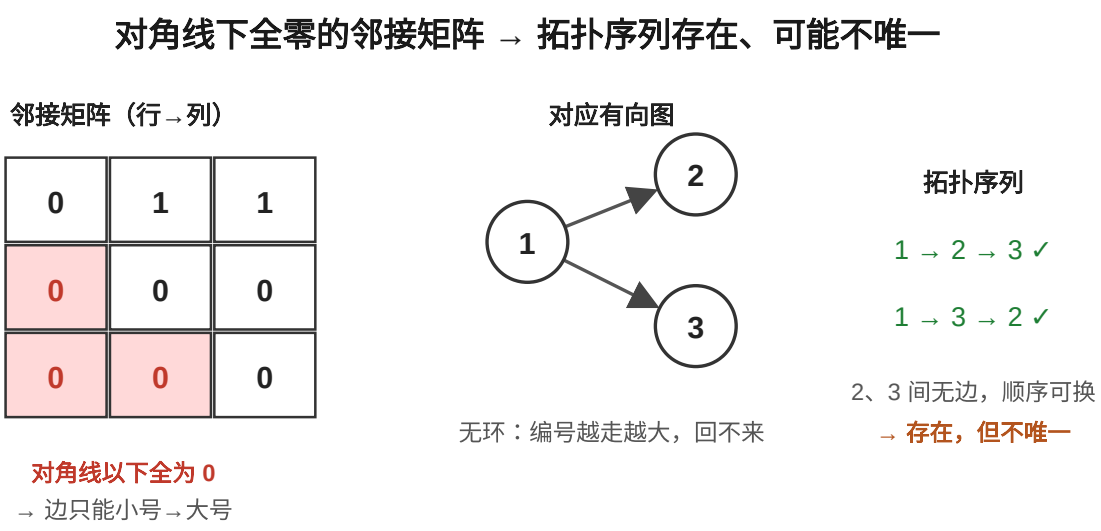
正确选项为什么对： 一定存在拓扑序列（按编号排就行），但不唯一（反例有两个），就是 C，选 C。

错误选项为什么错＋举例：

- 说不存在拓扑序列：边只能从小号指向大号，成不了环，无环图一定能排。教训：先判断有没有环，再谈排不排得出来。
- 说存在且唯一：反例里 2 和 3 没有边，谁先谁后都行。教训：没有边的两个顶点，顺序可以换，唯一性不成立。
- 把对角线以下全为零理解成“图是空的”：对角线以上可以有边，图不一定空。教训：读矩阵要分清对角线上下。

笔记本一句话： 第 6 题＋对角线下全零：边只能小号→大号，无环一定能排；2、3 无边时顺序可换，不唯一。

【图像】



按编号从小到大排就是一个拓扑序列；反例说明唯一性不成立

对角线下全零的邻接矩阵、对应有向图与两种拓扑序列

【定位】 数据结构 · 第6章 图 · 拓扑排序的存在性与唯一性

【易混对比】

- "存在"和"唯一"的区别：存在是排得出来，唯一是只能这么排；本题一定排得出来，但 2、3 间无边时顺序可换，不唯一。
- "对角线以下全零"和"矩阵全零"的区别：对角线以上可以有边，图不一定空；全零只是没边的特例。
- "有环"和"无环"的区别：边只能小号指向大号，编号越走越大、回不到出发点，所以无环；无环图一定有拓扑序列。

边只能小号指向大号，无环，按编号排就是一个拓扑序列；2、3 间无边时顺序可换，不唯一，选 C。

【坑位】

存在和唯一是两道题：先判有没有环，再判顺序能不能换。

误选 A（存在且唯一）：被反例打掉——矩阵里顶点 1 指向 2、指向 3，2 和 3 之间没有边，1、2、3 和 1、3、2 都是拓扑序列。教训：没有边的两个顶点，顺序可以换，唯一性不成立。误选 B（存在且不唯一）：说死了"不唯一"——本题是"可能不唯一"，边足够多时（比如 1→2、2→3、1→3 全连）拓扑序列也可以唯一，"不唯一"不是必然的。误选 D（无法确定是否存在）：错在没看出无环——边只能小号→大号，成不了环，无环图一定能排。

【套路】

- 第一步，读矩阵：对角线下全零，边只能从小编号指向大编号。
- 第二步，判环：沿着边走编号只增不减，回不到出发点，无环。
- 第三步，下结论：无环则一定存在拓扑序列（按编号从小到大排即可）；再用反例看唯一性——2、3 无边时顺序可换，所以可能不唯一。

读矩阵看边的方向，判环定存在性，找反例定唯一性。

【类比】

像爬楼梯：只能上不能下（边只能小号→大号），永远走不回出发点（无环）；从 1 楼开始往上排队就是一种排法（拓扑序列）；同一层（没边相连的顶点）的两个人谁先谁后都行（不唯一）。术语贴回去：楼层是"顶点编号"，上楼是"有向边"，排队是"拓扑序列"。

【口诀】 对角线下全是零，小号大号单向走；无环必定排得出，有边无边看唯一。

【考频】 「拓扑序列」在 2009–2022 年解析中出现 7 次 · 中频

第 7 题 · Dijkstra 算法从 a 出发时顶点被纳入集合 S 的先后次序（正确答案：C）

题干与选项：

如下有向带权图，若采用迪杰斯特拉（Dijkstra）算法求源点 a 到其他各顶点的最短路径，

得到的第一条最短路径的目标顶点是 b，第二条最短路径的目标顶点是 c，后续得到的

其余各最短路径的目标顶点依次是( )

3

1

5

- A. d, e, f
- B. e, d ,f
- C. f,d,e
- D. f,e,d

> 注：有向带权图以原卷为准

入手点： Dijkstra 的规矩是：每次从还没纳入的顶点里，挑一个离起点最近的“钉死”；谁的当前最短路小，谁先进。

专有名词从零解释：

- 先想“最短路”：从起点 a 到某个顶点的所有走法里，总长度最小的那一条。
- 再想“集合 S”：已经“钉死”的顶点集合——这些顶点到 a 的最短路已经确定，不会再变了。
- 再想“dist”：当前已经知道的、从 a 到某个顶点的最短路长度；还没纳入 S 的顶点，dist 还可能变小。

考查知识点：

- 先想纳入顺序：a 先进 S，接着是 b，再接着是 c，然后是 f，再是 d，最后是 e；也就是 a→b→c→f→d→e。
- 再想为什么 f 在 d 前面：f 的当前最短路是经 b、c 的路，长度是 4；d 的当前最短路是经 b 的路，长度是 5；4 比 5 小，所以 f 先被钉死。
- 再想为什么 d 在 e 前面：f 纳入后，d 的路已经确定是 5，e 还要更晚；顺序就是 f、d、e，就是 C。
- 拿生活例子验算：钉钉子先钉离手最近的——f 离起点 4 步远，d 离起点 5 步远，当然先钉 f。
- 边界情况：如果两个顶点的 dist 相等，先后可以换，但本题里 f 和 d 的 dist 不相等，顺序是定的。
- 易混对子：不要按字母顺序排——Dijkstra 不看字母，看 dist；不要按发现顺序排——先发现的顶点不一定先被钉死，要看 dist 最小的那个。

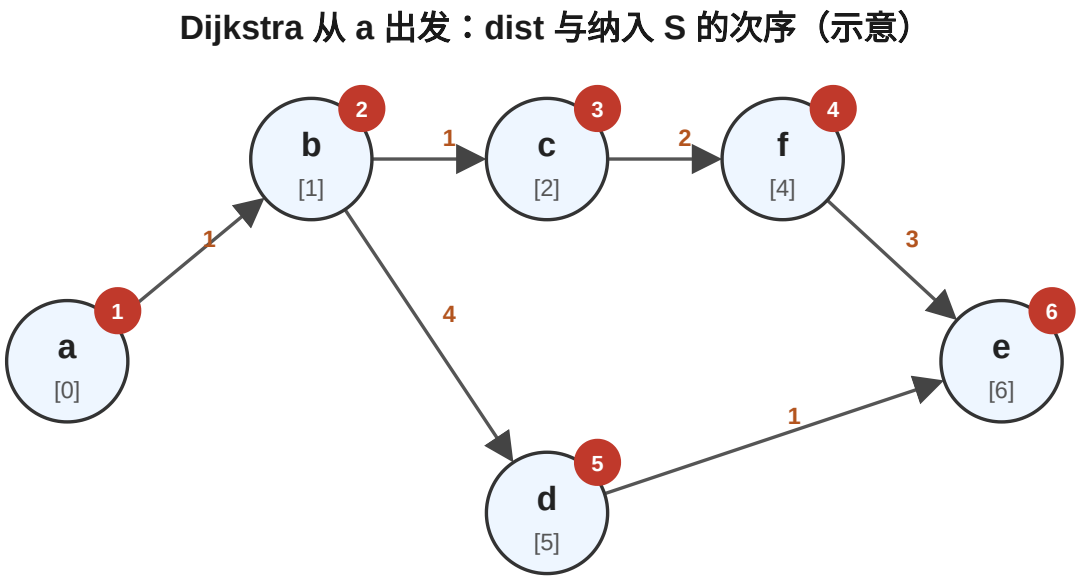
正确选项为什么对： 纳入顺序是 a→b→c→f→d→e，后续是 f、d、e，就是 C，选 C。

错误选项为什么错＋举例：

- 把 d 排在 f 前面：d 的路长 5，f 的路长 4，Dijkstra 每次挑最小的，f 必须先。教训：比的是 dist，不是字母、不是发现顺序。
- 把 e 排在 d 前面：e 的最短路要等 d、f 都定下来之后才定，不可能抢在前面。教训：纳入顺序是 dist 从小到大的顺序。
- 按字母顺序排（b、c、d、e、f）：字母顺序和 dist 顺序没有关系。教训：Dijkstra 的唯一标准是 dist 最小。

笔记本一句话： 第 7 题＋Dijkstra 纳入顺序：每次挑 dist 最小的钉死，a→b→c→f→d→e，f（4）比 d（5）先。

【图像】



圆圈数字 = 纳入 S 的次序；[ ] 内 = 纳入时的 dist

每次挑 S 外 dist 最小的纳入：a→b→c→f(4)→d(5)→e；权值按解析 dist 关系绘制

Dijkstra 从 a 出发的 dist 标注与顶点纳入 S 的次序示意图

【定位】 数据结构 · 第6章 图 · 最短路径（Dijkstra 算法）

【易混对比】

- "纳入 S"和"第一次发现"的区别：先发现的顶点 dist 还可能变小；要等 dist 最小的那个才被"钉死"纳入 S。
- "dist 最小"和"字母顺序"的区别：Dijkstra 只看 dist，不管字母；f 的 dist 是 4，d 的是 5，所以 f 先。
- "Dijkstra"和"BFS"的区别：BFS 按边数（层数）扩，Dijkstra 按 dist（权值和）挑；有权图不能按 BFS 的层数定顺序。

每次挑 dist 最小的纳入 S，a→b→c 之后是 f（4）→d（5）→e，选 C。

【坑位】

纳入顺序看的是 dist 从小到大，不是字母顺序、不是发现顺序。

误选 A（d、e、f）：把 d 排在 f 前面——d 的路长 5、f 的路长 4，Dijkstra 每次挑最小，f 必须先。教训：比的是 dist，不是字母、不是发现顺序。误选 B（e、d、f）：把 e 提前——e 的最短路要等 d、f 定下来之后才定，不可能抢在前面。误选 D（f、e、d）：把 e 排在 d 前面——d 定下来之后 e 才定。两个顶点 dist 相等时先后可换，但本题 f 和 d 的 dist 不相等，顺序是定的。

【套路】

- 第一步，a 进 S：起点 dist 为 0，先纳入。
- 第二步，每次挑 dist 最小的纳入：S 外的顶点里谁的 dist 最小，谁进 S。
- 第三步，纳入后松弛邻居：用新纳入的顶点更新 S 外顶点的 dist。
- 第四步，重复到全纳入：顺序是 a→b→c→f→d→e。

每次挑 S 外 dist 最小的钉死，纳入后松弛邻居，顺序即答案。

【类比】

像钉钉子：离手最近的先钉——f 离起点 4 步，d 离 5 步，先钉 f；已经钉死的不会再动（S 里的顶点 dist 不变）。术语贴回去：钉子是"顶点"，离手的距离是"dist"，钉死是"纳入 S"。

【口诀】 迪杰斯特拉挑最小，dist 小者先进 S；f 四 d 五 e 殿后，C 选项里对号座。

【考频】 「Dijkstra」在 2009–2022 年解析中出现 5 次 · 中频

第 8 题 · 最小生成树形状、权值与 Prim、Kruskal 算法的若干陈述正误判断（正确答案：A）

题干与选项：

下列关于最小生成树的说法中，正确的是( )

Ⅰ、最小生成树的代价唯一

Ⅱ、所有权值最小的边一定会出现在所有的最小生成树中

III、使用普里姆（Prim）算法从不同顶点开始得到的最小生成树一定相同

IV、使用普里姆算法和克鲁斯卡尔（Kruskal）算法得到的最小生成树总不相同

- A. 仅 I
- B. 仅 II
- C. 仅 Ⅰ、III
- D. 仅 II、IV

入手点： 一条一条验：形状是不是唯一、代价是不是唯一、Prim 和 Kruskal 在什么条件下结果相同。

专有名词从零解释：

- 先想“生成树”：把图里所有顶点连起来、又没有环的边集；n 个顶点，生成树恰好 n-1 条边。
- 再想“最小生成树”：所有生成树里，总权值最小的那一棵（可能不止一棵）。
- 再想“代价”：一棵树里所有边的权值加起来。
- 再想“Prim、Kruskal”：两种求最小生成树的算法；Prim 是从一个顶点出发、一次长一条边；Kruskal 是把边按权值从小到大排、一次加一条不成环的边。

考查知识点：

- 先想陈述 I （正确）：最小生成树的代价一定唯一——不管哪棵树是最小的，总权值都是同一个最小值；但形状可能不唯一——权值相同的边可以互换，换出来的树形状不同、代价相同。
- 再想陈述 II （错误）：多条最小权边成环时，总有某条不在某棵最小生成树中——一棵树不能有环，环上的边不可能全进同一棵树；所以“每条最小权边都一定在最小生成树中”这种说法是错的。
- 再想陈述III （错误）：N 个结点成环、N-1 条边权相等时，从不同顶点出发，Prim 先选的边不同，得到的树形状就不同（代价相同）；所以“从不同顶点出发 Prim 结果相同”是错的。
- 再想陈述IV （错误）：最小生成树唯一时，Prim 和 Kruskal 必然得到同一棵树，这是对的；但IV的原话把这个条件说错了——它说成了无条件的相同；最小生成树不唯一时，Prim 和 Kruskal 完全可以得到不同的树（形状不同、代价相同），所以IV是错的。
- 再想组合：只有 I 正确，就是 A。
- 拿生活例子验算：修路连通 4 个村子，有两条一样便宜的路可以二选一；总花费是定的，但具体修哪条可以换，树长得就不一样。
- 边界情况：所有边权都不相等时，最小生成树唯一，Prim 和 Kruskal 结果相同；边权有相等时，形状可能不唯一，代价还是唯一。
- 易混对子：不要把“代价唯一”当成“形状唯一”——代价是数，形状是边集，数唯一不等于边集唯一；不要把“Prim 和 Kruskal 都是求最小生成树的算法”当成“它俩结果一定相同”——不唯一时结果可以不同。

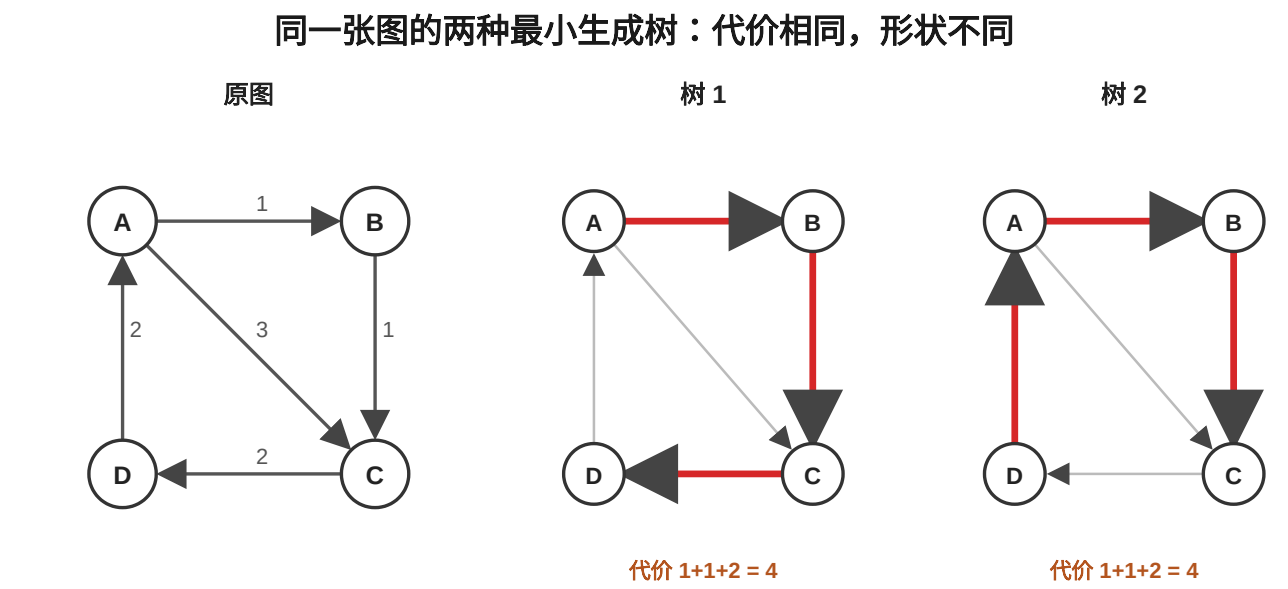
正确选项为什么对： 只有 I 正确，组合是仅 I ，就是 A，选 A。

错误选项为什么错＋举例：

- B（仅 II）：II 是错的——环上的最小权边不可能全进一棵树。教训：树不能有环，环上的边总有被排除的。
- C（仅 I 、III）：III是错的——环上边权相等时，从不同顶点出发 Prim 的结果形状不同。教训：起点不同，先选的边不同，树就不同。
- D（仅 II 、IV）：II 和IV都是错的。教训：II 错在环上边的取舍，IV错在把“唯一时相同”说成了无条件相同。

笔记本一句话： 第 8 题＋最小生成树：代价一定唯一，形状可能不唯一；只有 I 正确。

【图像】



代价都是 4（唯一），用的边不同（形状不唯一）

同一张图的两棵最小生成树：代价相同（都是 4）、形状不同

【定位】 数据结构 · 第6章 图 · 最小生成树（Prim 与 Kruskal）

【易混对比】

- "代价唯一"和"形状唯一"的区别：代价是总权值这个数，一定唯一；形状是边的集合，权值相同的边可以互换，形状可能不唯一。
- "最小权边"和"必在最小生成树中"的区别：多条最小权边成环时，环上的边不可能全进同一棵树，"所有最小权边都在所有最小生成树中"是错的。
- "Prim 从不同顶点出发"和"Prim 结果相同"的区别：环上 N-1 条边权相等时，起点不同、先选的边不同，树形就不同（代价相同）。

I 对（代价唯一），II 、III、IV错，只有 I 正确，选 A。

【坑位】

"代价唯一"不等于"形状唯一"，数唯一和集合唯一是两回事。

误选 B（仅 II）：II 是错的——环上的最小权边不可能全进一棵树，树不能有环。教训：环上的边总有被排除的。误选 C（仅 I 、III）：III是错的——环上边权相等时，从不同顶点出发 Prim 先选的边不同，得到的树形状不同。教训：起点不同，先选的边不同，树就不同。误选 D（仅 II 、IV）：II 和IV都是错的——IV把"最小生成树唯一时 Prim 和 Kruskal 结果相同"说成了无条件相同；最小生成树不唯一时，它俩完全可以得到不同的树（形状不同、代价相同）。

【套路】

- 第一步，验 I：代价是总权值这个数，最小值只有一个，对。
- 第二步，验 II：多条最小权边成环时，环上总有边进不了同一棵树，错。
- 第三步，验III：环上边权相等时，换起点换先选边，形状可变，错。
- 第四步，验IV：加"最小生成树唯一"这个条件才对，无条件说相同，错。
- 第五步，组答案：只有 I 正确，就是 A。

一条一条验：代价看数，形状看边集，环和起点是两个反例来源。

【类比】

像修路连村子：总花费（代价）是定的；但有两条一样便宜的路二选一时，具体修哪条可以换，路网（形状）就不同。术语贴回去：总花费是"代价"，路网是"生成树"，便宜的路是"小权边"，修路的方法是"Prim/Kruskal"。

【口诀】 代价必唯一，形状可换边；环上难全收，起点换则树形变。

【考频】 「最小生成树」在 2009–2022 年解析中出现 3 次 · 中频

第 9 题 · 3 阶 B-树删除关键字 78 后的树形结果（正确答案：D）

题干与选项：

已知一棵 3 阶 B-树，如下图所示。删除关键字 78 得到一棵新 B-树，其最右叶结点中的

关键字是( )

- A. 60
- B. 60, 62
- C. 62, 65
- D. 65

> 注：3 阶 B-树图以原卷为准

入手点： 先看被删关键字所在的结点还剩几个关键字、够不够最低要求；不够就向兄弟借，借的流程是“兄弟的最大关键字上移、双亲的关键字下移”。

专有名词从零解释：

- 先想“B-树”：一种多路平衡查找树，每个结点可以放多个关键字，查找、插入、删除都保持平衡。
- 再想“3 阶 B-树”：每个结点最多放 2 个关键字（阶数减 1），除根以外的每个结点最少放 1 个关键字。
- 再想“兄弟够借”：被删结点关键字不够了，但旁边的兄弟结点有多余的关键字（多于最少数），就可以从兄弟那里借一个过来补。
- 再想“双亲”：被删结点和兄弟结点的共同父结点；借关键字要经过双亲中转，不能直接拿。

考查知识点：

- 先想被删结点的情况：78 所在的结点删前只有 1 个关键字，正好是最少数；删掉 78 之后就空了，不合要求，必须借。
- 再想左兄弟的情况：左兄弟有 2 个关键字，比最少数 1 多 1 个，够借。
- 再想借的流程：左兄弟最大的关键字上移到双亲，双亲里那个比它大的关键字下移到被删结点；这样被删结点又有 1 个关键字了，左兄弟还剩 1 个，双亲的关键字个数不变，整棵树恢复平衡。
- 再想结果树的特征：根 45 不变，左子树是 17、35，右子树是 55、62，叶层从左到右依次是 10、21、37、47、60、65，就是 D。
- 拿生活例子验算：好比一排抽屉，中间一个抽屉空了，左边抽屉有富余，就把左边抽屉里最大的东西先放到中间隔板上、再把隔板上的东西放进空抽屉，三个抽屉都有东西了。
- 边界情况：如果左右兄弟都不够借，就要把被删结点和一个兄弟合并，再把双亲的一个关键字拉下来；本题左兄弟够借，不用走到合并。
- 易混对子：不要直接把兄弟的关键字塞进被删结点——中间必须经过双亲中转，否则顺序就乱了；不要以为删完根会变——本题借位不影响根，根还是 45。
- （注：A、B、C 三图的具体键位在源材料中未齐，这里只按借位规则与 D 图特征讲，不编造其余三图的键位。）

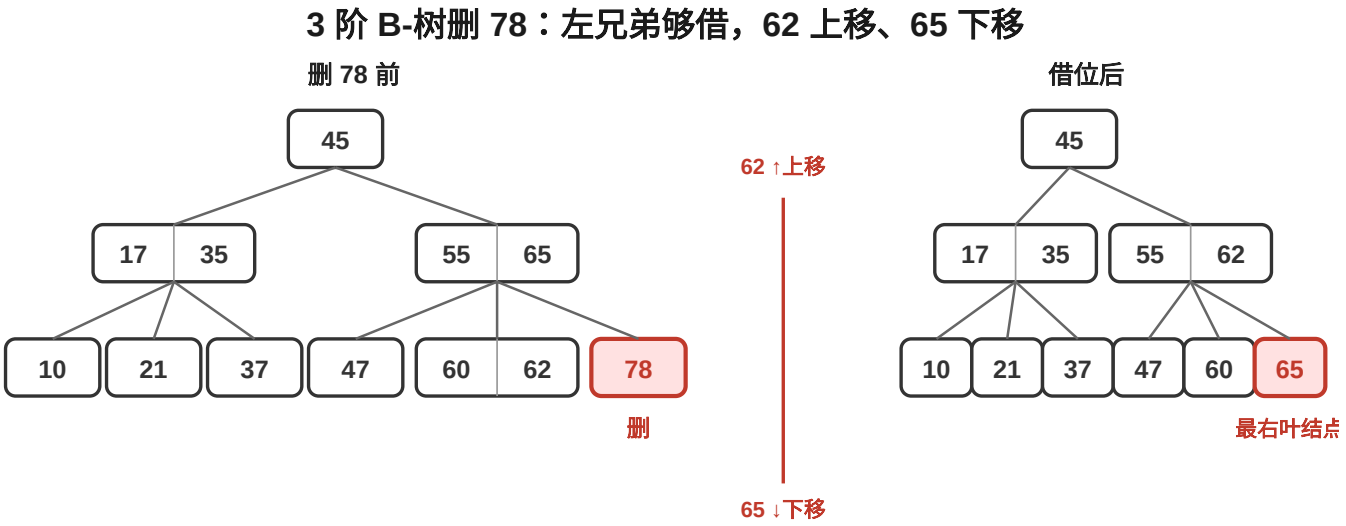
正确选项为什么对： 左兄弟够借，按“兄弟最大关键字上移、双亲关键字下移”的流程借完，树形就是 D 的特征（根 45、左 17、35、右 55、62、叶层 10、21、37、47、60、65），就是 D，选 D。

错误选项为什么错＋举例：

- 借位方向弄反（从右兄弟借、或者把双亲关键字直接拿走不补）：借完之后顺序乱了、或者双亲空了。教训：够借的是左兄弟，流程是上移加下移，两步都要有。
- 删完直接合并：兄弟够借时不用合并，合并是兄弟也不够借时才走的路。教训：先看兄弟够不够借，够借就借，不够借才合并。
- A、B、C 三图在 60、62、65 这几个键的落位上与借位流程不符：按正确流程借完，叶层应该是 10、21、37、47、60、65，键位放错的图都不对。教训：借完之后要按从小到大的顺序验一遍叶层。

笔记本一句话： 第 9 题＋3 阶 B-树删除：结点空了、左兄弟够借，兄弟最大关键字上移、双亲关键字下移，结果是 D 图。

【图像】



借位流程：左兄弟最大关键字 62 上移，双亲关键字 65 下移

3 阶 B-树删 78：左兄弟够借，62 上移、65 下移，最右叶结点为 65

【定位】 数据结构 · 第8章 查找 · B-树的删除（借位）

【易混对比】

- "借"和"合并"的区别：兄弟够借（关键字多于最少数）就借，不够借才合并；本题左兄弟有 2 个关键字，够借，不用合并。
- "直接拿"和"经双亲中转"的区别：借关键字必须走"兄弟最大关键字上移、双亲关键字下移"两步，直接塞进被删结点会乱序。
- "最少关键字"和"最多关键字"的区别：3 阶 B-树每结点最多 2 个（阶数减 1），除根外最少 1 个；78 所在结点删前只有 1 个，删后空了才触发借位。

78 所在结点删空，左兄弟有 2 个关键字够借，"兄弟最大上移、双亲下移"借完，最右叶结点是 65，选 D。

【坑位】

借位是两步：兄弟最大关键字先上移、双亲关键字再下移，一步都不能少。

借位方向弄反（从右兄弟借、或把双亲关键字拿走不补）：借完之后顺序乱了，或者双亲空了。教训：够借的是左兄弟，流程是上移加下移，两步都要有。删完直接合并：兄弟够借时不用合并，合并是两边都不够借时才走的路。教训：先看兄弟够不够借，够借就借，不够借才合并。A、B、C 在 60、62、65 的落位上与借位流程不符：按正确流程借完，叶层从左到右是 10、21、37、47、60、65。

【套路】

- 第一步，看被删结点：78 所在结点删前只有 1 个关键字，删后空了，不合最少数要求。
- 第二步，看兄弟够不够借：左兄弟有 2 个关键字，比最少数多 1 个，够借。
- 第三步，走借位流程：左兄弟最大关键字 62 上移到双亲，双亲的关键字 65 下移到被删结点。
- 第四步，验叶层：从小到大验一遍叶层，最右叶结点是 65。

删空看兄弟，够借走"上移+下移"，最后验一遍叶层顺序。

【类比】

像一排抽屉：中间一个抽屉空了，左边抽屉有富余，就把左边抽屉里最大的东西先放到中间隔板上、再把隔板上的东西放进空抽屉，三个抽屉都有东西了。术语贴回去：抽屉是"结点"，隔板是"双亲结点"，东西是"关键字"，空抽屉是"删 78 后的结点"。

【口诀】 删空先看兄弟借，左兄有余不用合；最大上移双亲下，叶层验序六十五。

【考频】 「B-树」在 2009–2022 年解析中出现 2 次 · 低频

第 10 题 · 每趟排序都能确定一个元素最终位置的排序方法组合判断（正确答案：A）

题干与选项：

在内部排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列

排序方法中，每一趟排序结束都至少能够确定一个元素最终位置的方法是( )

Ⅰ.简单选择排序 Ⅱ.希尔排序 Ⅲ.快速排序

Ⅳ.堆排序 Ⅴ.二路归并排序

- A. 仅Ⅰ、Ⅲ、Ⅳ
- B. 仅Ⅰ、Ⅲ、Ⅴ
- C. 仅Ⅱ、Ⅲ、Ⅳ
- D. 仅Ⅲ、Ⅳ、Ⅴ

入手点： 一个一个验：每趟结束时，有没有一个元素已经站在它最终该站的位置上、以后再也不会动。

专有名词从零解释：

- 先想“最终位置”：整个表排好序之后，这个元素所在的位置；一旦站上去，以后各趟都不再动它。
- 再想“简单选择排序”：每趟从没排好的部分里挑最小的，放到已排好部分的末尾。
- 再想“快速排序”：每趟挑一个枢轴，把比它小的放左边、比它大的放右边，枢轴就站在最终位置上。
- 再想“堆排序”：每趟把堆顶（最大的或最小的）和表尾交换，表尾那个元素就定下来了。
- 再想“希尔排序、二路归并排序”：希尔是分组插排，增量越缩越小；二路归并是把两个有序段合成一个更长的有序段。

考查知识点：

- 先想Ⅰ简单选择排序（对）：每趟挑出的最小元素放到已排好部分的末尾，这个位置就是它最终的位置，以后不动。
- 再想Ⅲ快速排序（对）：每趟结束时，枢轴左边的都比它小、右边的都比它大，枢轴的位置就是最终位置，以后不动。
- 再想Ⅳ堆排序（对）：每趟把堆顶和表尾交换，表尾那个元素就是这一趟定的最终位置，以后不动。
- 再想Ⅱ希尔排序（错）：每趟只是在增量分组内局部有序，增量缩小后元素还会动，没有元素在某一趟就定下来。
- 再想Ⅴ二路归并排序（错）：每趟只是把短的有序段合并成长的有序段，元素在段内的相对位置还会变，要到最后一趟才全部定下来。
- 再想组合：Ⅰ、Ⅲ、Ⅳ正确，就是 A。
- 拿生活例子验算：选班长每轮定一个（简单选择）、一刀切开两边再也不动中间那个（快速排序）、每次把最大的搬到队尾（堆排序），都是“一趟定一个”；希尔和归并是“先局部排、最后才定”，不是一趟定一个。
- 边界情况：表里只有一个元素时，第一趟就定下来了，但这不影响五种方法的定性。
- 易混对子：不要把“每趟都有进展”等同于“每趟定一个最终位置”——希尔和归并每趟也有进展，但没有元素在当趟就定下来；不要把快速排序的枢轴当成“只是大概位置”——枢轴左右两边已经分好大小，它的位置就是最终位置。

正确选项为什么对： Ⅰ、Ⅲ、Ⅳ每趟都定一个最终位置，组合是仅Ⅰ、Ⅲ、Ⅳ，就是 A，选 A。

错误选项为什么错＋举例：

- B（仅Ⅰ、Ⅲ、Ⅴ）：Ⅴ二路归并不是一趟定一个，它每趟只是合并有序段。教训：归并的“有序段变长”不等于“某个元素定下来”。
- C（仅Ⅱ、Ⅲ、Ⅳ）：Ⅱ希尔排序每趟只是分组内局部有序，增量缩小后元素还会动。教训：局部有序不是最终位置。
- D（仅Ⅲ、Ⅳ、Ⅴ）：漏了Ⅰ简单选择（每趟最小元素到位），多了Ⅴ二路归并。教训：一个一个验，不能凭印象凑组合。

笔记本一句话： 第 10 题＋一趟定一个：简单选择、快排枢轴、堆排序根与表尾交换，Ⅰ、Ⅲ、Ⅳ。

【图像】



Ⅱ 希尔：分组内局部有序，增量缩小后还会动；Ⅴ 归并：段合并，元素最后才定

每趟结束：Ⅰ定最小、Ⅲ定枢轴、Ⅳ定表尾 —— 站上去就再也不动

选择、快排、堆排序每趟各定一个元素最终位置的示意图

【定位】 数据结构 · 第8章 排序 · 各排序方法每趟的效果比较

【易混对比】

- "每趟有进展"和"每趟定一个最终位置"的区别：希尔、归并每趟也有进展，但没有元素在当趟就站定最终位置。
- "快速排序的枢轴"和"大概位置"的区别：枢轴左边都小、右边都大，它的位置就是最终位置，不是大概。
- "二路归并的段合并"和"元素定位置"的区别：每趟只是把短的有序段合并成长的有序段，段内元素的相对位置还会变，要到最后一趟才全部定下来。

Ⅰ每趟定最小、Ⅲ每趟定枢轴、Ⅳ每趟定表尾，Ⅱ、Ⅴ当趟定不下来，组合仅Ⅰ、Ⅲ、Ⅳ，选 A。

【坑位】

"有进展"不等于"定位置"，希尔和归并是最后才定。

误选 B（含Ⅴ）：二路归并每趟只是短段并长段，段内相对位置还会变，要到最后一趟才定。教训：段有序不等于元素定位置。误选 C（含Ⅱ）：希尔每趟只在增量分组内局部有序，增量缩小后元素还会动。教训：分组有序不等

于最终位置。误选 D（含 Ⅴ、不含 Ⅰ）：漏了简单选择（每趟定的最小元素位置以后不动），又收了归并。

【套路】

- 第一步，验 Ⅰ 简单选择：每趟挑最小的放到已排好部分的末尾，位置以后不动，对。
- 第二步，验 Ⅱ 希尔：每趟只在增量分组内局部有序，没有元素当趟定下来，错。
- 第三步，验Ⅲ快速排序：每趟枢轴左边都小、右边都大，枢轴位置以后不动，对。
- 第四步，验Ⅳ堆排序：每趟堆顶和表尾交换，表尾元素以后不动，对。
- 第五步，验 Ⅴ 二路归并：每趟只是段合并，元素要到最后一趟才定，错。
- 第六步，组答案：Ⅰ、Ⅲ、Ⅳ正确，就是 A。

一个一个验：看每趟结束有没有元素"站上去就再也不动"。

【类比】

像排队定位置：选班长每轮定一个（简单选择）、一刀切开、中间那人两边再也不动（快速排序）、每次把最高的搬到队尾（堆排序），都是一趟定一个；希尔是"先小组内排、最后才定"，归并是"先排小段、最后才拼成整表"。术语贴回去：班长是"最小元素"，一刀是"枢轴划分"，最高的是"堆顶"，队尾是"最终位置"。

【口诀】 选择定最小，快排定枢轴，堆排定表尾；希尔归并非当趟，一三四对选 A。

【考频】 「排序」在 2009–2022 年解析中出现 46 次 · 高频

第 11 题 · 折半插入排序与直接插入排序在趟数、移动次数、辅助空间和比较次数上的异同（正确答案：D）

题干与选项：

对一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是

( )

- A. 排序的总趟数
- B. 元素的移动次数
- C. 使用辅助空间的数量
- D. 元素之间的比较次数

入手点： 四个方面一个一个比：折半改的只是“找位置用的比较方法”，找位置之外的事（趟数、移动、空间）都没变。

专有名词从零解释：

- 先想“直接插入排序”：前面的表已经排好，新元素从后往前一个一个比，找到位置插进去。
- 再想“折半插入排序”：前面的表已经排好，新元素用折半查找找位置，找到位置再插进去；找位置快了，但插进去时还是要一个一个挪。
- 再想“总趟数”：n 个元素要插 n-1 趟（第一个不用插）。
- 再想“辅助空间”：排序过程中额外用的空间；两个方法都只用一个临时变量放待插元素，是 O(1)。

考查知识点：

- 先想总趟数：两个方法都是 n-1 趟，相同。
- 再想移动次数：两个方法挪元素的方式一模一样——找到位置后，把位置后面的元素一个一个往后挪；挪多少次只取决于初始序列，和找位置的方法没关系，所以相同。
- 再想辅助空间：两个方法都只用一个临时变量，是 O(1)，相同。
- 再想比较次数：这就是不同的那个方面——折半查找找位置，比较次数和初始状态没关系，是 O(n log n)；直接插入从后往前一个一个比，比较次数和初始状态有关，表本来就有序时只要 O(n)，完全逆序时要 $O(n^2)$。
- 再想结论：只有比较次数不同，就是 D。
- 拿生活例子验算：找一本书插进排好的书架——直接插入是从后往前一本一本比，折半插入是翻一半再翻一半；找到空位后，挪书的动作是一样的，挪几本只看原来有多乱。
- 边界情况：表本来就有序时，直接插入每趟只比 1 次，总共 O(n)；折半插入每趟还是要折半比，总共 O(n log n)，反而比直接插入多。
- 易混对子：不要以为折半插入“全方位更快”——它只省了比较，移动一次没少；不要把“移动次数”和“比较次数”混在一起——移动看初始序列的逆序程度，比较看找位置的方法。

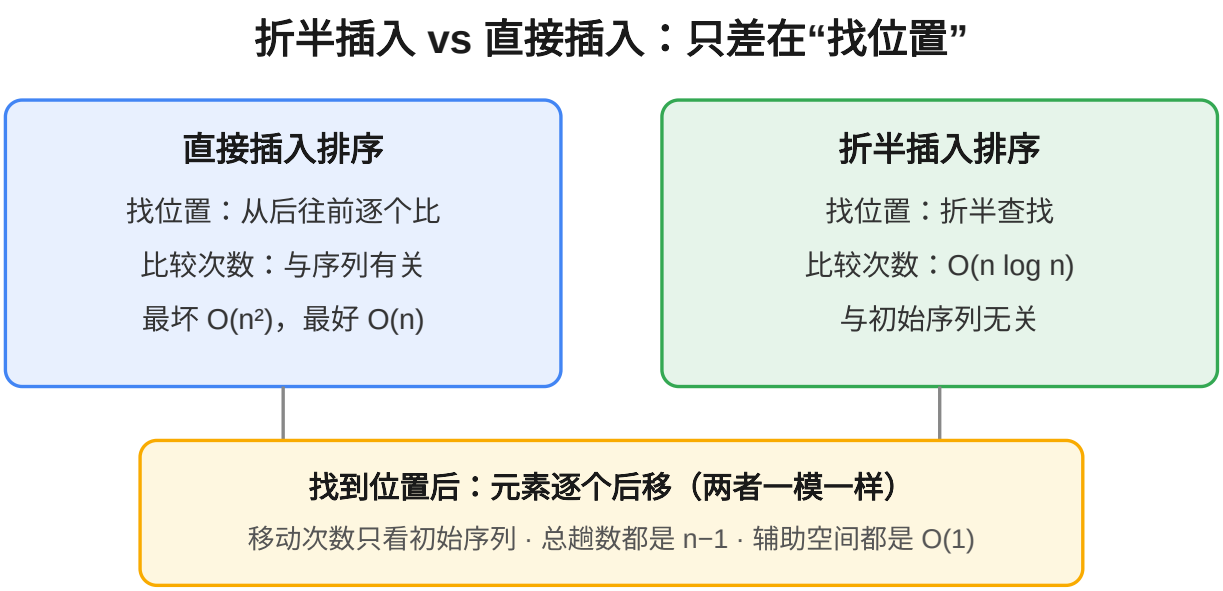
正确选项为什么对： 总趟数、移动次数、辅助空间三方面都相同，只有比较次数不同，就是 D，选 D。

错误选项为什么错＋举例：

- A（总趟数）：两个方法都是 n-1 趟。教训：折半改的是找位置，不是趟数。
- B（移动次数）：两个方法挪元素的方式一模一样，挪多少只看初始序列。教训：找位置快了，挪书的动作一个没少。
- C（辅助空间）：两个方法都只用一个临时变量，是 O(1)。教训：折半查找不用额外数组，空间没变。

笔记本一句话： 第 11 题＋折半 vs 直接插入：折半只省比较，趟数、移动、空间都相同，只有比较次数不同。

【图像】



折半插入与直接插入对比：找位置方式不同、挪元素相同

【定位】 数据结构 · 第8章 排序 · 折半插入与直接插入的异同

【易混对比】

- "比较次数"和"移动次数"的区别：比较看找位置用的方法（逐个比还是折半找）；移动看找到位置之后挪元素的动作，两者挪法一模一样、次数只看初始序列有多乱。
- "折半插入"和"直接插入"的区别：折半只换了找位置的方法，趟数、移动方式、辅助空间都没换。

总趟数、移动次数、辅助空间三方面相同，只有找位置的比较次数不同，选 D。

【坑位】

折半只省了比较，挪元素的动作一个没少，别被"折半"二字骗成全方位更快。

A（总趟数）错在：两个方法都是 n−1 趟，折半改的是找位置，不是趟数。B（移动次数）错在：找到位置后挪法一模一样，挪多少只看初始序列，和找位置的方法没关系。C（辅助空间）错在：两个方法都只用一个临时变量放待插元素，是 O(1)，折半查找不用额外数组。反过来想：如果表本来就有序，直接插入每趟只比 1 次、总共 O(n)，折半插入每趟照样折半比、总共 O(n log n)，反而更多——这正好说明"只有比较次数不同"。

【套路】

- 第一步，比趟数：n 个元素都要插 n−1 趟，两者相同。
- 第二步，比移动：找到位置后都是把后面的元素逐个后移，次数只看初始序列，两者相同。
- 第三步，比空间：都只用一个临时变量，O(1)，两者相同。
- 第四步，比比较：折半查找找位置，O(n log n) 与序列无关；直接插入逐个比，最坏 $O(n^2)$、最好 O(n)，两者不同。

前三项逐个打勾相同，第四项不同，答案就是"比较次数"。

【类比】

像往排好的书架上插一本书：直接插入是从后往前一本一本比，折半插入是翻一半再翻一半找空位；找到空位后，把后面的书一本本往后挪的动作是一样的，挪几本只看原来有多乱。术语贴回去：翻书找空位就是"比较找位置"，挪书就是"元素移动"。

【口诀】 折半只管找位置，挪书趟数都不变；四项里头三项同，比较不同选 D。

【考频】 「插入排序」在 2009–2022 年解析中出现 11 次 · 高频

第 12 题 · CPU 提速后程序总运行时间的计算（正确答案：D）

题干与选项：

假定基准程序 A 在某计算机上的运行时间为 100 秒，其中 90 秒为 CPU 时间，其余为

I/O 时间。若 CPU 速度提高 50%，I/O 速度不变，则运行基准程序 A 所耗费的时间是( )

- A. 55
- B. 60
- C. 65
- D. 70

入手点： CPU 部分和 I/O 部分分开算：CPU 部分按提速比例缩，I/O 部分不动，最后加起来。

专有名词从零解释：

- 先想“CPU 时间”：程序在处理器上真正算东西的时间。
- 再想“I/O 时间”：程序等输入输出（比如读写磁盘）的时间，这段时间 CPU 在等，不在算。
- 再想“速度提高 50%”：新速度是旧速度的 1.5 倍，不是 2 倍。

考查知识点：

- 先想 CPU 部分：原来 90 秒，速度变成 1.5 倍，时间变成 90÷1.5＝60 秒。
- 再想 I/O 部分：I/O 速度不变，还是 10 秒。
- 再想总和：60＋10＝70 秒，就是 D。
- 拿生活例子验算：做饭 90 分钟、等水开 10 分钟；灶火大了 1.5 倍，做饭变成 60 分钟，等水开还是 10 分钟，总共 70 分钟。
- 边界情况：如果程序全是 CPU 时间，总时间就按 1.5 倍缩；如果全是 I/O 时间，总时间一点不变。
- 易混对子：不要把“提高 50%”当成“时间减半”——速度是 1.5 倍，时间是除以 1.5，不是除以 2；不要把 I/O 时间也按 1.5 倍缩——题目说了 I/O 速度不变。

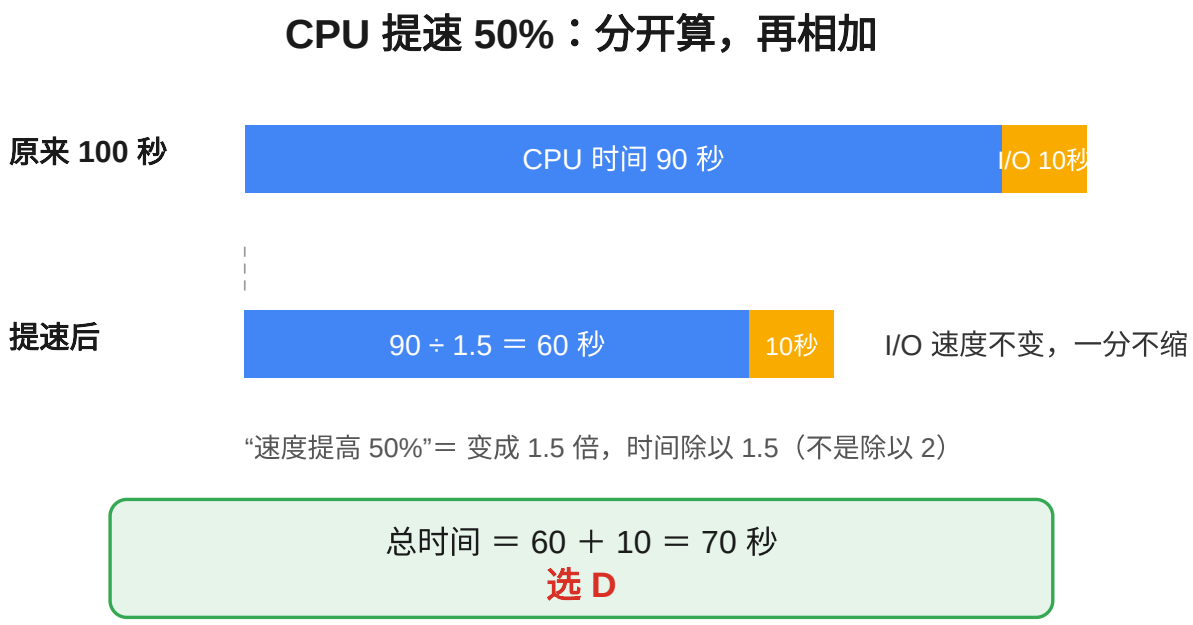
正确选项为什么对： CPU 部分 90÷1.5＝60 秒，I/O 部分 10 秒不变，总共 70 秒，就是 D，选 D。

错误选项为什么错＋举例：

- 按“时间减半”算（90÷2＋10＝55 秒）：把“速度提高 50%”理解成了“速度翻倍”。教训：提高 50% 是变成 1.5 倍，时间除以 1.5。
- 把 I/O 时间也缩了：I/O 速度没变，10 秒一分不能少。教训：分开算，CPU 归 CPU，I/O 归 I/O。
- 只算 CPU 部分（60 秒）：漏了 I/O 的 10 秒。教训：总时间是两部分之和。

笔记本一句话： 第 12 题＋提速算时间：CPU 部分除以 1.5，I/O 部分不动，90÷1.5＋10＝70 秒。

【图像】



CPU 提速前后时间条：CPU 部分从 90 秒缩到 60 秒，I/O 部分 10 秒不动

【定位】 计算机组成原理 · 第1章 计算机系统概述 · CPU 性能与加速比

【易混对比】

- "速度提高 50%"和"速度翻倍"的区别：提高 50% 是变成 1.5 倍，时间除以 1.5；翻倍才是变成 2 倍，时间除以 2。
- "CPU 时间"和"I/O 时间"的区别：CPU 时间跟着 CPU 速度变，I/O 时间跟着 I/O 速度走；本题 I/O 速度不变，10 秒一分不能缩。

CPU 部分 90÷1.5＝60 秒，I/O 部分 10 秒不动，总共 70 秒，选 D。

【坑位】

"提高 50%"是变成 1.5 倍、时间除以 1.5，不是除以 2；I/O 的 10 秒一分不能缩。

A（55 秒）是把"提高 50%"当成了速度翻倍，按 90÷2＋10 算的——1.5 倍和 2 倍差着半档。B（60 秒）是只算了 CPU 部分、漏了 I/O 的 10 秒——总时间是两部分之和。还有一种错法是把 I/O 时间也按 1.5 倍缩——题目说了 I/O 速度不变，缩它没道理。

【套路】

- 第一步，拆开两部分：CPU 时间 90 秒，I/O 时间 10 秒。
- 第二步，算 CPU 部分：速度变成 1.5 倍，时间 90÷1.5＝60 秒。
- 第三步，算 I/O 部分：I/O 速度不变，还是 10 秒。
- 第四步，相加：60＋10＝70 秒。

分开算、各自按自己的速度缩、最后相加。

【类比】

像做饭 90 分钟、等水开 10 分钟：灶火大了 1.5 倍，做饭变成 60 分钟，等水开还是 10 分钟，总共 70 分钟。术语贴回去：灶火就是"CPU 速度"，等水开就是"I/O 等待时间"。

【口诀】 速度一点五，时间除一点五；CPU 会缩 I/O 不动，六十加十选 D。

【考频】 「CPU 性能」在 2009–2022 年解析中出现 6 次 · 中频

第 13 题 · 16 位无符号数转换为 32 位无符号数后的十六进制表示（正确答案：B）

题干与选项：

假定编译器规定 int 和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

unsigned short x = 65530;

unsigned int y = x;

得到 y 的机器数为( )

- A. 0000 7FFAH
- B. 0000 FFFAH
- C. FFFF 7FFAH
- D. FFFF FFFAH

入手点： 先算出这个 16 位数是多少，再按“无符号扩展”的规矩往高位补 0。

专有名词从零解释：

- 先想“无符号数”：只有 0 和正数，没有负数；16 位无符号数最大是 FFFFH（65535）。
- 再想“零扩展”：把短的无符号数变长时，高位全部补 0，数值不变；这是无符号数扩展的唯一正确办法。
- 再想“符号扩展”：把有符号数变长时，高位全部补符号位（正数补 0、负数补 1）；这是有符号数的办法，不能用在无符号数上。

考查知识点：

- 先想这个 16 位数是多少：16 位无符号数的最大值是 FFFFH，按题意减去 5H，得到 FFFAH。
- 再想零扩展：FFFAH 变成 32 位，高 16 位全部补 0，就是 0000FFFAH，就是 B。
- 再想为什么不能用符号扩展：这个数是无符号数，没有符号位；按有符号数做符号扩展，会把最高位的 1 当成负号，高位全补 1，变成 FFFFFFFAH，那是把无符号数错当成有符号数的结果。
- 拿生活例子验算：一个两位数 99 要写成四位数，无符号就是前面补两个 0 写成 0099；不能因为最高位是 9 就前面补 99 写成 9999，那是另一回事。
- 边界情况：16 位数最高位是 0 时，零扩展和符号扩展结果一样（都是补 0），容易蒙对；本题最高位是 1，两种办法结果不一样，正好考这个区别。
- 易混对子：不要把无符号扩展和有符号扩展混用——无符号永远补 0；不要把低 16 位也改了——扩展只动高位，低 16 位 FFFAH 原样保留。

正确选项为什么对： 无符号数零扩展，高位补 0，FFFAH 变成 0000FFFAH，就是 B，选 B。

错误选项为什么错＋举例：

- A（00007FFAH）：低 16 位变成了 7FFAH，等于把原数最高位的 1 丢了再零扩展。教训：扩展只动高位，低 16 位 FFFAH 不能变。
- C（FFFF7FFAH）：高位补了 1，低 16 位又不是原值，两头都错了。教训：无符号扩展高位只能补 0，低位必须原样保留。
- D（FFFFFFFAH）：把无符号数按有符号数做了符号扩展，高位全补 1。教训：看到最高位是 1 就补 1，那是有符号数的办法，无符号数不能用。

笔记本一句话： 第 13 题＋无符号扩展：高位补 0，低位原样保留，FFFAH→0000FFFAH。

【图像】



无符号数零扩展：FFFA 高 16 位补 0 变成 0000FFFA

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 无符号数的零扩展

【易混对比】

- "零扩展"和"符号扩展"的区别：零扩展是无符号数用的，高位全补 0；符号扩展是有符号数用的，高位全补符号位。
- "低 16 位"和"高 16 位"的区别：扩展只动高位，低 16 位 FFFAH 原样保留，一个都不能改。

无符号数扩展只认补 0：高位补 0、低 16 位原样，0000FFFAH，选 B。

【坑位】

看到最高位是 1 就想补 1，那是有符号数的习惯；无符号数永远补 0。

D（FFFFFFFAH）是把无符号数按有符号数做了符号扩展——最高位的 1 被当成了负号，高位全补 1，这是把两家规矩搞混了。A（00007FFAH）是低 16 位被改成了 7FFAH——扩展只动高位，低位 FFFAH 一个都不能动。C（FFFF7FFAH）是两头都错：高位补了 1、低位又不是原值。本题最高位是 1，两种扩展结果不一样，正好考这个区别；如果最高位是 0，两种办法都是补 0，反而考不出来。

【套路】

- 第一步，认类型：unsigned short 转 unsigned int，无符号数。
- 第二步，算原值：65530＝65535−5＝FFFAH。
- 第三步，零扩展：高 16 位补 0，低 16 位原样，得 0000FFFAH。

先认无符号、再算十六进制、最后高位补 0。

【类比】

像把两位数 99 写成四位数：无符号就是前面补两个 0 写成 0099；不能因为最高位是 9 就前面补 99 写成 9999，那是另一套规矩。术语贴回去：补 0 就是"零扩展"，补 99 就是"符号扩展"。

【口诀】 无符号，就补零；高位补零低位留；见一补一是有符号，两家规矩别混头。

【考频】 「零扩展」在 2009–2022 年解析中出现 2 次 · 低频

第 14 题 · IEEE 754 单精度浮点数能表示的最大整数（正确答案：D）

题干与选项：

float 类型（即 IEEE754 单精度浮点数格式）能表示的最大正整数是( )

- A. 2^126 - 2^103
- B. 2^127 - 2^104
- C. 2^127 - 2^103
- D. 2^128 - 2^104

入手点： 先想单精度格式的三段，再想“最大整数”要满足什么：尾数全是 1、指数尽可能大，但指数大到让尾数每一位都是整数位为止。

专有名词从零解释：

- 先想“IEEE 754 单精度”：1 位符号位、8 位阶码、23 位尾数，一共 32 位；数值＝（1＋尾数）×2 的（阶码减 127）次方。
- 先想“阶码”：存在 8 位段里的数，减去 127 才是真正的指数；阶码全 1（255）是特殊值（无穷大、非数），不能用来表示正常数。
- 再想“尾数”：小数点后面的 23 位；前面还有一个隐藏的 1，合起来是 1.尾数。

考查知识点：

- 先想符号：最大整数是正数，符号位是 0。
- 再想阶码：最大的正常阶码是 254，对应指数 254－127＝127。
- 再想尾数：23 位全是 1，1.111……1（23 个 1）＝2－2^-23。
- 再想合起来：(2－2^-23)×2^127＝2^128－2^104，就是 D。
- 再想为什么是“最大整数”：指数是 127 时，尾数的 23 位全是整数位（小数点在 23 位之后），整个数没有小数部分，是个整数；指数再大，阶码就到特殊值了，不能表示正常数。
- 拿生活例子验算：23 位尾数全是 1，就像 23 个 1 排在小数点后面，乘以 2^127 之后，小数点往右挪 127 位，23 位小数全变成了整数位，还多出 104 位整数位。
- 边界情况：尾数全是 0、指数是 127 时，值是 2^127，比 D 小；阶码是 255 时不是正常数，不能算。
- 易混对子：不要写成 2^128——那是把尾数当成 2（1.111……1 不是 2，差了 2^-23 这一项，乘出来差 2^104）；不要把阶码 254 当成指数 254——指数要减 127。

正确选项为什么对： 符号 0、指数 127、尾数全 1，(2－2^-23)×2^127＝2^128－2^104，就是 D，选 D。

错误选项为什么错＋举例：

- 写成 2^128：把 1.111……1 当成了 2，漏了减 2^-23 这一项。教训：尾数全 1 是 2－2^-23，不是 2。
- 阶码没减 127：把阶码 254 直接当指数用，算出来的数大得离谱。教训：单精度指数＝阶码减 127。
- 尾数没算隐藏的 1：只算 0.111……1，少了一个 1。教训：规格化浮点数小数点前面有个隐藏的 1。

笔记本一句话： 第 14 题＋单精度最大整数：符号 0、指数 127、尾数全 1，(2－2^-23)×2^127＝2^128－2^104。

【图像】

单精度最大整数：符号 0 · 指数 127 · 尾数全 1

符号	阶码（8 位）	尾数（23 位全 1）
0	**11111110 = 254**	**111…1（23 个 1）**

指数 = 阶码 − 127 = 254 − 127 = 127

1.111…1 = $2 - 2^{-23}$（含隐藏的 1）

值 = $(2 - 2^{-23}) \times 2^{127} = 2^{128} - 2^{104}$

答案：$2^{128} - 2^{104}$ → 选 D

注意：阶码 255 是特殊值（无穷大/非数），指数不能再大了

尾数全 1 是 $2 - 2^{-23}$，不是 2（差了 2^{104} 这一项）

单精度格式：符号 0、阶码 254、尾数 23 个 1，算出 2^128−2^104

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · IEEE 754 单精度表示范围

【易混对比】

- "阶码"和"指数"的区别：阶码是存下来的 8 位数（最大正常值 254），指数是阶码减 127 之后的真指数（最大 127）。
- "2^128"和"2^128−2^104"的区别：尾数全 1 是 1.111…1＝2−2^−23，不是 2，乘出来要减掉 2^104 这一项。

符号 0、指数 127、尾数全 1，(2−2^−23)×2^127＝2^128−2^104，选 D。

【坑位】

阶码 254 要减 127 才是真指数；尾数全 1 是 2−2^−23，不是 2。

A（2^126−2^103）是把指数取成了 126——阶码 254 减 127 是 127，不是 126。B（2^127−2^104）是漏了小数点前面那个隐藏的 1——只算了 0.111…1，少了一个 1。C（2^127−2^103）同样是指数小了一档。另外注意：阶码 255 是特殊值（无穷大、非数），不能用来表示正常数，所以指数不能再往上取了。

【套路】

- 第一步，定符号：最大正整数，符号位 0。
- 第二步，定指数：最大正常阶码 254，指数＝254−127＝127。
- 第三步，定尾数：23 位全 1，加上隐藏的 1 是 2−2^−23。
- 第四步，乘出来：(2−2^−23)×2^127＝2^128−2^104。

符号 0、阶码 254 减 127、尾数全 1 带隐藏位，乘开就是答案。

【类比】

23 个 1 排在小数点后面，乘以 2^127 之后，小数点往右挪 127 位，23 位小数全变成了整数位，还多出 104 位整数位。术语贴回去：小数点右移就是"指数"的作用，挪 127 位就是"指数 127"。

【口诀】 指数取一百二十七，尾数全一二十三；二减二的负二十三，乘开相减选 D。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 10 次 · 高频

第 15 题 · 结构体按边界对齐、小端存放时各成员的存放地址（正确答案：D）

题干与选项：

某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short

型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

struct{

int a;

char b;

short c;

}record;

record.a = 273;

若 record 变量的首地址为 0xC008，则地址 0xC008 中内容及 record.c 的地址分别是( )

- A. 0x00、0xC00D
- B. 0x00、0xC00E
- C. 0x11、0xC00D
- D. 0x11、0xC00E

入手点： 先按成员大小和对齐要求把每个成员的地址排出来，再按小端把多字节成员的字节顺序写出来。

专有名词从零解释：

- 先想“结构体”：把几个不同类型的数据打包在一起的一个整体；成员一个接一个存，但每个成员的地址要满足对齐要求。
- 再想“边界对齐”：一个占 k 字节的成员，它的地址必须是 k 的倍数；比如 2 字节成员的地址必须是偶数，不能放在奇数地址上。
- 再想“小端存放”：多字节的数，低字节放在低地址、高字节放在高地址；比如 0x00000111，低字节 0x11 放在最小的那个地址上。

考查知识点：

- 先想成员大小：a 占 4 字节、b 占 1 字节、c 占 2 字节，一共 7 字节；按边界对齐后，整个结构体占 8 字节（多出 1 字节空位）。
- 再想排地址：a 从 0xC008 开始，占 C008～C00B；b 在 C00C，占 1 字节；c 是 2 字节成员，地址必须是偶数，C00D 是奇数、放不下，空出来，c 从 C00E 开始，占 C00E～C00F。
- 再想小端：a 的值是 0x00000111，按小端从低地址到高地址依次是 0x11、0x01、0x00、0x00，正好占 C008～C00B。
- 再想答案：c 的地址是 0xC00E，就是 D。
- 拿生活例子验算：书架上放书，4 本一套的书占 4 格，1 本的书占 1 格，2 本一套的书只能从偶数格开始放；奇数格空着没办法，只能空出来。
- 边界情况：如果 c 是 1 字节成员，就不用空 C00D，直接放 C00D；如果 b 不存在，c 也可以直接放 C00C（偶数）。

- 易混对子：不要忘了对齐空位——7 字节成员按对齐排要占 8 字节，空的那 1 字节也要算；不要用大端读小端机器——大端是高字节放低地址，和小端正好反过来。

正确选项为什么对： a 占 C008～C00B，b 在 C00C，C00D 因对齐空出来，c 在 C00E，就是 D，选 D。

错误选项为什么错＋举例：

- 把 c 放到 C00D：C00D 是奇数，2 字节成员不能放奇数地址。教训：对齐要求先算，再排地址。
- 把 a 的字节顺序写反：小端是低字节放低地址，0x11 在 C008，不是在 C00B。教训：先分清小端还是大端。
- 忘了 b 占 1 字节：b 不占位置，c 的地址就往前算错了。教训：每个成员都要占位置，1 字节也是 1 字节。

笔记本一句话： 第 15 题＋结构体对齐小端：a 在 C008～C00B，b 在 C00C，C00D 空出来，c 在 C00E。

【图像】

结构体对齐 ＋ 小端：c 从 C00E 开始

地址	内容	成员
C008	**0x11**	a（int，4 字节），273 ＝ 0x111，小端低字节在前
C009	0x01	
C00A	0x00	
C00B	0x00	
C00C	b	b（char，1 字节）
C00D	空	奇地址，2 字节成员住不下，对齐空出
C00E	c	c（short，2 字节）从偶地址 C00E 开始

C008 内容 0x11，c 的地址 0xC00E → 选 D

内存布局：a 占 C008～C00B（小端首字节 0x11），b 在 C00C，C00D 空出，c 从 C00E 开始

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 边界对齐与小端存放

【易混对比】

- "边界对齐"和"紧挨着放"的区别：对齐要求 k 字节成员的地址是 k 的倍数，2 字节成员不能住奇地址；紧挨着放不管这个，会把 c 塞进 C00D。
- "小端"和"大端"的区别：小端是低字节放低地址，0x11 在 C008；大端是高字节放低地址，0x11 就跑到 C00B 去了。

a 占 C008～C00B（小端首字节 0x11），b 在 C00C，C00D 为对齐空出，c 从偶地址 C00E 开始，选 D。

【坑位】

2 字节成员不能住奇地址，C00D 必须空出来；小端低字节在前，C008 里是 0x11 不是 0x00。

A、C 把 c 放到 C00D——C00D 是奇数，2 字节成员住不下，对齐要求先算、再排地址。A、B 把 C008 的内容写成 0x00——那是大端读法；小端是低字节在前，273＝0x00000111，低字节 0x11 在最小的地址 C008。还有一个算

法：a 4 字节、b 1 字节、c 2 字节一共 7 字节，按对齐排要占 8 字节，空的那 1 字节也要算进去。

【套路】

- 第一步，排成员：a 4 字节占 C008～C00B，b 1 字节占 C00C。
- 第二步，算对齐：c 是 2 字节成员、地址要是偶数，C00D 是奇数、跳过，c 从 C00E 开始。
- 第三步，摆字节：273＝0x111，小端低字节 0x11 放 C008，高字节往后排。

先按大小排地址、再按对齐跳空位、最后按小端摆字节。

【类比】

像在书架上放书：4 本一套的占 4 格，1 本的占 1 格，2 本一套的只能从偶数格开始放，奇数格空着也没办法。术语贴回去：偶数格就是"地址是 2 的倍数"，空着的格子就是"对齐空位"。

【口诀】 对齐先算再排位，奇地址上不住二；小端低字在前头，C008 里面是 0x11。

【考频】 「边界对齐」在 2009–2022 年解析中出现 8 次 · 中频

第 16 题 · Flash 存储器的性质归属判断（正确答案：A）

题干与选项：

下列关于闪存（Flash Memory）的叙述中，错误的是( )

- A. 信息可读可写，并且读、写速度一样快
- B. 存储元由 MOS 管组成，是一种半导体存储器
- C. 掉电后信息不丢失，是一种非易失性存储器
- D. 采用随机访问方式，可替代计算机外部存储器

入手点： 把 Flash 的每一条性质往选项上套：可读可写、浮栅电荷、属 ROM、先擦后写、写得慢、断电不丢、随机访问、组成 SSD——全对得上的就是它。

专有名词从零解释：

- 先想“Flash”：一种能擦能写的只读存储器，U盘、固态硬盘里用的就是它。
- 再想“EEPROM”：电可擦可编程的只读存储器；Flash 就是在它基础上进一步发展出来的，擦写单位更大、成本更低。
- 再想“浮栅”：MOS 管里一个悬空的栅极，电荷困在里面出不来，靠有没有电荷来记 0 和 1；断电了电荷还在，所以数据不丢。
- 再想“非易失”：断电之后数据还在；反过来，断电就丢数据的叫易失。

考查知识点：

- 先想 A 的每一条：Flash 是 EEPROM 进一步发展来的；可读可写；靠 MOS 浮栅里的电荷存信息；归类上属于 ROM（只读存储器家族）；写入之前必须先擦除；写比读慢不少；断电不丢数据，是非易失的；可以随机访问（想读哪块就读哪块，不用顺序找）；固态硬盘就是由 Flash 组成的。
- 再想为什么这些合起来只能是 Flash：可读可写排除了纯 ROM；浮栅电荷排除了磁盘和光盘；先擦后写是 Flash 的标志性写法；非易失加随机访问加 SSD，合起来就是 Flash 的画像。
- 再想其余选项：B（半导体存储器）、C（非易失性存储器）、D（计算机外部存储器）只知道名称、具体表述未齐，这里不硬编它们的原文；判断方法是把 A 的每一条往 Flash 身上套，全对得上，而其余选项的描述里至少有一条与 Flash 对不上。
- 拿生活例子验算：U盘就是 Flash——拔了电文件还在（非易失），能反复写（可读可写），写大文件比读慢，写之前要先擦掉旧的（先擦后写）。
- 边界情况：Flash 也分 NOR 和 NAND，读写特性和擦写单位有差别，但“先擦后写、非易失”这两条是共有的。
- 易混对子：不要把 Flash 当成 RAM——RAM 断电就丢，Flash 不丢；不要把 Flash 当成磁盘——磁盘是磁记录、机械寻道，Flash 是电荷存储、随机访问；不要以为 Flash 可以像 RAM 那样按字节随意改写——改写之前必须先擦除。

正确选项为什么对： A 的每一条都对得上 Flash（EEPROM 发展、可读可写、浮栅电荷、属 ROM、先擦后写、写慢、非易失、随机访问、SSD），就是 A，选 A。

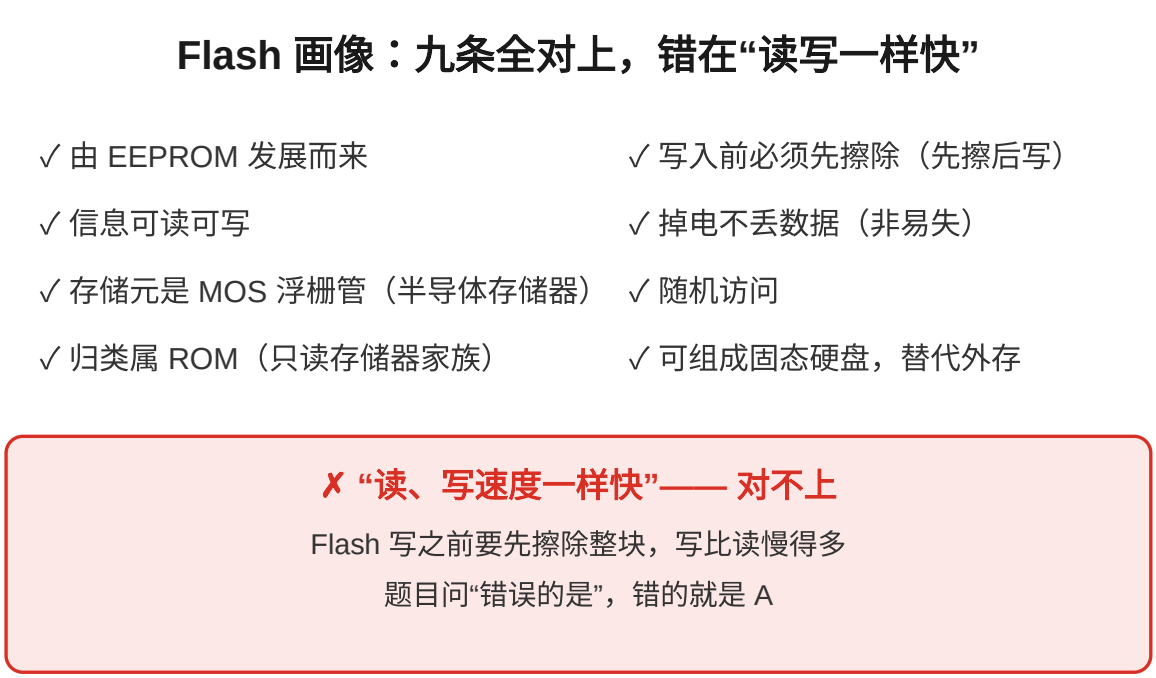
错误选项为什么错＋举例：

- 把 Flash 当成易失性存储器：断电丢数据的说法安到 Flash 头上是错的。教训：Flash 的电荷断电还在，是非易失的。

- 把 Flash 当成磁盘类外存：磁记录、机械寻道那一套不是 Flash 的。教训：Flash 是半导体电荷存储，没有机械动作。
- 以为 Flash 可以按字节随意改写：不知道先擦后写。教训：Flash 写之前必须先擦除，这是它的标志性限制。
- （B、C、D 的具体表述在源材料中未齐，这里不编造原文；按 A 的正确描述与上面这些通用陷阱判断。）

笔记本一句话： 第 16 题＋Flash 画像：EEPROM 发展、浮栅电荷、属 ROM、先擦后写、写慢、非易失、随机访问、SSD。

【图像】



Flash 画像卡：九条真特点全对上，唯独"读写一样快"对不上

【定位】 计算机组成原理 · 第3章 存储器层次结构 · Flash 存储器的特性

【易混对比】

- "Flash"和"RAM"的区别：RAM 断电丢数据，Flash 断电不丢；RAM 可按字节改写，Flash 改写之前必须先擦除。
- "读"和"写"的区别：Flash 读得快、写得慢，A 说"读、写速度一样快"对不上。

Flash 写比读慢得多，"读、写速度一样快"这条对不上，错误的是 A。

【坑位】

题目问的是"错误的是"，真特点一条都不能选；Flash 先擦后写、写得慢，这两条最爱考。

B（存储元由 MOS 管组成、是半导体存储器）是真特点——Flash 靠 MOS 浮栅里的电荷记 0 和 1。C（掉电不丢失、非易失）是真特点——电荷困在浮栅里出不来，断电还在。D（随机访问、可替代外部存储器）是真特点——想读哪块就读哪块，U 盘、固态硬盘就是 Flash 做的。A 错在"读写一样快"：Flash 写之前要先擦除整块，写比读慢不少。

【类比】

U 盘就是 Flash：拔了电文件还在（非易失），能反复写（可读可写），写大文件比读慢，改文件前要先擦掉旧的（先擦后写）。术语贴回去：擦掉旧的就是"擦除"，擦除是整块擦、写是按页写。

【口诀】 Flash 读写不一样，写前先擦是老规；浮栅电荷断电在，U 盘固态都是它。

【考频】 「Flash」在 2009–2022 年解析中出现 2 次 · 低频

第 17 题 · 2 路组相联 Cache 的地址映射与替换、命中判断（正确答案：C）

题干与选项：

假设某计算机按字编址，Cache 有 4 个行，Cache 和主存之间交换的块大小为 1 个字。

若 Cache 的内容初始为空，采用 2 路组相联映射方式和 LRU 替换策略。访问的主存地

址依次为 0,4,8,2,0,6,8,6,4,8 时，命中 Cache 的次数是( )

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 先按组映射把每个块号分到组里，再按组内 2 路、LRU 替换一步步模拟，数命中次数。

专有名词从零解释：

- 先想“Cache”：CPU 和主存之间的一小块高速存储器，存着最近用过的数据块；CPU 要数据先看 Cache 里有没有。
- 再想“命中”：要找的块正好在 Cache 里，不用去主存拿；没找到叫缺失，要去主存拿，还要把新块装进 Cache。
- 再想“2 路组相联”：Cache 分成若干组，每组有 2 个位置；一个主存块只能进它对应的那个组，但在组里 2 个位置任选一个。
- 再想“LRU 替换”：组满了要装新块时，把组里最久没用过的那个块换出去。

考查知识点：

- 先想组映射：0～1、4～5、8～9 号块进第 0 组，2～3、6～7 号块进第 1 组。
- 再想一步步模拟（访问序列是 0、4、8、2、0、6、8、6、4、8）：
- 访 0（组 0）：缺失，组 0 装入 0。
- 访 4（组 0）：缺失，组 0 装入 4，组 0 已满（0、4）。
- 访 8（组 0）：缺失，组 0 已满，按 LRU 换出最久没用的 0，装入 8，组 0 是（4、8）。
- 访 2（组 1）：缺失，组 1 装入 2。
- 访 0（组 0）：缺失（0 已经被 8 换出去了），组 0 已满，换出最久没用的 4，装入 0，组 0 是（8、0）。
- 访 6（组 1）：缺失，组 1 装入 6，组 1 已满（2、6）。
- 访 8（组 0）：命中，这是第 1 次命中；8 变成组 0 里最新用过的。
- 访 6（组 1）：命中，这是第 2 次命中。
- 访 4（组 0）：缺失，组 0 已满，换出最久没用的 0，装入 4，组 0 是（8、4）。
- 访 8（组 0）：命中，这是第 3 次命中。
- 再想总数：命中 3 次，就是 C。
- 拿生活例子验算：两个抽屉一组，组里只能放两样东西；东西满了再放新的，就把最久没动的那样拿出来——第 5 次访 0 时 0 已经被拿出来了，所以是缺失，不是命中。
- 边界情况：组里还没满时，新块直接装进去，不用换出；命中不改变组里的块数，只改变“谁最新用过”。

- 易混对子：不要把“组”当成“整个 Cache”——一个块只能进它对应的组，不能随便放进别的组；不要把 LRU 方向弄反——换出的是最久没用的，不是最新用的；不要把第 5 次访 0 当成命中——0 在第 3 步就被 8 换出去了。

正确选项为什么对： 按组映射加组内 LRU 模拟，命中是第 7 次访 8、第 8 次访 6、第 10 次访 8，一共 3 次，就是 C，选 C。

错误选项为什么错＋举例：

- A（1 次）：可能是把组内路数当成 1（直接映射）算的，或者只数到了某一次命中。教训：2 路组相联每组有 2 个位置，模拟要按组内 2 路走。
- B（2 次）：漏了一次命中，比如第 8 次访 6 没数进去。教训：10 次访问要一次不漏地模拟。
- D（4 次）：多算了一次，比如把第 5 次访 0 当成了命中。教训：第 5 次访 0 时，0 已经被换出去了，是缺失。

笔记本一句话： 第 17 题＋2 路组相联 LRU：按组模拟 10 次访问，命中是第 7、8、10 次，一共 3 次。

【图像】

2 路组相联 LRU：10 步模拟，命中 3 次

组号 ＝ 块号 ÷ 2 取余：0、4、8 → 组 0；2、6 → 组 1

访 0（组0） 缺失，装入 0	访 6（组1） 缺失，装入 6，组 1 满（2、6）
访 4（组0） 缺失，装入 4，组 0 满（0、4）	**访 8（组0） 命中（第 1 次）**
访 8（组0） 缺失，换出最久没用的 0，装入 8	**访 6（组1） 命中（第 2 次）**
访 2（组1） 缺失，装入 2	访 4（组0） 缺失，换出最久没用的 0，装入 4
访 0（组0） 缺失！0 已被换出，换出 4，装入 0	**访 8（组0） 命中（第 3 次）**

命中：第 7 次访 8、第 8 次访 6、第 10 次访 8

一共 3 次 → 选 C

换出的是最久没用的块，不是最新用的

2 路组相联 LRU 十步模拟：命中在第 7、8、10 次，一共 3 次

【定位】 计算机组成原理 · 第3章 存储器层次结构 · 组相联映射与 LRU 替换

【易混对比】

- "组"和"整个 Cache"的区别：一个块只能进它对应的组（组号＝块号÷2 取余），不能随便放进别的组。
- "命中"和"缺失"的区别：要的块在组里就是命中；不在组里就是缺失，要装进来，组满了就按 LRU 换出最久没用的。

按组模拟 10 次访问，命中在第 7、8、10 次，一共 3 次，选 C。

【坑位】

第 5 次访 0 是缺失——0 早在第 3 步被 8 换出去了，别当成命中；换出的是最久没用的，不是最新用的。

A（1 次）可能是把每组路数当成 1（按直接映射）算的，或者只数到了某一次命中——2 路组相联每组有 2 个位置，要按组内 2 路模拟。B（2 次）是漏数了一次命中——10 次访问要一次不漏地模拟，比如第 8 次访 6 别漏。D（4 次）是多算了一次——把第 5 次访 0 当成了命中，0 在第 3 步就被 8 换出去了。

【套路】

- 第一步，分组：组号＝块号÷2 取余，0、4、8 进组 0，2、6 进组 1。
- 第二步，逐个模拟：缺失就装进组，组满了按 LRU 换出最久没用的；命中就记一次，并把该块标成最新用过。
- 第三步，数命中：第 7 次访 8、第 8 次访 6、第 10 次访 8，一共 3 次。

先分组、再一步步模拟、最后数命中，别跳步。

【类比】

像两个抽屉一组、每组放两样东西：满了再放新的，就把最久没动的那样拿出来。第 5 次要 0 的时候，0 早就被 8 换出去了，所以是缺失。术语贴回去：抽屉就是"组"，最久没动的就是"LRU 换出的块"。

【口诀】 块号分组先进组，满了换出最久候；十次访问三命中，组 0 组 1 分开走。

【考频】 「Cache」在 2009–2022 年解析中出现 29 次 · 高频

第 18 题 · 字段直接编码的微指令中互斥微命令字段所需的总位数（正确答案：C）

题干与选项：

某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，

共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控

制字段至少有( )

- A. 5 位
- B. 6 位
- C. 15 位
- D. 33 位

入手点： 一个字段要能区分“类里每个微命令＋什么都不发”这几种情况，位数就是能装下这几种情况的最小二进制位数，5 个字段加起来就是答案。

专有名词从零解释：

- 先想“微命令”：控制 CPU 干一个最小动作的信号，比如“把寄存器 A 的内容送到总线”。
- 再想“互斥微命令”：同一时刻只能发一个的微命令，它们被分到同一个类里；不同类的微命令可以同时发。
- 再想“字段直接编码”：每个互斥类占微指令里的一个字段，字段的值直接表示发哪个微命令；字段里还要留一个值表示“这个类什么都不发”。
- 再想“留空状态”：字段全 0（或者某个特定值）表示这个类不发任何微命令；没有留空状态，字段就没法表示“不发”。

考查知识点：

- 先想每类要多少位：位数 k 要满足 2^k ≥（类里微命令个数＋1 个留空状态）。
- 7 个微命令的类：2^3＝8≥8，要 3 位。
- 3 个微命令的类：2^2＝4≥4，要 2 位。
- 12 个微命令的类：2^3＝8＜13，不够，2^4＝16≥13，要 4 位。
- 5 个微命令的类：2^3＝8≥6，要 3 位。
- 6 个微命令的类：2^3＝8≥7，要 3 位。
- 再想加起来：3＋2＋4＋3＋3＝15 位，就是 C。
- 拿生活例子验算：一个类有 12 种选择、再加一个“不选”，一共 13 种情况，3 位二进制只能表示 8 种、不够，4 位能表示 16 种、够了。
- 边界情况：类里只有 1 个微命令时，加留空一共 2 种情况，要 1 位；类里微命令个数加 1 正好是 2 的幂时，位数就是那个幂次。
- 易混对子：不要把 5 个字段的位数直接按微命令个数加起来——位数是按 2 的幂次算的，不是按个数加的；不要把 12 个微命令的类算成 3 位——3 位只能表示 8 种，13 种情况装不下；不要把不同类的字段合并——直接编码是每类一个字段，位数要分别算再相加。

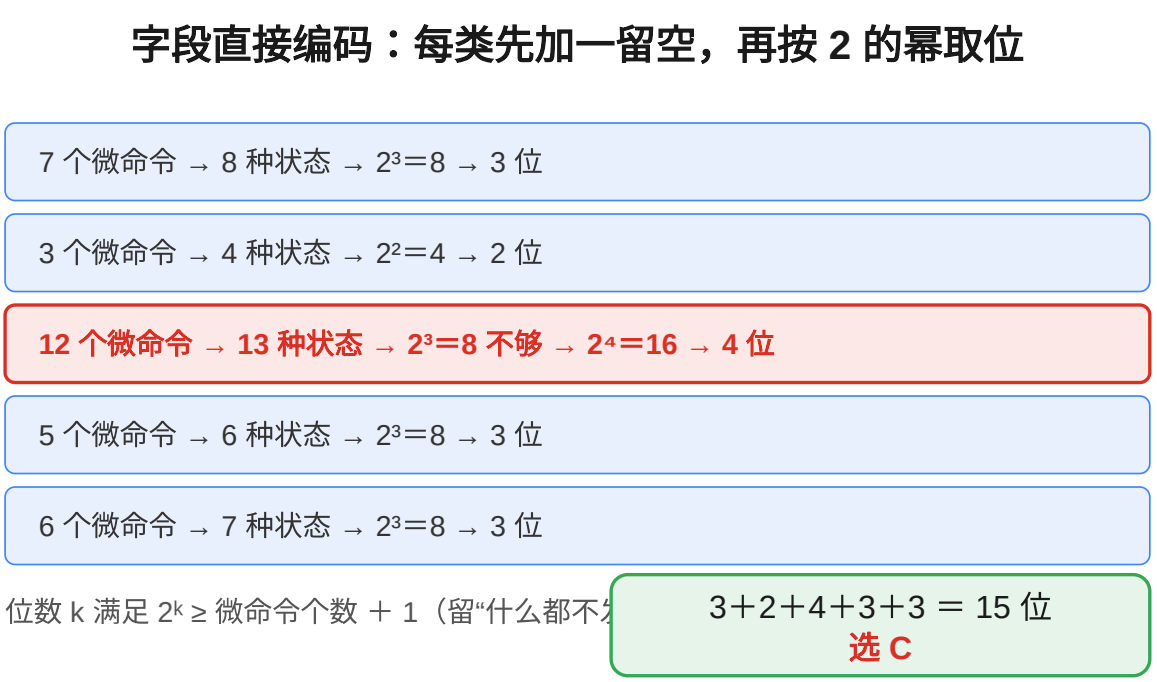
正确选项为什么对： 5 个字段分别要 3、2、4、3、3 位，加起来 15 位，就是 C，选 C。

错误选项为什么错＋举例：

- 位数算少了：把 12 个微命令的类算成 3 位，13 种情况装不下。教训：2^k 要大于等于“微命令个数＋1”，不是大于等于微命令个数。
- 忘了留空状态：字段没有“什么都不发”这个值，类里不发命令时没法表示。教训：每个字段都要留一个值表示不发。
- 把 5 个类当成一个字段算：直接编码是每类一个字段，位数分别算再相加。教训：互斥类分开编码，不能合起来。

笔记本一句话： 第 18 题＋字段直接编码：每类位数满足 2^k≥个数＋1，3＋2＋4＋3＋3＝15 位。

【图像】



【定位】 计算机组成原理 · 第5章 中央处理器 · 微指令字段直接编码

【易混对比】

- "互斥类"和"全部微命令"的区别：同类同时只能发一个、所以同类占一个字段；不同类可同时发，所以字段分开算、位数相加。
- "个数"和"个数＋1"的区别：字段还要留一个"什么都不发"的状态，2 的几次方要大于等于个数＋1，不是大于等于个数。

5 个字段各要 3、2、4、3、3 位，加起来 15 位，选 C。

【坑位】

12 个微命令的类要 4 位不是 3 位（13 种状态 3 位装不下）；每个字段都要留"不发"的空状态。

A（5 位）是把类数当成了位数——5 个类不等于 5 位，位数要按 2 的幂次算。D（33 位）是按"每个微命令占 1 位"的直接编码算的——那是另一种编码方法，不是字段直接编码。最常见的错是把 12 个微命令的类算成 3 位：3 位只能表示 8 种，12＋1＝13 种情况装不下，要 4 位（16 种）。

【套路】

- 第一步，算每类状态数：微命令个数＋1（留一个"不发"）。
- 第二步，算每类位数：找最小的 k 使 2^k ≥ 状态数：7→3，3→2，12→4，5→3，6→3。
- 第三步，相加：3＋2＋4＋3＋3＝15。

每类先加一留空、再按 2 的幂取位、最后相加。

【类比】

像 5 个抽屉各装几样东西、再各留一个空格：12 样东西加空格是 13 格，3 位编号只能编 8 个号、不够，4 位能编 16 个号、才够。术语贴回去：抽屉就是"互斥类"，编号就是"字段编码"，空格就是"不发任何微命令的状态"。

【口诀】 互斥一类一字段，个数加一留空位；二的幂次往上取，五类相加十五位。

【考频】 「微指令」在 2009–2022 年解析中出现 4 次 · 中频

第 19 题 · 总线突发传送方式下传送地址与数据的总时间（正确答案：C）

题干与选项：

设总线频率为 100MHz，宽度为 32 位，地址/数据线复用，每传输一个地址或数据占用

一个时钟周期。若该总线支持突发（猝发）传输方式，则一次“主存写”总线事务传输

128 位数据所需要的时间至少是( )

- A. 20ns
- B. 40ns
- C. 50ns
- D. 80ns

入手点： 突发传送的规矩是“地址只传一次，数据连续传”；地址占 1 个周期，数据按总线宽度分几次传，两个时间加起来。

专有名词从零解释：

- 先想“总线”：CPU、内存、I/O 设备之间传数据的公共通道；一次传多少位，看总线宽度。
- 再想“突发传送”：一次传送一批连续数据，只在开头传一次地址，后面数据一个接一个传，不用每个数据都重新传地址。
- 再想“时钟周期”：总线工作的节拍，本题是 10ns 一拍；传一个数据占 1 拍。

考查知识点：

- 先想地址部分：地址只传一次，占 1 个时钟周期，是 10ns。
- 再想数据部分：数据一共 128 位，总线一次传 32 位，要传 128÷32＝4 次，占 4 个时钟周期，是 40ns。
- 再想总和：10＋40＝50ns，就是 C。
- 拿生活例子验算：寄一箱书，地址只写一次（10 秒写地址），书一共 128 本、一次搬 32 本，要搬 4 趟（40 秒），总共 50 秒。
- 边界情况：数据正好是总线宽度的整数倍时，次数就是相除的结果；数据不是整数倍时，最后一次也要占一个周期（多余的位空着）。
- 易混对子：不要把地址也按 4 次算——突发传送地址只传一次；不要把 128 位当成 128 次——一次传 32 位，不是 1 位；不要只算数据不算地址——地址那 1 个周期也要算。

正确选项为什么对： 地址 10ns＋数据 4×10ns＝50ns，就是 C，选 C。

错误选项为什么错＋举例：

- 只算数据（40ns）：漏了地址那 1 个周期。教训：突发传送也要先传地址。
- 地址也按 4 次算：突发传送地址只传一次，不是每个数据都传地址。教训：突发的意思就是地址只出现一次。
- 128 位当成 128 次：总线一次传 32 位，不是 1 位。教训：先看总线宽度，再算次数。

笔记本一句话： 第 19 题＋突发传送：地址 1 拍＋数据 128÷32＝4 拍，(1＋4)×10ns＝50ns。

【图像】

突发传送：地址只传一次，数据连续传

100MHz → 1 拍 = 10ns；地址/数据复用，传一个占 1 拍



数据 128 位 ÷ 32 位 = 4 次，共 4 拍 = 40ns

突发只在开头传一次地址，后面不用重复传

总时间 = (1+4) × 10ns = 50ns

选 C（漏了地址会掉进 40ns 的坑）

突发传送时间轴：地址 1 拍 10ns＋数据 4 拍 40ns＝50ns

【定位】 计算机组成原理 · 第6章 总线 · 突发传送

【易混对比】

- "突发传送"和"单次传送"的区别：突发是地址只传一次、数据连续传；单次是每个数据都要重新传地址。
- "128 位"和"128 次"的区别：总线一次传 32 位，128 位只要传 4 次，不是 128 次。

地址 1 拍 10ns＋数据 4 拍 40ns＝50ns，选 C。

【坑位】

突发地址只传一次，漏了地址那 1 拍就掉进 40ns 的坑；先算周期 10ns 再乘拍数。

B（40ns）是只算了数据、漏了地址那 1 个周期——突发传送也要先传地址。D（80ns）是把地址也按 4 次算了——突发的意思就是地址只出现一次，每个数据都传地址那是单次传送。还有一种错是把 128 位当成 128 次——先看总线宽度，一次传 32 位。

【套路】

- 第一步，算周期：100MHz，一拍 10ns。
- 第二步，算地址拍数：突发地址只传 1 次，1 拍。
- 第三步，算数据拍数：128÷32＝4 拍。
- 第四步，相加：(1＋4)×10ns＝50ns。

周期先算、地址只记一次、数据按宽度除、最后相加。

【类比】

像寄一箱书：地址只写一次（10 秒写地址），128 本书一次搬 32 本、要搬 4 趟（40 秒），总共 50 秒。术语贴回去：写地址就是"传送地址"，搬书就是"传送数据"，一趟就是"一个时钟周期"。

【口诀】 突发地址只一回，数据除宽得拍数；一拍十纳秒别忘，加起五十选 C。

【考频】 「突发传送」在 2009–2022 年解析中出现 41 次 · 高频

第 20 题 · USB 总线特性的正误判断（正确答案：D）

题干与选项：

下列关于 USB 总线特性的描述中，错误的是( )

- A. 可实现外设的即插即用和热拔插
- B. 可通过级联方式连接多台设备
- C. 是一种通信总线，连接不同外设
- D. 同时可传输 2 位数据，数据传输率高

入手点： 一条一条对：即插即用、热插拔、菊花链、设备数、速度、串行还是并行，哪个说法和 USB 的真面目对不上。

专有名词从零解释：

- 先想“USB”：电脑上最常见的外设总线，鼠标、键盘、U盘都走它。
- 再想“即插即用”：设备插上去，系统自己认出来、自己装好驱动，不用人手动配置。
- 再想“热插拔”：电脑开着机也能插拔设备，不用先关机。
- 再想“菊花链”：设备可以一个接一个串起来，像链子一样扩出去。
- 再想“串行总线”：一次只传 1 位数据；反过来，一次传多位数据的叫并行总线。

考查知识点：

- 先想 USB 的真特点：即插即用、热插拔、菊花链连接、一个控制器最多扩充到 127 个设备、高速模式 480Mbps，这些都是对的。
- 再想错的那一条：D 说“可同时传输 2 位数据”——USB 是串行总线，一次只传 1 位，不可能同时传 2 位，所以 D 是错的，就是答案 D。
- 拿生活例子验算：串行像单车道，一次只能过一辆车；并行像多车道，一次能过好几辆；USB 是单车道，说它一次过 2 辆就是错的。
- 边界情况：USB 3.0 以后速度更快了，但还是串行传数据，一次 1 位这个本质没变。
- 易混对子：不要把“高速 480Mbps”理解成“一次传很多位”——480Mbps 是每秒传 480 兆位，是一位一位快速传，不是同时传多位；不要把菊花链理解成并行——链子是设备一个接一个，不是数据多位并行。

正确选项为什么对： USB 是串行总线，一次只传 1 位，D 说“同时传 2 位”是错的，题目问错的是哪一条，就是 D，选 D。

错误选项为什么错＋举例：

- 选了即插即用那条：即插即用是 USB 的真特点，不是错的。教训：题目问的是“错的是哪一条”，真特点不能选。
- 选了热插拔那条：热插拔也是 USB 的真特点。教训：先分清每条说的是真是假，再按题目问的挑。
- 选了 127 个设备那条：一个 USB 控制器最多带 127 个设备，是对的。教训：数字也要验，127 这个数本身没错。

笔记本一句话： 第 20 题＋USB：串行总线一次只传 1 位，“同时传 2 位”是错的。

【图像】

USB 是串行总线：一次只传 1 位

✓ 可实现外设的即插即用

✓ 支持热拔插（开机也能插拔）

✓ 可级联，一台控制器最多带 127 台设备

✓ 是一种通信总线，连接不同外设

串行 = 单车道

一次只传 1 位

480Mbps 是 1 位 1 位快速传

✗ “同时可传输 2 位数据”—— 对不上

“高速”不等于“并行”，一次传多位的是并行总线

题目问“错误的是”，错的就是 D



【定位】 计算机组成原理 · 第6章 总线 · USB 总线特性

【易混对比】

- "串行"和"并行"的区别：串行一次只传 1 位，并行一次传多位；USB 是串行，"同时传 2 位"对不上。
- "传输率高"和"一次传多位"的区别：480Mbps 是 1 位 1 位快速传，不是同时传多位。

USB 是串行总线、一次只传 1 位，"同时传 2 位"是错的，选 D。

【坑位】

题目问"错误的是"，真特点（即插即用、热拔插、级联）一条都不能选；"高速"不等于"并行"。

A（即插即用）是真特点——设备插上去系统自己认出来、自己装好驱动。B（级联连多台）是真特点——菊花链一个接一个扩，一个控制器最多带 127 个设备。C（通信总线连不同外设）是真特点。D 错在"同时传 2 位"：串行总线一次只传 1 位，USB 3.0 以后速度更快了、但一次 1 位这个本质没变。

【类比】

串行像单车道、一次只能过一辆车，并行像多车道、一次过好几辆；USB 是单车道，说它一次过 2 辆就是错的。术语贴回去：车道就是"数据线位数"，过车速度就是"传输率"。

【口诀】 USB 是串行，一次只传一；即插又热拔，级联连多机；同时传两位，错的就是 D。

【考频】 「USB」在 2009–2022 年解析中出现 5 次 · 中频

第 21 题 · I/O 接口向 CPU 发送命令字、状态字和中断类型时走哪组总线（正确答案：D）

题干与选项：

下列选项中，在 I/O 总线的数据线上传输的信息包括( )

Ⅰ. I/O 接口中的命令字 Ⅱ. I/O 接口中的状态字 Ⅲ. 中断类型号

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅰ、Ⅲ
- C. 仅 Ⅱ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 先看方向：这三种信息都是接口发往 CPU 的；再看三组总线的方向：地址线和控制线是 CPU 发往接口的单向线，接口往 CPU 回送信息只能走数据线。

专有名词从零解释：

- 先想“数据线”：双向的线，CPU 和接口互相传数据都走它。
- 再想“地址线”：CPU 发往接口的单向线，告诉接口“我要找哪个设备、哪个寄存器”。
- 再想“控制线”：CPU 发往接口的单向线，告诉接口“你要干什么”（读、写、复位等等）。
- 再想“命令字、状态字、中断类型”：命令字是 CPU 给接口的命令（其实是 CPU→接口方向）；状态字是接口告诉 CPU“我现在什么状态”；中断类型是接口告诉 CPU“是哪种中断找你”。

考查知识点：

- 先想方向：状态字和中断类型是接口发往 CPU 的；命令字虽然是 CPU 下的命令，但题目问的是一组信息整体走哪组线，关键是接口往 CPU 回送信息的那条路。
- 再想地址线和控制线为什么不行：地址线和控制线都是 CPU 发往接口的单向线，方向反了，接口没法用它们往 CPU 送信息。
- 再想数据线为什么行：数据线是双向的，接口往 CPU 送状态字、中断类型，只能走数据线，就是 D。
- 拿生活例子验算：老师发指令用喇叭（单向），学生回话用对讲机（双向）；学生要回话，只能用对讲机，不能用喇叭。
- 边界情况：CPU 给接口下命令时，走的是控制线加地址线；但接口给 CPU 回信息时，走的是数据线；方向不同，线就不同。
- 易混对子：不要以为“命令”字面上有“命令”就走控制线——题目问的是接口向 CPU 送信息的那条路；不要以为中断类型走控制线——控制线是 CPU 发命令的，不是接口报中断的。

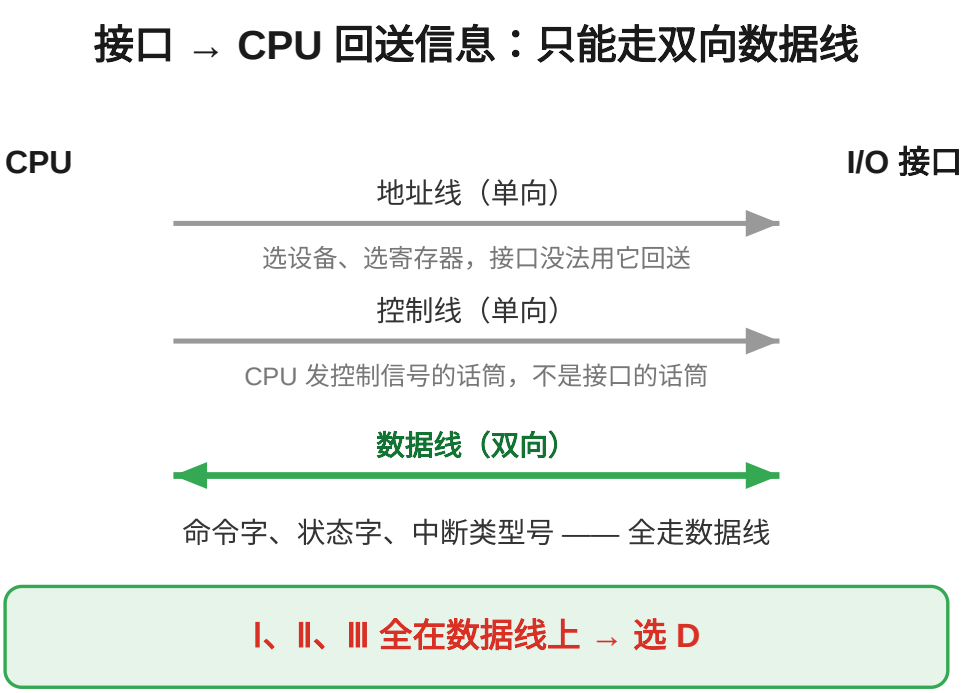
正确选项为什么对： 接口往 CPU 送信息，地址线和控制线方向反了，只有双向的数据线能走，就是 D，选 D。

错误选项为什么错＋举例：

- 选地址线：地址线是 CPU 选设备的单向线，接口不能用它往 CPU 送信息。教训：先看方向，方向反了就走不通。
- 选控制线：控制线是 CPU 发控制信号的单向线，接口不能用它报状态、报中断。教训：控制线是 CPU 的话筒，不是接口的话筒。
- 只想到命令字、没想到状态字和中断类型：题目问的是三种信息整体的路。教训：三种信息里有两种是接口往 CPU 送的，路只能是数据线。

笔记本一句话： 第 21 题＋接口→CPU：地址线、控制线是 CPU→接口的单向线，接口回送信息只能走双向数据线。

【图像】



三组总线方向：地址线、控制线 CPU→接口单向，数据线双向，命令字/状态字/中断类型号都走数据线

【定位】 计算机组成原理 · 第6章 总线 · 总线上传输的信息

【易混对比】

- "数据线"和"地址线、控制线"的区别：数据线是双向的，地址线和控制线是 CPU 发往接口的单向线。
- "命令字"和"控制线"的区别：命令字是 CPU 下给接口的内容、走数据线；控制线是 CPU 发控制信号的线，两回事。

接口往 CPU 回送信息，地址线和控制线方向反了走不通，只有双向的数据线能走， Ⅰ 、 Ⅱ 、Ⅲ全在数据线上，选 D。

【坑位】

先看方向：接口→CPU 的信息只能走双向线；别被"命令"二字骗去选控制线。

地址线是 CPU 选设备的单向线——告诉接口"我要找哪个设备、哪个寄存器"，接口不能用它往 CPU 送信息。控制线是 CPU 发控制信号的单向线——"读、写、复位"是 CPU 的话筒，不是接口的话筒，接口不能用它报状态、报中断。状态字和中断类型号都是接口发往 CPU 的，只能走数据线；命令字是 CPU→接口的内容，同样走数据线。三种信息整体的路，就是数据线。

【套路】

- 第一步，看信息方向：状态字、中断类型是接口→CPU；命令字是 CPU→接口。
- 第二步，看线的方向：地址线、控制线是 CPU→接口的单向线，数据线是双向的。
- 第三步，定线：接口回送信息只能走数据线，三种信息都走数据线。

先看信息方向、再看线的方向，方向对得上才能走。

【类比】

老师发指令用喇叭（单向），学生回话用对讲机（双向）；学生要回话，只能用对讲机，不能用喇叭。术语贴回去：喇叭就是"地址线、控制线"，对讲机就是"数据线"。

【口诀】 地址控制单向走，接口回话没法用；数据双向都能传，命令状态中断号。

【考频】 「数据线」在 2009–2022 年解析中出现 18 次 · 高频

第 22 题 · 响应外部中断时中断隐指令完成的操作组合判断（正确答案：B）

题干与选项：

响应外部中断的过程，中断隐指令完成的操作，除保护断点外，还包括( )

Ⅰ. 关中断 Ⅱ. 保存通用寄存器的内容 Ⅲ. 形成中断服务程序入口地址并送 PC

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅰ、Ⅲ
- C. 仅 Ⅱ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 先分清两拨事：中断隐指令是硬件自动做的三件事，保存通用寄存器是进了服务程序之后、程序自己做的事；两拨事不能混。

专有名词从零解释：

- 先想“外部中断”：外设找 CPU 有事，比如键盘按了一个键、磁盘读完了，CPU 正在干的活要先停一下。
- 再想“中断隐指令”：CPU 响应中断时，硬件自动执行的一条“看不见的指令”——没有操作码、不用取指，硬件直接做完三件事。
- 再想“三件事”：关中断（不让新的中断打断这一步）、保护断点（把下一条指令的地址存起来、好回来接着干）、引出中断服务程序（找到服务程序的入口地址、跳过去）。
- 再想“保存通用寄存器”：把 CPU 里通用寄存器的内容存起来，这是进了服务程序之后、服务程序自己做的第一件事，不属于中断隐指令。

考查知识点：

- 先想 Ⅰ 和Ⅲ： Ⅰ、Ⅲ合起来就是中断隐指令做的事（关中断、保护断点、引出服务程序），就是 B 的组合。
- 再想 Ⅱ 为什么不属于： Ⅱ 是保存通用寄存器，这是服务程序进门后自己做的事，不是硬件自动做的隐指令。
- 再想组合：只含 Ⅰ、Ⅲ的组合是 B，就是答案。
- 拿生活例子验算：电话打进来，硬件自动做的事是“先按住电话不让别的电话插进来、记下刚才说到哪、找到接电话的人”；接电话的人坐下来先把桌上的东西收好，那是他自己做的事，不是电话系统自动做的。
- 边界情况：中断隐指令做完之后，服务程序才开始执行；服务程序的第一件事往往就是保存现场（通用寄存器），但这一步已经不属于隐指令了。
- 易混对子：不要把“保护断点”和“保存通用寄存器”混在一起——断点是下一条指令的地址，通用寄存器是 CPU 里的工作数据，两回事；不要把服务程序做的事都算进隐指令——隐指令只有硬件自动做的三件。

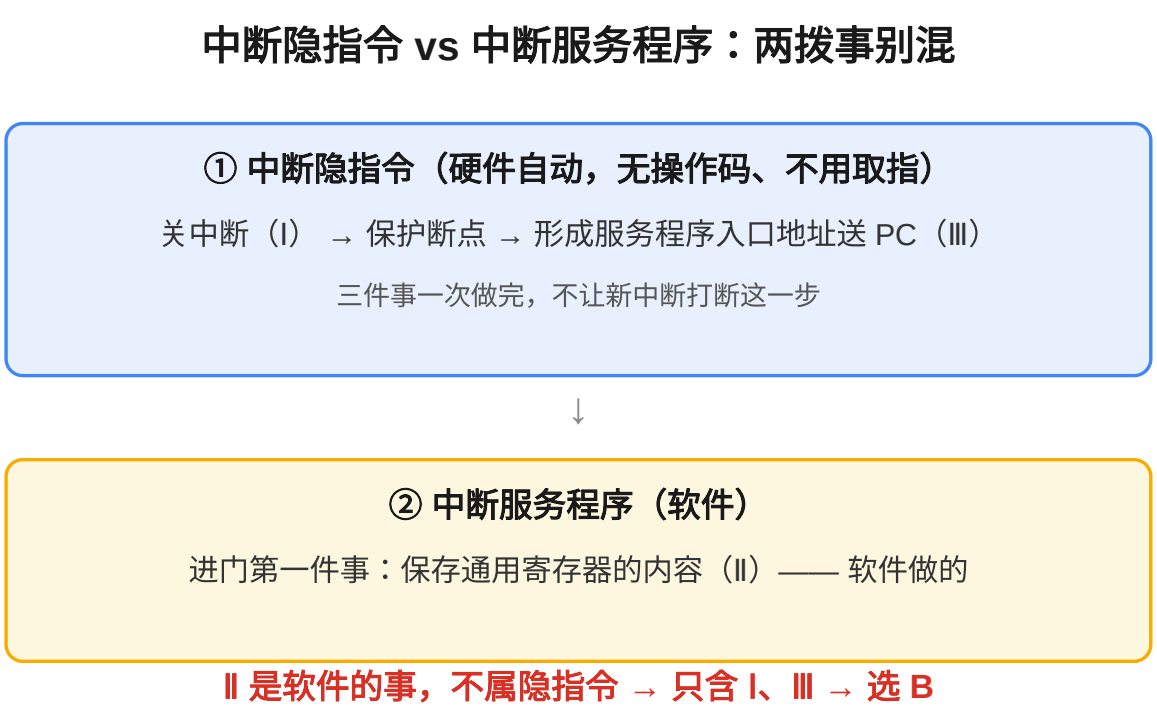
正确选项为什么对： 中断隐指令只做关中断、保护断点、引出服务程序， Ⅰ、Ⅲ就是它，就是 B，选 B。

错误选项为什么错＋举例：

- A（仅 Ⅰ、Ⅱ）：含了 Ⅱ 保存通用寄存器，那是服务程序做的事，不是隐指令。教训：隐指令是硬件自动做的，服务程序做的事不算。
- C（仅 Ⅱ、Ⅲ）：同样含了 Ⅱ。教训：只要组合里有 Ⅱ，就是把服务程序的事算进了隐指令。
- D（ Ⅰ、Ⅱ、Ⅲ）：全选，把 Ⅱ 也算进去了。教训：隐指令只有三件事里的那几项， Ⅱ 不在其中。

笔记本一句话： 第 22 题＋中断隐指令：硬件自动做关中断、保护断点、引出服务程序；保存通用寄存器是服务程序自己的事。

【图像】



【定位】 计算机组成原理 · 第5章 中央处理器 · 中断隐指令

【易混对比】

- "中断隐指令"和"中断服务程序"的区别：隐指令是硬件自动做的三件事（关中断、保护断点、引出服务程序），服务程序是软件做的事（保存通用寄存器是它进门后的第一件事）。
- "保护断点"和"保存通用寄存器"的区别：断点是下一条指令的地址，通用寄存器是 CPU 里的工作数据，两回事。

隐指令只做关中断、保护断点、引出服务程序；保存通用寄存器是服务程序自己的事，只含 I 、III的组合是 B。

【坑位】

组合里只要有"保存通用寄存器"，就是把服务程序的事算进了隐指令，全错。

A（含 II）、C（含 II）、D（含 II）都把保存通用寄存器算进了隐指令——那是进了服务程序之后、程序自己做的第一件事，不是硬件自动做的。隐指令是"看不见的指令"：没有操作码、不用取指，硬件直接做完关中断、保护断点、引出服务程序这三件。服务程序做的事再多，也不属于隐指令。

【套路】

- 第一步，列隐指令三件事：关中断、保护断点、引出服务程序入口送 PC。
- 第二步，认 II 的归属：保存通用寄存器是服务程序做的，不是硬件自动的。
- 第三步，对组合：只含 I 、III的是 B。

先分清硬件自动还是软件做，再对组合，见 II 就排除。

【类比】

电话打进来，电话系统自动做三件事：按住不让别的电话插进来（关中断）、记下刚才说到哪（保护断点）、找到接电话的人（引出服务程序）；接电话的人坐下来先把桌上的东西收好，那是他自己做的事，不是电话系统自动做的。术语贴回去：电话系统就是"中断隐指令（硬件）"，接电话的人就是"中断服务程序（软件）"。

【口诀】 隐指令，硬件干：关中断、护断点、引程序；存寄存器软件事，见 II 排除选 B。

【考频】 「中断隐指令」在 2009–2022 年解析中出现 60 次 · 高频

第 23 题 · 系统调用、外部中断、进程切换、缺页中断“在用户态发生”与“在核心态执行”的区分（正确答案：C）

题干与选项：

下列选项中，不可能在用户态发生的事件是( )

- A. 系统调用
- B. 外部中断
- C. 进程切换
- D. 缺页

入手点： 一个一个看“发生在哪里、执行在哪里”：跨态的（用户态发生、核心态执行）都不算“只在核心态”，只有从头到尾都在核心态的才算。

专有名词从零解释：

- 先想“用户态”：用户程序跑的状态，能做的事受限，不能直接碰硬件。
- 再想“核心态”：操作系统跑的状态，能碰硬件、能做任何事。
- 再想“系统调用”：用户程序请操作系统帮忙（比如读文件），请求在用户态发出，活在核心态干。
- 再想“外部中断”：外设打断 CPU，CPU 正在用户态跑也被打断，转到核心态处理。
- 再想“缺页中断”：用户程序要的数据不在内存里，访问时在用户态触发缺页，到核心态把数据调进来。
- 再想“进程切换”：把 CPU 从一个进程换给另一个进程，由操作系统完成。

考查知识点：

- 先想 A 系统调用：请求在用户态发出，活在核心态干，是跨态的，不是“只在核心态”。
- 再想 B 外部中断：外设可以在用户程序跑的时候到来，打断的是用户态，处理在核心态，也是跨态的。
- 再想 D 缺页中断：用户程序在用户态访问内存时触发缺页，调页的活在核心态干，也是跨态的。
- 再想 C 进程切换：切换这件事本身只由操作系统在核心态完成，用户程序不能自己切换进程，就是 C。
- 拿生活例子验算：用户请前台帮忙（系统调用）、电话打进来（外部中断）、要的文件不在柜子里（缺页），都是“用户这边起头、前台那边干活”；只有换班（进程切换）是前台内部的事。
- 边界情况：进程切换也可能由中断或系统调用引发，但切换这个动作本身只在核心态做。
- 易混对子：不要把“在核心态执行”当成“只在核心态发生”——系统调用和缺页都是在核心态执行，但它们是在用户态触发的；不要把“用户程序不能做”当成“只在核心态发生”的唯一标准——要看发生和执行是不是都在核心态。

正确选项为什么对： 只有进程切换从头到尾都在核心态，就是 C，选 C。

错误选项为什么错＋举例：

- 选系统调用：系统调用是用户态发出请求、核心态执行，跨态了。教训：“在核心态执行”不等于“只在核心态”。
- 选外部中断：外部中断可以在用户态跑的时候到来，也是跨态的。教训：中断到来时 CPU 可能正在用户态。
- 选缺页中断：缺页是在用户态访问内存时触发的，调页在核心态，也是跨态的。教训：触发点在用户态，就不算“只在核心态”。

笔记本一句话： 第 23 题＋只在核心态：系统调用、外部中断、缺页都是用户态触发、核心态执行，只有进程切换只在核心态。

【图像】



四种事件"发生态"与"执行态"对照图：只有进程切换不可能在用户态发生

【定位】 操作系统 · 第1章 操作系统引论 · 用户态与核心态

【易混对比】

- "在用户态发生"和"在核心态执行"的区别：发生看事件被触发的时刻——CPU 是不是可能正跑在用户态；执行看处理它的代码跑在哪个态。一个事件可以用户态发生、核心态执行（跨态），只有从头到尾都在核心态才算"不可能在用户态发生"。
- "进程切换"和"引发切换的原因"的区别：进程切换可以由系统调用、外部中断引发，但切换动作本身（保存现场、恢复新进程的上下文）只由内核在核心态完成；起因跨态，不改变切换动作本身的态。

系统调用用户态发起、外部中断可打断用户态程序、缺页在用户态访存时触发，只有进程切换从头到尾只在核心态，选 C。

【坑位】

把"在核心态执行"直接当成"只在核心态发生"，是这道题最集中的失分点。

A 系统调用：用户程序在用户态执行陷入指令（陷阱）发出请求，"发生"就在用户态；请求真正处理时跑进核心态是"执行"，这是跨态，排除。B 外部中断：外设发出的异步信号可以在 CPU 正在用户态跑程序的时刻到来，把用户态的执行打断，再转入核心态处理——触发的那一刻 CPU 处在用户态，所以能在用户态"发生"，排除。D 缺页：用户程序在用户态执行访存指令、发现页面不在内存时触发缺页中断，触发点在用户态，调页的活在核心态干，跨态，排除。只有 C 进程切换：把 CPU 从一个进程换给另一个进程的切换代码，只能由内核在核心态跑，用户程序自己做不了，从头到尾与用户态执行无关，"不可能在用户态发生"就是它。

【套路】

- 第一步，掐住"发生"的判定标准：事件被触发的那一刻，CPU 是不是可能正跑在用户态。
- 第二步，逐个过：系统调用的陷入指令在用户态执行、外部中断打断的是用户态的执行、缺页在用户态访存时触发，三者都满足"能在用户态发生"。
- 第三步，锁定剩下的：只有进程切换的动作只由内核在核心态完成，对应选项 C。

发生看触发时刻的态，执行看干活代码的态；从头到尾只在核心态的才能选。

【类比】

像公司：员工在工位打电话请前台帮忙（系统调用的陷入指令）、窗外突然响起火警、员工正干活被打断（外部中断）、员工要的文件不在柜子里（缺页），三件事的起因都发生在员工自己的工位上（用户态）；而给员工换座位（进程切换）只能是行政在后台系统里操作，员工自己换不了，整件事只发生在后台（核心态）。术语贴回去：员工在工位干活就是"用户态执行"，前台/行政干活就是"核心态执行"，起因发生在不在工位就是"发生态"。

【口诀】 调用陷入用户发，外中断可断用户态，缺页访存时触发；换进程只在核心态，C 是唯一不可能。

【考频】 「用户态与核心态」在 2009–2022 年解析中出现 49 次 · 高频

第 24 题 · 子程序调用与中断调用在保存现场上的差别（正确答案：B）

题干与选项：

中断处理和子程序调用都需要压栈以保护现场，中断处理一定会保存而子程序调用不需

要保存其内容的是( )

- A. 程序计数器
- B. 程序状态字寄存器
- C. 通用数据寄存器
- D. 通用地址寄存器

入手点： 先想子程序调用保存什么，再想中断调用为什么要多保存一样：子程序是自己安排好的，中断是随机打断的，断点处的状态也得原样留着。

专有名词从零解释：

- 先想“断点”：被打断之后、回来要接着执行的那条指令的地址；存在程序计数器（PC）里。
- 再想“程序状态字（PSW)”：一串状态位，记着运算结果的标志（比如有没有进位、是不是零)、中断屏蔽位（允不允许中断打断）等等；断点处的状态全靠它留着。
- 再想“子程序调用”：程序自己写好的“去办个事再回来”，什么时候去、去哪，都是程序里定好的。
- 再想“中断调用”：外设随机打断程序，什么时候来、来的时候程序跑到哪，全是随机的。

考查知识点：

- 先想子程序调用保存什么：只保存断点，也就是下一条指令的地址（PC)；因为是自己安排的，状态字里的标志位在子程序里用完可以不管，回来接着算不受影响。
- 再想中断调用多保存什么：除了断点，还要保存程序状态字（PSW)；因为中断是随机来的，断点处的标志位、中断屏蔽位必须原样恢复，否则回来之后程序的状态就变了。
- 再想答案：多保存的是程序状态字寄存器，就是 B。
- 拿生活例子验算：自己出门办事，只要记住回家地址（PC）就行；突然被电话打断，除了记住说到哪，还要记住当时的语气、桌上东西的摆放（PSW)，回来才能原样接上。
- 边界情况：通用寄存器也是现场的一部分，但那是进了服务程序之后按需保存的，不是“中断调用比子程序调用多保存”的这道题要的差别。
- 易混对子：不要把“保存现场”等同于“保存所有寄存器”——这道题问的是调用本身多保存的那一样，是 PSW；不要以为子程序调用也要保存 PSW——子程序是自己安排的，状态可以不管，回来不受影响。

正确选项为什么对： 子程序调用只保存断点 PC，中断调用还要多保存 PSW，就是 B，选 B。

错误选项为什么错＋举例：

- A（程序计数器)：PC 是两者都要保存的断点，不是中断多保存的那一样。教训：题目问的是“多保存什么”，共有的不算。
- C（通用数据寄存器)：通用寄存器是服务程序里按需保存的，不是调用本身的差别。教训：分清“调用保存的”和“服务程序保存的”。
- D（通用地址寄存器)：同样不是调用本身多保存的那一样。教训：这道题的差别只有一个，就是 PSW。

笔记本一句话： 第 24 题＋子程序 vs 中断：子程序只保存断点 PC，中断还要多保存 PSW。

【图像】



子程序调用只保存 PC、中断调用多保存 PSW 的现场保护对比图

【定位】 操作系统 · 第1章 操作系统引论 · 中断处理与子程序调用的现场保护

【易混对比】

- "程序计数器（PC）"和"程序状态字（PSW）"的区别：PC 只记断点——回来从哪条指令接着跑；PSW 记断点处的一整套状态——运算标志位、中断屏蔽位等，回来之后状态必须原样。
- "子程序调用"和"中断调用"的区别：子程序是程序自己安排好的"去办个事再回来"，去哪、什么时候去都定好了；中断是随机打断，什么时候来、断在什么状态下，全是未知的。

子程序调用只保存断点 PC，中断调用随机打断、必须多保存 PSW 才能原样恢复，选 B。

【坑位】

题目问的是"中断多保存的那一样"，两者共有的 PC 不算答案。

A 程序计数器是两者都要保存的断点，是共有的，不是差别，选它就掉进了"共有当差别"的坑；C、D 通用寄存器是进了中断服务程序之后按需保存的现场的一部分，不是"调用"这个动作本身带来的差别。正确的差别只有一个：中断是随机打断，断点处的标志位、中断屏蔽位必须原样留着，否则回来之后程序状态就变了，所以中断调用要多保存程序状态字寄存器（PSW），就是 B。子程序调用是自己安排的，状态字里的标志位在子程序里用完可以不管，回来接着算不受影响，所以只保存 PC。

【套路】

- 第一步，看调用时机：自己安排的（子程序）还是随机打断的（中断）。
- 第二步，看断点状态要不要原样：随机打断的，断点处的标志位和屏蔽位必须原样恢复，多保存 PSW；自己安排的，只保存断点 PC。
- 第三步，对选项：多保存的是程序状态字寄存器，对应 B。

先看时机定不定，再看状态要不要原样，多出来的那个就是答案。

【类比】

自己出门去隔壁办事，只要记住回家地址（PC）就行，家里的东西怎么摆不用记；突然被电话打断出门，除了记住说到哪（PC），还要记住当时的语气、桌上东西的摆放（PSW），回来才能原样接上。术语贴回去：回家地址就是"断点 PC"，语气和摆放就是"程序状态字 PSW"。

【口诀】 自调只记断点 PC，中断多记 PSW；随机打断状态留，回来原样不翻车，选 B。

【考频】 「中断与子程序调用」在 2009–2022 年解析中出现 60 次 · 高频

第 25 题 · 虚拟存储器的实现基础与容量决定因素（正确答案：B）

题干与选项：

下列关于虚拟存储器的叙述中，正确的是( )

- A. 虚拟存储只能基于连续分配技术
- B. 虚拟存储只能基于非连续分配技术
- C. 虚拟存储容量只受外存容量的限制
- D. 虚拟存储容量只受内存容量的限制

入手点： 先想虚拟存储器是怎么“变大”的：只把用到的部分调进内存；再想这种“调进调出”要求内存分配必须是哪种方式；最后想容量看的是谁的寻址能力。

专有名词从零解释：

- 先想“虚拟存储器”：让程序觉得内存比实际大得多的技术；程序用的地址是逻辑地址，真正用到的部分才调进物理内存。
- 再想“离散分配”：一个程序占的内存块不要求连在一起，可以东一块、西一块；分页、分段、段页式都是离散分配。
- 再想“连续分配”：一个程序占的内存必须是一整块连着的，中间不能断。
- 再想“寻址范围”：CPU 能发出的地址个数；比如 32 位地址，能发出 2^32 个不同地址，寻址范围就是 4GB。

考查知识点：

- 先想为什么只能是离散分配：虚拟存储器要“只调入用到的部分、用完可以换出去”，这要求程序的内存块可以零散存放、按需调动；连续分配要求一整块连着，没法按需调一页、换一页，所以建不出虚拟存储器。
- 再想离散分配有哪几种：请求分页、请求分段、请求段页式，都是离散分配，都能建虚拟存储器。
- 再想容量由什么决定：虚拟存储器的容量是 CPU 能寻址的范围（地址总位数定的），不是内存条的大小，也不是硬盘的大小。
- 拿生活例子验算：图书馆的书架是离散的（书可以放在不同架子上），借书证上能写的索书号范围就是你能借到的书的范围；书架本身多大、仓库多大，都不决定你能借多少种书。
- 边界情况：内存条再大，虚拟容量也不会超过寻址范围；硬盘再大，虚拟容量也不会跟着变大。
- 易混对子：不要把虚拟容量当成“内存＋外存”——那是物理上能放下的总量，不是虚拟存储器逻辑上的容量；不要以为连续分配也能建虚拟存储器——连续分配没法按需调页，建不出来。

正确选项为什么对： 虚拟存储器只能建在离散分配（请求分页/分段/段页式）上，容量由 CPU 寻址范围决定，就是 B，选 B。

错误选项为什么错＋举例：

- 说容量由内存容量决定：内存条是物理内存，虚拟容量是逻辑上的寻址范围。教训：虚拟和物理要分开，容量看寻址能力。
- 说容量由外存容量决定：外存是换页的后备仓库，不是容量的决定因素。教训：仓库再大，索书号范围不变，能借的种数就不变。
- 说连续分配也能建虚拟存储器：连续分配一整块连着，没法按需调页。教训：虚拟存储器的前提就是离散分配。

笔记本一句话： 第 25 题＋虚拟存储器：只能建在离散分配上，容量由 CPU 寻址范围决定。

【图像】



虚拟存储器容量等于 CPU 寻址范围、与内存和外存大小无关的示意图

【定位】 操作系统 · 第3章 内存管理 · 虚拟存储器的实现基础与容量

【易混对比】

- "连续分配"和"离散分配"的区别：连续分配要求一个程序占一整块连着的内存，没法按需调一页、换一页；离散分配允许程序的内存块东一块、西一块零散存放，才能按需调入调出。
- "虚拟容量"和"内存加外存"的区别：虚拟容量是逻辑上的寻址能力（CPU 能发出多少个不同地址），内存加外存是物理上能放下多少东西；仓库再大，索书号的范围不变。

虚拟存储器只能建在离散分配（请求分页/分段/段页式）上，容量由 CPU 的寻址范围决定，选 B。

【坑位】

容量看的是 CPU 能发出多少个地址，不是内存条有多大、硬盘有多大。

A 说"只能基于连续分配技术"：连续分配一整块连着，没法按需调页、置换页面，建不出虚拟存储器，直接错；C 说"容量只受外存容量限制"：外存只是换页的后备仓库，仓库再大，CPU 发不出的地址照样用不了；D 说"容量只受内存容量限制"：内存条是物理内存，虚拟容量是逻辑寻址范围，内存条翻倍，32 位 CPU 的 4GB 上限一分都不会涨。记住：虚拟和物理要分开，容量看寻址能力。

【套路】

- 第一步，定实现基础：虚拟存储器要按需调页、置换页面，只有离散分配（请求分页/分段/段页式）做得到，连续分配出局。
- 第二步，定容量：容量是逻辑地址能表示的范围，由 CPU 地址总线位数决定，跟内存、外存大小无关。

先看能不能按需调页定基础，再看地址位数定容量。

【类比】

图书馆的书可以放在不同的架子上（离散分配），借书证上能写的索书号范围（寻址范围）决定了你能借到多少种书；书架本身有多大（内存）、仓库有多大（外存），都不改变索书号的范围。术语贴回去：索书号范围就是"CPU寻址范围"，书架就是"物理内存"，仓库就是"外存"。

【口诀】 虚拟要离散，连续建不出；容量看寻址，内存外存靠边站，选 B。

【考频】 「虚拟存储器」在 2009–2022 年解析中出现 4 次 · 中频

第 26 题 · 设备管理软件的层次划分（正确答案：A）

题干与选项：

操作系统的 I/O 子系统通常由四个层次组成，每一层明确定义了与邻近层次的接口。其

合理的层次组织排列顺序是( )

- A. 用户级 I/O 软件、设备无关软件、设备驱动程序、中断处理程序
- B. 用户级 I/O 软件、设备无关软件、中断处理程序、设备驱动程序
- C. 用户级 I/O 软件、设备驱动程序、设备无关软件、中断处理程序
- D. 用户级 I/O 软件、中断处理程序、设备无关软件、设备驱动程序

入手点： 从最贴近硬件的那层往上数：先是接住中断的，再是直接摆弄设备的，再是给上层统一接口的，最后是应用程序用的。

专有名词从零解释：

- 先想“中断处理程序”：硬件一打断，第一个跑起来的程序；先把中断接住、认清楚是哪种中断。
- 再想“设备驱动程序”：直接和具体设备打交道的程序，知道这个设备的寄存器怎么摆弄、命令怎么写。
- 再想“与设备无关的系统调用处理层”：不管底下是什么设备，都给上层同一个接口；做设备无关的事，比如给设备起统一的名字、做缓冲、做错误处理。
- 再想“用户层”：应用程序直接用的那层，比如 C 语言库里的读写函数。

考查知识点：

- 先想四层从下到上：最底层是中断处理程序，第二层是设备驱动程序，第三层是与设备无关的系统调用处理层，最上层是用户层，就是 A。
- 再想为什么这样排：中断是最贴近硬件的，当然最底层；驱动要摆弄具体设备，贴着中断层；无关层要屏蔽设备差别，坐在驱动上面；用户层离硬件最远，坐在最上面。
- 再想每层干什么：中断层接中断，驱动层摆弄设备，无关层统一接口、做缓冲和错误处理，用户层给应用程序用。
- 拿生活例子验算：饭店里，传菜员最贴近厨房（中断层），厨师做具体菜（驱动层），领班不管哪道菜都按一个流程上菜（无关层），客人只管点菜（用户层）。
- 边界情况：有的小系统会把某两层合并，但标准划分就是这四层，顺序不能乱。
- 易混对子：不要把驱动层和无关层上下颠倒——驱动是贴设备的，无关层是屏蔽差别的，无关层在驱动上面；不要把中断处理程序放到上面——中断是最底层、最先跑的。

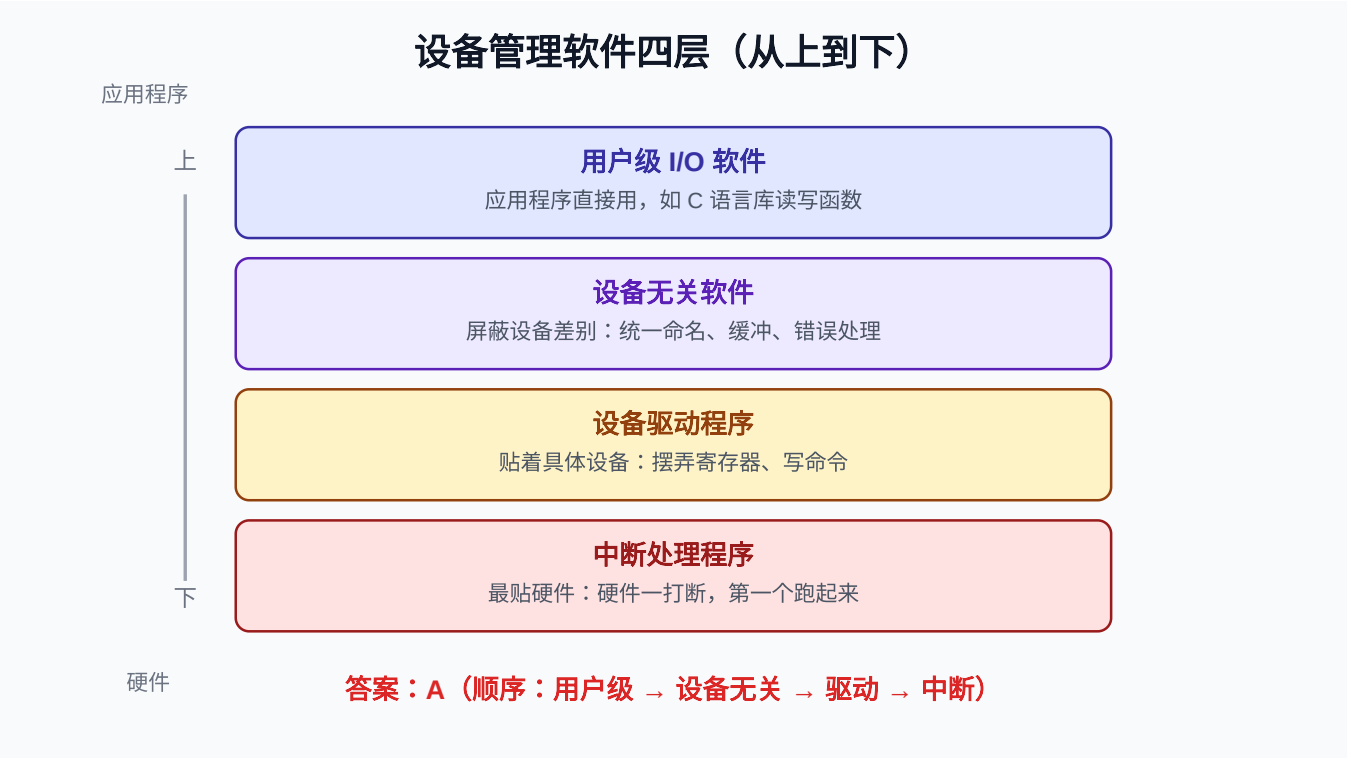
正确选项为什么对： 从下到上是中断处理程序、设备驱动程序、与设备无关的系统调用处理层、用户层，就是 A，选 A。

错误选项为什么错＋举例：

- 把驱动层放到无关层上面：驱动是贴具体设备的，无关层是屏蔽差别的，上下不能反。教训：越贴硬件越在下，越通用越在上。
- 把中断处理程序放到上面：中断是最先跑的、最贴硬件的，只能在最底层。教训：中断来了谁先接住，谁就是最底层。
- 层数数错（少一层、多一层）：标准划分就是四层。教训：用户层、无关层、驱动层、中断层，一层不能少。

笔记本一句话： 第 26 题＋设备管理四层：从下到上是中断、驱动、无关层、用户层。

【图像】



设备管理软件从下到上四层结构图

【定位】 操作系统 · 第5章 输入/输出（I/O）管理 · 设备管理软件的层次划分

【易混对比】

- "设备驱动程序"和"设备无关软件"的区别：驱动是贴着具体设备的，知道这个设备的寄存器怎么摆弄；设备无关软件是屏蔽设备差别的，给上层统一接口、做缓冲和错误处理，坐在驱动上面。
- "中断处理程序"和"设备驱动程序"的区别：中断处理程序是硬件一打断第一个跑起来的，最贴近硬件；驱动是被调用去摆弄设备的。一个是"接住中断"，一个是"操作设备"。

从下到上依次是中断处理程序、设备驱动程序、设备无关软件、用户级 I/O 软件，选项 A 的顺序就是这个，选 A。

【坑位】

越贴硬件越在下，越通用越在上；驱动和无关层的上下最容易颠倒。

B 把中断处理程序放到了第三层：中断是最先跑的、最贴硬件的，只能在最底层，不可能坐在驱动上面；C 把设备驱动程序放到了第二层（从上往下数）：驱动是贴具体设备的，无关层是屏蔽差别的，无关层必须在驱动上面，两者不能反；D 把中断处理程序放到了第二层：同样犯了"中断不在最底层"的错。记住口诀里的顺序：中断最底接信号，驱动贴着摆设备，无关统一接口，用户最高点单。

【套路】

- 第一步，找最贴硬件的：中断来了谁先接住谁就是最底层——中断处理程序。
- 第二步，贴着它往上排：摆弄具体设备的是驱动，再往上是屏蔽差别的设备无关软件，最上面是应用程序用的用户级 I/O 软件。
- 第三步，对选项：从上到下"用户级、设备无关、驱动、中断"对应 A。

先定最底层，再贴着往上排，最后和选项对口径。

【类比】

像饭店：传菜员最贴近厨房、第一个接住上菜的信号（中断处理程序）；厨师按具体菜式下锅（设备驱动程序）；领班不管哪道菜都按一个流程上菜、对客人一个说法（设备无关软件）；客人只管点菜（用户级 I/O 软件）。术语贴回去：厨房就是"硬件设备"，点菜就是"应用程序的 I/O 请求"。

【口诀】 中断底层先接住，驱动二层摆设备；无关三层统接口，用户顶层来点单，A 序别颠倒。

【考频】 「设备管理软件层次」在 2009–2022 年解析中出现 17 次 · 高频

第 27 题 · 用银行家算法思想判断安全序列的起始进程（正确答案：D）

题干与选项：

假设 5 个进程 P0、P1、P2、P3、P4 共享三类资源 R1、R2、R3，这些资源总数分别为

18、6、22。T0 时刻的资源分配情况如下表所示，此时存在的一个安全序列是( )

已分配资源 资源最大需求

进程

R1 R2 R3 R1 R2 R3

P0 3 2 3 5 5 10

P1 4 0 3 5 3 6

P2 4 0 5 4 0 11

P3 2 0 4 4 2 5

P4 3 1 4 4 2 4

A P0，P2，P4，P1，P3 B P1，P0，P3，P4，P2

C P2，P1，P0，P3，P4 D P3，P4，P2，P1，P0

入手点： 先看初始 Available 下谁能跑，再看跑完一个之后 Available 变成多少、剩下的谁能跑；开头跑错了，后面就卡住了。

专有名词从零解释：

- 先想“Need”：一个进程还差多少资源才能跑完；Need＝最大需求减去已经分到的。
- 再想“Available”：系统现在手里还剩多少资源，能分给进程。
- 再想“安全序列”：一个进程排队顺序，按这个顺序跑，每个进程轮到时手里的资源都够它跑完，跑完它还把资源还回来，后面就越跑越顺。
- 再想“银行家算法”：操作系统放贷的规矩——只有按某个安全顺序能让所有进程都跑完，才敢把资源借出去。

考查知识点：

- 先想初始数据：Need 是 P0（2,3,7）、P1（1,3,3）、P2（0,0,6）、P3（2,2,1）、P4（1,1,0）；Available 是（2,3,3）。
- 再想谁能第一个跑：P0 差第 3 类资源 7＞3，跑不起来；P2 差第 3 类资源 6＞3，也跑不起来；所以以它俩开头的选项直接出局。
- 再想 P1 开头和 P3 开头两条路：先跑 P1，Available 变成（6,3,6），这时 P0 还是跑不起来（第 3 类 7＞6），这条路走不通，排除 B。
- 再想先跑 P3：Available 变成（4,3,7），剩下的 P4（1,1,0）、P2（0,0,6）、P1（1,3,3）、P0（2,3,7）全都够了；按 P3、P4、P2、P1、P0 的顺序走下去，每个进程跑完都还资源，后面只会越跑越顺，全程安全，就是 D。
- 拿生活例子验算：手里有 2 张 3 类券，P0 要 7 张第 3 类券、P2 要 6 张，都不够，只能先让 P1 或 P3 跑；P3 跑完还回来，手里变成 4、3、7，剩下的谁都够了。

- 边界情况：一个进程跑完归还资源后，Available 只会变大不会变小，所以只要某一步之后所有剩下的 Need 都够，后面任何顺序都安全。
- 易混对子：不要只看第一个进程够不够——第一个够了，后面卡住也不算安全序列；不要把 Need 和最大需求搞混——Need 是还差多少，不是总共要多少。

正确选项为什么对： 先 P3 跑完 Available 变成（4,3,7），剩下的 Need 全够，按 P3、P4、P2、P1、P0 走完全程安全，就是 D，选 D。

错误选项为什么错＋举例：

- 以 P0 或 P2 开头的选项：初始 Available（2,3,3）下，P0 差第 3 类 7＞3，P2 差第 3 类 6＞3，第一个就跑不起来。教训：安全序列的第一个进程必须在初始 Available 下就能跑。
- 以 P1 开头的选项：先跑 P1 之后 Available 是（6,3,6），P0 还是跑不起来（7＞6），后面卡住了。教训：第一个能跑不等于整条路都通，要一路验到最后。
- 只看 Need 最小的进程开头：Need 最小的 P4（1,1,0）开头不一定能带出整条安全路，关键是跑完之后 Available 能不能带起剩下的。教训：安全序列要整条验，不能只看开头。

笔记本一句话： 第 27 题＋银行家安全序列：先看初始谁能跑，再看跑完 Available 够不够带起剩下，P3、P4、P2、P1、P0 安全。

【图像】



银行家算法按 P3、P4、P2、P1、P0 顺序验证安全序列的过程图

【定位】 操作系统 · 第2章 进程管理 · 死锁避免、银行家算法安全序列

【易混对比】

- "Need"和"最大需求"的区别：最大需求是进程总共要多少，Need 是还差多少才能跑完（最大需求减去已分配）；判断能不能跑，看的是 Need 不是最大需求。

- "安全序列"和"随便一个能跑的顺序"的区别：安全序列要求按这个顺序每个进程轮到时资源都够、跑完归还资源，一路验到最后；只看第一个能跑不算数。

初始 Available 是 (2,3,3)：P0、P2 首步就卡，P1 起步第二步卡，只有 P3 起步能带出 P3、P4、P2、P1、P0 的全程安全路，选 D。

【坑位】

第一个能跑不等于整条路都通，安全序列要一路验到最后。

先算 Need：P0(2,3,7)、P1(1,3,3)、P2(0,0,6)、P3(2,2,1)、P4(1,1,0)；初始 Available(2,3,3)（总数减已分配：R1 18−16、R2 6−3、R3 22−19）。以 P0 开头的 A：P0 差第 3 类 7＞3，首步就跑不起来；以 P2 开头的 C：P2 差第 3 类 6＞3，同样首步卡死。以 P1 开头的 B：P1 跑完 Available 变成 (6,3,6)，此时 P0 还是差第 3 类 7＞6，后面卡住了。以 P3 开头的 D：P3 跑完归还，Available 变成 (4,3,7)；P4(1,1,0)够→(7,4,11)；P2(0,0,6)够→(11,4,16)；P1(1,3,3)够→(15,4,19)；P0(2,3,7)够→跑完，全程安全。错选 B 的，一般是只验了第一步、没往后验；错选 A/C 的，是把 Need 和最大需求搞混、或者没算 Available。

【套路】

- 第一步，算 Need 和 Available：Need＝最大需求−已分配；Available＝资源总数−已分配总数。
- 第二步，筛首发：在初始 Available 下 Need 全够的才能开头，P0、P2 首步就卡，直接排除 A、C。
- 第三步，试跑并更新：P1 起步跑完 Available 变成 (6,3,6)，P0 仍不够，B 走不通；P3 起步跑完变成 (4,3,7)，剩下的 P4、P2、P1、P0 逐个都够。
- 第四步，对选项：能走完全程的是 P3、P4、P2、P1、P0，对应 D。

算 Need、筛首发、试跑更新一路验，能走完全程的才是安全序列。

【类比】

像银行放贷：手里有 2、3、3 三种券（Available），每个客户还差多少券（Need）写得清清楚楚；先借给还差得少、还得起的 P3，他还回来之后手里的券变多了，后面的客户就越借越顺。术语贴回去：券就是"资源"，还回来就是"进程结束归还资源"，借得出去的排队顺序就是"安全序列"。

【口诀】 先算还差多少，再看手里多少；首发能跑不算数，一路跑通才安全，P3 开头选 D。

【考频】 「银行家算法」在 2009–2022 年解析中出现 8 次 · 中频

第 28 题 · 缺页中断、read 系统调用陷阱以及 open 与 read 参数的正误判断（正确答案：A）

题干与选项：

若一个用户进程通过 read 系统调用读取一个磁盘文件中的数据，则下列关于此过程的

叙述中，正确的是( )

Ⅰ.若该文件的数据不在内存，则该进程进入睡眠等待状态

Ⅱ.请求 read 系统调用会导致 CPU 从用户态切换到核心态

III.read 系统调用的参数应包含文件的名称

- A. 仅Ⅰ、Ⅱ
- B. 仅 Ⅰ、III
- C. 仅 Ⅱ、III
- D. Ⅰ、Ⅱ和III

入手点： 一句一句验：缺页时进程会怎么样、read 是不是要陷入核心态、open 和 read 各自要传什么参数。

专有名词从零解释：

- 先想“缺页中断”：程序要的数据不在内存里，访问时触发的中断；操作系统把数据从外存调进来，程序才能接着跑。
- 再想“系统调用陷阱”：用户程序请操作系统帮忙时，从用户态陷入核心态的那个入口；read 就是一种系统调用。
- 再想“open”：打开文件，告诉系统“按路径名和文件名找到这个文件”，系统返回一个文件描述符（一个整数编号）。
- 再想“read”：读文件，不用再给路径名，只要给文件描述符、缓冲区首地址、要读的字节数。

考查知识点：

- 先想第Ⅰ句（对）：缺页时，数据不在内存，进程要阻塞起来等；操作系统把数据调进来之后，再把进程唤醒接着跑。
- 再想第Ⅱ句（对）：read 是系统调用，用户程序在用户态调用它，要通过陷阱陷入核心态，由操作系统在核心态完成读操作。
- 再想第III句（对）：open 的参数里有路径名和文件名（要告诉系统找哪个文件）；read 的参数里没有路径名，只有 open 返回的文件描述符、缓冲区首地址、要读的字节数 n。
- 再想结论：三句都对，就是 A。
- 拿生活例子验算：借书时，open 是告诉管理员书名、拿到借书号；read 是拿着借书号、说放到哪个桌子、读多少页；书不在架子上时（缺页），先等管理员从仓库取来，再接着读。
- 边界情况：文件已经打开过，read 还是要文件描述符，不能直接给路径名；缺页不只发生在读文件，访问任何不在内存的数据都会缺页。
- 易混对子：不要以为 read 也要传路径名——路径名只在 open 时用一次，read 认的是描述符；不要以为缺页时进程继续跑——数据没调进来之前，进程只能阻塞等着；不要把系统调用当成普通函数调用——普通函数不换态，系统调用要陷入核心态。

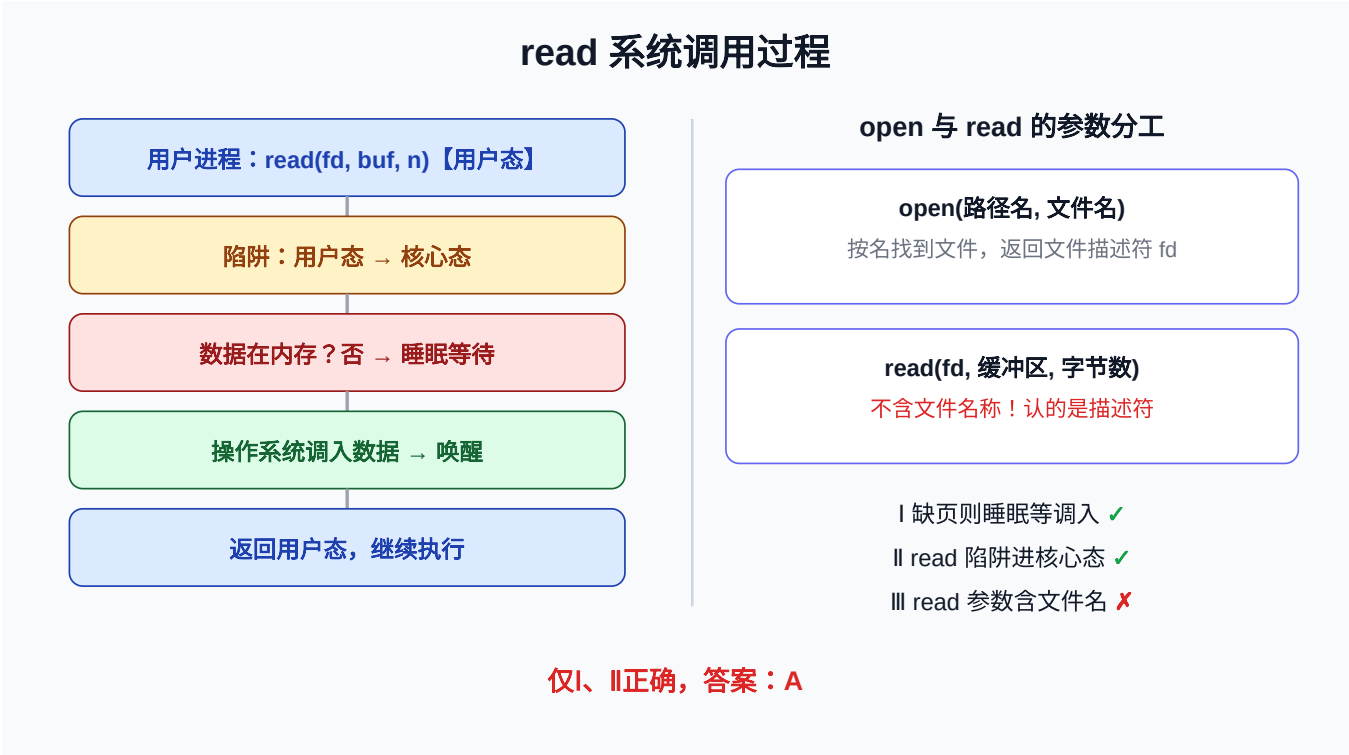
正确选项为什么对： 三句都对（缺页阻塞调入后唤醒、read 要陷入核心态、open 传路径名 read 传描述符），就是 A，选 A。

错误选项为什么错＋举例：

- 漏了其中一句的组合：三句都是对的，凡是漏掉其中一句的组合都不对。教训：一句一句验，不能凭印象凑。
- 以为 read 也要传路径名：路径名只在 open 时用，read 用描述符。教训：open 和 read 的参数分工要分清。
- 以为缺页不阻塞：数据没进来之前，进程只能等着。教训：缺页中断的处理就是阻塞、调入、唤醒。

笔记本一句话： 第 28 题＋缺页 read open：缺页阻塞调入后唤醒，read 陷入核心态，open 传路径名、read 传描述符。

【图像】



用户进程 read 系统调用陷入核心态、缺页时睡眠等待的流程图

【定位】 操作系统 · 第1章 操作系统引论（系统调用）/ 第4章 文件管理 · read 系统调用过程

【易混对比】

- "open 的参数"和"read 的参数"的区别：open 要告诉系统找哪个文件，参数里有路径名和文件名，返回一个文件描述符；read 不再给路径名，认的是 open 返回的文件描述符，外加缓冲区首地址和要读的字节数。
- "系统调用"和"普通函数调用"的区别：普通函数调用不换态；系统调用要通过陷阱从用户态陷入核心态，由操作系统在核心态完成。

Ⅰ对（数据不在内存则睡眠等调入）、Ⅱ对（read 是系统调用，要陷入核心态）、Ⅲ错（read 的参数是文件描述符不是文件名称），"仅Ⅰ、Ⅱ"选 A。

【坑位】

路径名只在 open 时用一次，read 认的是描述符，别把 open 的参数安到 read 头上。

Ⅰ：文件数据不在内存，进程确实要进入睡眠等待状态，等操作系统把数据调进来再唤醒，正确；Ⅱ：read 是系统调用，用户程序在用户态调用它，必须通过陷阱切换到核心态执行，正确；Ⅲ：read 系统调用的参数是文件描

述符、缓冲区首地址、读取字节数，不包含文件名称——文件名称是 open 的参数，open 按名找到文件后返回描述符，read 拿描述符办事。所以III错，组合"仅 Ⅰ、Ⅱ"就是 A。错选 D 的，是以为 read 也要传文件名；错选 B 的，是漏验了 Ⅱ。

【套路】

- 第一步，逐句验：缺页→睡眠等调入（Ⅰ对）；read 是系统调用→陷阱进核心态（Ⅱ对）；read 参数→描述符不是文件名（III错）。
- 第二步，组答案：对的是 Ⅰ、Ⅱ，对应"仅 Ⅰ、Ⅱ"，选 A。

一句一句验，参数归属别张冠李戴。

【类比】

去图书馆借书：open 是告诉管理员书名、拿到借书号（文件描述符）；read 是拿着借书号、说书放到哪个桌子（缓冲区）、读多少页；书不在架子上（缺页），就先在旁边等着（睡眠），等管理员从仓库取来再读。术语贴回去：借书号就是"文件描述符"，书名就是"文件名称"，仓库取书就是"调页"。

【口诀】 open 报名拿号，read 拿号办事；缺页先睡等，read 必进核；仅一二选 A。

【考频】 「read 系统调用」在 2009–2022 年解析中出现 26 次 · 高频

第 29 题 · 两个作业 CPU 与 I/O 交错运行时正确的甘特图判断（正确答案：B）

题干与选项：

一个多道批处理系统中仅有 P1 和 P2 两个作业，P2 比 P1 晚 5ms 到达，它们的计算和

I/O 操作顺序如下：

P1：计算 60ms，I/O80ms，计算 20ms

P2：计算 120ms，I/O40ms，计算 40ms

若不考虑调度和切换时间，则完成两个作业需要的时间最少是( )

- A. 240ms
- B. 260ms
- C. 340ms
- D. 360ms

> 注：甘特图以原卷为准（部分核对）

入手点： 画时间线：单 CPU 一次只能给一个作业，I/O 和 CPU 可以并行；P1 先到先占 CPU，P1 去做 I/O 时 CPU 空出来给 P2，P1 的 I/O 做完了想回 CPU 还要等 P2 让出来。

专有名词从零解释：

- 先想“甘特图”：横轴是时间、纵轴是作业和设备的图；哪段时间谁在用 CPU、谁在做 I/O，一眼就能看出来。
- 再想“CPU 段、I/O 段”：一个作业干活是 CPU 一段、I/O 一段、CPU 一段交替着来；CPU 段要占 CPU，I/O 段占 I/O 设备，不占 CPU。
- 再想“单 CPU 竞争”：只有一个 CPU，两个作业的 CPU 段不能重叠，谁先到谁先占，后到的要等。

考查知识点：

- 先想两个作业的时长段：P1 是计算 60ms、I/O 80ms、计算 20ms；P2 是计算 120ms、I/O 40ms、计算 40ms；P2 比 P1 晚 5ms 到达。
- 再想时间线（从 0 开始）：
- 0–60：P1 先到，先占 CPU，算第一段 60ms。
- 60–140：P1 去做 I/O（80ms）；CPU 空出来，P2（5ms 就到了、一直在等）上 CPU，算第一段。
- 140 时 P1 的 I/O 做完了，想回 CPU 算第二段，但 CPU 正被 P2 占着，只能等。
- 60–180：P2 的第一段 CPU（120ms）从 60 算到 180。
- 180–200：P2 第一段算完，去做 I/O；CPU 空出来，P1 回来算第二段 20ms（180–200）。
- 180–220：P2 做 I/O（40ms），和 P1 的第二段 CPU 并行。
- 220–260：P2 的 I/O 做完了，回 CPU 算第二段 40ms（220–260）。
- 终点是 260ms，就是 B。
- 拿生活例子验算：一个人先上灶（P1），炒完菜去洗菜（I/O），灶空出来给第二个人（P2）；第一个人洗完菜回来，灶还被第二个人占着，只能等第二个人下灶。
- 边界情况：P2 晚到 5ms，不影响大局（P1 0–60 一直占着 CPU，P2 到了也只能等）；如果 P1 的 I/O 比 P2 的第一段 CPU 短，P1 就不用等那么久。

- 易混对子：不要把 I/O 和 CPU 当成串行的——P1 做 I/O 的时候，P2 可以同时用 CPU；不要漏了 P1 第二段等 CPU 的 40ms（140–180）——P1 的 I/O 做完了不等于马上能算；不要把 P2 的 I/O 和 CPU 段顺序弄反——P2 是先算 120ms，再 I/O 40ms，再算 40ms。

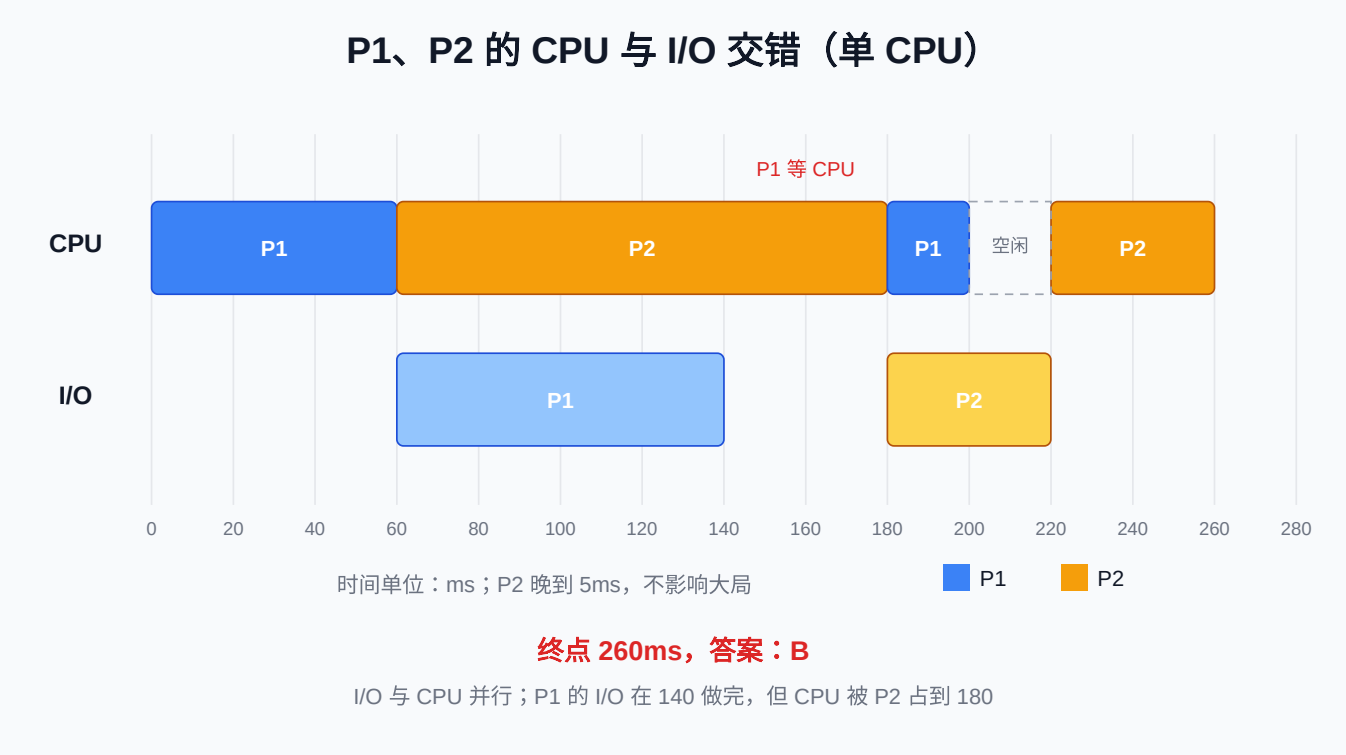
正确选项为什么对： 按单 CPU 竞争画时间线，终点是 260ms，P1 第二段在 180–200，P2 第二段在 220–260，就是 B，选 B。

错误选项为什么错＋举例：

- 漏了 P1 等 CPU 的时间：P1 的 I/O 在 140 做完，但 CPU 被 P2 占到 180，P1 第二段只能从 180 开始。教训：I/O 做完不等于马上能算，要看 CPU 空不空。
- 把 I/O 和 CPU 算成串行的：P1 做 I/O 的 60–140 和 P2 用 CPU 的 60–180 是并行的，不能加到一起。教训：I/O 设备和 CPU 是两样东西，可以同时干活。
- 终点算错：终点是 P2 第二段算完的 260ms，不是 P1 结束的 200ms。教训：两个作业都结束才算完，最后结束的是 P2。

笔记本一句话： 第 29 题＋两作业甘特图：单 CPU 一次一个，I/O 和 CPU 并行，P1 第二段等到 180–200，终点 260ms。

【图像】



P1 与 P2 的 CPU 段和 I/O 段交错执行的甘特图

【定位】 操作系统 · 第2章 进程管理 · 多道批处理系统中作业的 CPU 与 I/O 交错（甘特图）

【易混对比】

- "CPU 段"和"I/O 段"的区别：CPU 段占 CPU，I/O 段占 I/O 设备不占 CPU；单 CPU 一次只能给一个作业的 CPU 段，但 I/O 段可以和另一个作业的 CPU 段并行。
- "I/O 做完"和"能用 CPU"的区别：I/O 做完了只是万事俱备，CPU 还被别人占着就得等；做完不等于马上能算。

单 CPU 一次一个，I/O 与 CPU 并行画时间线：P1 第二段要等 P2 让出 CPU（180–200），P2 最后一段 220–260，终点 260ms，选 B。

【坑位】

P1 的 I/O 在 140 做完，但 CPU 被 P2 占到 180，这 40ms 的等待最容易漏。

时间线（P2 晚到 5ms，不影响大局）：0–60 P1 占 CPU 算第一段；60–140 P1 做 I/O，CPU 空出来给 P2 算第一段（120ms，从 60 到 180）；140 时 P1 的 I/O 做完了，但 CPU 还被 P2 占着，P1 只能等；180 时 P2 第一段算完去做 I/O（40ms，到 220），CPU 空出来，P1 回来算第二段 20ms（180–200）；220 时 P2 的 I/O 做完，回 CPU 算第二段 40ms（220–260）。终点是 260ms，就是 B。错选 A（240ms）的，一般是漏了 P1 等 CPU 的 40ms，或者把 I/O 和 CPU 当成串行相加；错选 C/D 的，是把并行段重复计算了。

【套路】

- 第一步，列段表：P1 计算 60、I/O 80、计算 20；P2 计算 120、I/O 40、计算 40；P2 晚到 5ms。
- 第二步，画时间线：单 CPU 一次只排一个 CPU 段，I/O 段和别人的 CPU 段并行；谁的 I/O 先做完、CPU 还没空，谁就等着。
- 第三步，找终点：两个作业都结束才算完，最后结束的是 P2（260ms），对应 B。

列段表、画时间线找等待、终点看最后结束的那个。

【类比】

两个人共用一个灶（单 CPU）：P1 先上灶炒 60 分钟，炒完去洗菜（I/O）80 分钟，灶空出来给 P2 炒 120 分钟；P1 洗完菜回来，灶还被 P2 占着，只能等 P2 下灶；P2 去洗菜时 P1 赶紧回灶炒最后 20 分钟；P2 洗完菜再上灶炒 40 分钟收尾。术语贴回去：灶就是"CPU"，洗菜就是"I/O 操作"，等灶就是"等待 CPU"。

【口诀】 单核一次管一段，I/O 并行别串算；做完要等 CPU 空，终点看谁最后完，260 选 B。

【考频】 「甘特图」在 2009–2022 年解析中出现 2 次 · 低频

第 30 题 · 哪些情形会引起处理机调度、哪种情形不引起处理机调度（正确答案：C）

题干与选项：

若某单处理器多进程系统中有多个就绪态进程，则下列关于处理机调度的叙述中错误的

是( )

- A. 在进程结束时能进行处理机调度
- B. 创建新进程后能进行处理机调度
- C. 在进程处于临界区时不能进行处理机调度
- D. 在系统调用完成并返回用户态时能进行处理机调度

入手点： 先想调度是怎么被触发的：时间片用完、I/O 完成、进程阻塞、新进程到达，这些都是触发事件；再看临界区是不是触发事件——它不是。

专有名词从零解释：

- 先想“处理机调度”：操作系统决定下一个时间片把 CPU 给哪个进程。
- 再想“触发事件”：让操作系统停下来、重新想想 CPU 给谁的那些事，比如时间片用完了、等 I/O 的进程的 I/O 做完了、跑着的进程自己阻塞了。
- 再想“临界区”：一段一次只允许一个进程执行的代码，里面访问的是共享资源；进程在临界区里正常执行，不会因为“在临界区里”就触发调度。

考查知识点：

- 先想临界区为什么不触发调度：进程在临界区里只是在正常执行代码，执行本身不会让操作系统重新调度；临界区要的是互斥（一次只进一个），互斥靠信号量、锁来保证，不是靠调度来保证。
- 再想哪些会触发调度：时间片用完（该换人了）、I/O 完成（等 I/O 的进程可以跑了）、进程阻塞（跑着的进程自己等住了，CPU 得换人），这些都是调度点。
- 再想答案：不引起处理机调度的是“进程处于临界区”，就是 C。
- 拿生活例子验算：一个人正在单人间里换衣服（临界区），换衣服本身不会让管理员换人；只有到钟了（时间片用完）、等的东西到了（I/O 完成）、这个人自己说要等（阻塞），管理员才会换人。
- 边界情况：进程在临界区里时间片用完了，还是会调度——但那是时间片用完触发的，不是临界区触发的；两件事要分开。
- 易混对子：不要把“临界区需要互斥”理解成“进了临界区就调度”——互斥是锁的事，调度是时间片和事件的事；不要把“进程在临界区里不能被打断”当成真话——时间片用完照样可以打断，临界区不等于关中断。

正确选项为什么对： 临界区本身不是调度触发事件，时间片用完、I/O 完成、阻塞等才是，就是 C，选 C。

错误选项为什么错＋举例：

- 选了时间片用完那条：时间片用完是最典型的调度触发事件。教训：到钟就换人，这是调度最基本的触发点。
- 选了 I/O 完成那条：I/O 做完了，等 I/O 的进程可以跑了，操作系统要重新调度。教训：I/O 完成会唤醒进程，唤醒就可能调度。
- 选了进程阻塞那条：跑着的进程自己阻塞了，CPU 空出来了，必须调度。教训：CPU 空出来不调度，CPU 就闲着了。

笔记本一句话： 第 30 题＋调度触发：时间片用完、I/O 完成、阻塞会触发，临界区本身不会。

【图像】



四种情形能否触发处理机调度的判断图

【定位】 操作系统 · 第2章 进程管理 · 处理机调度的时机

【易混对比】

- "临界区"和"时间片用完"的区别：临界区是"一次只进一个"的代码段，要的是互斥，靠锁和信号量保证；时间片用完是"到钟换人"的调度触发事件。互斥是锁的事，调度是时间的事。
- "在临界区里"和"被调度打断"的区别：进程在临界区里正常执行代码，"在临界区"本身不是调度事件；但时间片用完了照样可以打断，临界区不等于关中断。

题目问的是"错误的"：时间片用完、I/O 完成、进程阻塞、新进程到达、系统调用返回都能触发调度，只有"在临界区"本身不是调度触发事件，选 C。

【坑位】

先看清题干问的是"错误的"，别把能调度的当答案。

A 进程结束时：跑着的进程没了，CPU 空出来，必须调度，对的；B 创建新进程后：新进程进了就绪队列，可能引起调度（比如抢占式下优先级更高），"能进行"是对的；D 系统调用完成并返回用户态时：返回前是调度的好时机，对的。C 说"在进程处于临界区时不能进行处理机调度"：临界区本身根本不是调度触发事件，这句话错在把"临界区"和"调度"绑在了一起——临界区要的是互斥不是调度，进程在临界区里时间片用完了照样会被调度打断。所以错误的是 C，选 C。错选 A/B/D 的，一般是没看清"错误的"三个字。

【套路】

- 第一步，认题型：题干问"错误的"，先把这个定死，看到"不能"先别急着选。
- 第二步，列触发点：进程结束、创建新进程、时间片用完、I/O 完成、进程阻塞、系统调用返回，都是调度点。
- 第三步，找异类：临界区管的是互斥不是调度，C 是异类，选 C。

先认"问错"题型，再列触发点，找管互斥不调度的异类。

【类比】

一个人正在单人间里换衣服（临界区）：换衣服本身不会让管理员换人，单人间要的是"一次只进一个"（互斥）；只有到钟了（时间片用完）、等的东西到了（I/O 完成）、这个人自己说要等一会儿（阻塞），管理员才会换人。术语贴回去：单人间就是"临界区"，管理员换人就是"处理机调度"，一次只进一个就是"互斥"。

【口诀】 结束创建和返回，时间用完 I/O 完，都能触发调度忙；临界区里只互斥，本身不换 CPU 岗，问错选 C。

【考频】 「处理机调度」在 2009–2022 年解析中出现 7 次 · 中频

第 31 题 · 引入线程后进程与线程的分工及用户级线程的特点（正确答案：A）

题干与选项：

下列关于进程和线程的叙述中，正确的是( )

- A. 不管系统是否支持线程，进程都是资源分配的基本单位
- B. 线程是资源分配的基本单位，进程是调度的基本单位
- C. 系统级线程和用户级线程的切换都需要内核的支持
- D. 同一进程中的各个线程拥有各自不一的地址空间

入手点： 先分工：进程管资源，线程管执行；再想用户级线程的管理者是谁：应用程序自己，不是内核。

专有名词从零解释：

- 先想“进程”：一个跑着的程序，有自己的地址空间、打开的文件这些资源；进程是资源分配的单位。
- 再想“线程”：进程里的一条执行线；同进程的线程共享地址空间，切换起来比进程切换轻得多；线程是调度的单位。
- 再想“用户级线程”：线程的管理（创建、切换、调度）全由应用程序自己做，内核不知道这些线程的存在。
- 再想“内核级线程”：线程的管理由内核做，内核知道每个线程；和用户级线程正好反过来。

考查知识点：

- 先想分工：进程是资源分配单位（地址空间、文件这些资源按进程分）；线程是调度单位（CPU 按线程分）；同进程的线程共享地址空间。
- 再想用户级线程的特点：管理全由应用程序完成，不需要内核干预；内核意识不到这些线程的存在，只看到进程。
- 再想答案：分工加用户级线程特点合起来，就是 A。
- 拿生活例子验算：进程像一家公司（资源按公司分），线程像公司里的员工（活按员工排）；用户级线程像公司内部自己排班，老板（内核）只看到公司，看不到每个员工怎么排。
- 边界情况：用户级线程切换不用进核心态，所以切换快；但一个用户级线程阻塞了，整个进程里的线程都动不了（因为内核只看到进程）。
- 易混对子：不要把线程当成资源分配单位——地址空间是按进程分的，不是按线程分的；不要以为用户级线程要内核调度——用户级线程的调度是应用程序自己做的，内核不知道；不要把用户级线程和内核级线程的特点互换。

正确选项为什么对： 进程管资源、线程管调度、同进程线程共享地址空间，用户级线程由应用程序管理、内核意识不到，就是 A，选 A。

错误选项为什么错＋举例：

- 把线程说成资源分配单位：资源是按进程分的，线程共享进程的资源。教训：分工是进程管资源、线程管执行。
- 说用户级线程要内核调度：用户级线程的调度是应用程序自己做的。教训：用户级的意思就是用户自己管，内核不管。
- 说同进程线程不共享地址空间：同进程的线程共享地址空间，这是线程轻量的原因。教训：共享地址空间是线程相对进程最大的不同。

笔记本一句话： 第 31 题＋进程线程分工：进程管资源，线程管调度；用户级线程应用程序自己管，内核意识不到。

【图像】



进程是资源分配单位、线程是调度单位及用户级线程管理示意图

【定位】 操作系统 · 第2章 进程管理 · 进程与线程的分工、用户级线程的特点

【易混对比】

- "进程"和"线程"的区别：进程是资源分配的单位（地址空间、打开的文件按进程分）；线程是调度的单位（CPU 按线程分），同进程的线程共享地址空间。
- "用户级线程"和"内核级线程"的区别：用户级线程的创建、切换、调度全由应用程序自己做，内核意识不到这些线程的存在；内核级线程由内核管理，内核知道每个线程。

进程管资源、线程管调度、同进程线程共享地址空间；用户级线程由应用程序自己管理，内核意识不到，选 A。

【坑位】

分工一句话：进程分资源，线程分 CPU；用户级三个字的意思就是用户自己管。

B 把分工说反了：线程是调度的基本单位，进程是资源分配的基本单位，B 正好颠倒；C 说"系统级线程和用户级线程的切换都需要内核的支持"：用户级线程的切换是应用程序自己做的，不需要内核支持，内核甚至意识不到这些线程；D 说"同一进程中的各个线程拥有各自不同的地址空间"：同进程的线程共享进程的地址空间，这正是线程切换比进程切换轻的原因。错选 B 的，是分工口诀没背牢；错选 C 的，是把用户级线程当成了内核级线程。

【套路】

- 第一步，核分工：资源按谁分（进程）、CPU 按谁分（线程）。
- 第二步，核管理者：用户级线程谁来管（应用程序自己），内核知不知道（不知道）。
- 第三步，逐项排除：B 颠倒了分工，C 给用户级线程加了内核支持，D 拆散了共享地址空间，剩下 A。

分工看"谁分什么"，线程看"谁来管"，逐项排除。

【类比】

进程像一家公司：资源（办公室、资金）按公司分；线程像公司里的员工：活按员工排，大家共用一间办公室（共享地址空间）；用户级线程像公司内部自己排班，老板（内核）只看到公司，看不到每个员工怎么排。术语贴回去：办公室就是"地址空间"，排班就是"线程调度"，老板就是"内核"。

【口诀】 进程分资源，线程分 CPU；用户级自己排，内核只见进程名，选 A。

【考频】 「线程」在 2009–2022 年解析中出现 9 次 · 中频

第 32 题 · 哪些措施能提高磁盘 I/O 性能、哪种措施基本无效（正确答案：B）

题干与选项：

下列选项中，不能改善磁盘设备 I/O 性能的是( )

- A. 重排 I/O 请求次序
- B. 在一个磁盘上设置多个分区
- C. 预读和滞后写
- D. 优化文件物理块的分布

入手点： 一条一条看它省不省时间：省寻道时间、省传输次数、省等待时间的都有效；只改逻辑划分、不改变磁头移动和传输的，基本无效。

专有名词从零解释：

- 先想“寻道时间”：磁头从当前磁道移到目标磁道的时间；这是磁盘 I/O 里最花时间的部。
- 先想“I/O 调度”：把一批 I/O 请求重新排个序，让磁头少来回跑。
- 再想“缓冲区、预读、滞后写”：缓冲区是内存里的一块暂存区；预读是提前把可能要读的数据读进来；滞后写是写操作先放缓冲区、攒一攒再真正写盘。
- 再想“分区”：把一个磁盘在逻辑上切成几个区，比如 C 盘、D 盘；分区只改逻辑划分，不改物理结构。

考查知识点：

- 先想 A 重排 I/O 请求（有效）：按磁道顺序重排请求，磁头少来回跑，寻道时间省下来了。
- 再想 C 缓冲区加预读、滞后写（有效）：缓冲把多次小 I/O 合并，预读把等待藏起来，滞后写把写盘推迟合并，总的 I/O 次数和等待时间都少了。
- 再想 D 优化文件物理块分布（有效）：把一个文件的数据块放在相邻磁道上，顺序读的时候磁头不用大范围跳，寻道时间少了。
- 再想 B 一个磁盘上设多个分区（基本无效）：分区只是逻辑切分，不改变磁头移动距离、不改变传输率，磁盘本身的速度一点没变。
- 再想答案：基本无效的是 B，就是答案。
- 拿生活例子验算：书架上的书重新按顺序排（重排请求）、一次多拿几本（缓冲预读）、把一套书放相邻格子（优化分布），都能省时间；只在书架上贴几个分区标签（分区），找书的速度一点没变。
- 边界情况：分区对管理有好处（比如分开存放系统和数据），但对 I/O 性能基本没帮助；题目问的是性能，不是管理。
- 易混对子：不要把“分区”和“分盘”混在一起——多个物理盘并行能提速，一个盘切多个分区不能；不要以为“分区让文件更集中”——分区不改变数据块的物理分布。

正确选项为什么对： 一个磁盘上设多个分区只改逻辑划分，不改变磁头移动和传输率，基本无效，就是 B，选 B。

错误选项为什么错＋举例：

- 选了重排 I/O 请求：重排请求省寻道时间，是真有效的。教训：磁头少跑路就是省时间。
- 选了缓冲区加预读、滞后写：缓冲合并小 I/O、预读藏等待，都是真有效的。教训：次数少了、等待藏起来了，性能就上去了。

- 选了优化物理块分布：数据块放相邻磁道，顺序读少寻道，是真有效的。教训：物理分布变了，寻道时间才变。

笔记本一句话： 第 32 题＋磁盘 I/O 性能：重排请求、缓冲预读滞后写、优化分布都有效；一个盘切多分区基本无效。

【图像】



四种磁盘 I/O 措施有效性判断图

【定位】 操作系统 · 第5章 输入/输出（I/O）管理 · 磁盘 I/O 性能优化措施

【易混对比】

- "分区"和"优化物理块分布"的区别：分区只是在逻辑上把一个盘切成几个区（C 盘、D 盘），不改变数据块的物理位置；优化分布是把一个文件的数据块放到相邻磁道上，真正改变了磁头要跑的路。
- "重排 I/O 请求"和"优化物理块分布"的区别：重排是把一批请求的执行顺序按磁道排好（改顺序），优化分布是把数据放的位置排好（改位置）；一个管顺序，一个管位置，都省寻道。

重排请求、预读滞后写、优化分布都真省时间；一个磁盘上设多个分区只改逻辑划分，不改变磁头移动和传输率，基本无效，选 B。

【坑位】

题目问的是"不能改善性能"，别把真有效的三个当答案；分区不等于加盘。

A 重排 I/O 请求次序：按磁道顺序重排，磁头少来回跑，寻道时间省下来了，真有效；C 预读和滞后写：缓冲把多次小 I/O 合并，预读把等待藏起来，滞后写把写盘推迟合并，I/O 次数和等待时间都少了，真有效；D 优化文件物理块的分布：数据块放相邻磁道，顺序读时磁头不用大范围跳，真有效。B 在一个磁盘上设置多个分区：分区只是逻辑切分，磁头该跑多远跑多远，传输率一点没变，对性能基本没帮助（分区的好处在管理，不在速度）。错选 A/C/D 的，是把有效措施当成了无效；还有一种错法是把"一个盘切多分区"当成"多个物理盘并行"——多个盘并行能提速，一个盘切分区不能。

【套路】

- 第一步，看省不省时间：省寻道、省传输次数、省等待时间的都有效。
- 第二步，逐项过：重排请求省寻道（有效），预读滞后写省次数藏等待（有效），优化分布省寻道（有效），分区只改逻辑划分（基本无效）。
- 第三步，锁定：基本无效的是分区，对应 B。

省时间才有效，只改逻辑划分不算数。

【类比】

书架上的书重新按顺序排（重排请求）、一次多拿几本（缓冲预读）、把一套书放进相邻格子（优化分布），找书都更快；只在书架上贴几个分区标签（分区），找书的速度一点没变。术语贴回去：书架就是"磁盘"，找书的路就是"寻道"，分区标签就是"逻辑分区"。

【口诀】 重排请求少跑路，预读滞后藏等待；分布优化邻道放，分区贴签白忙活，选 B。

【考频】 「磁盘 I/O 性能」在 2009–2022 年解析中出现 56 次 · 高频

第 33 题 · ICMP、UDP、TCP、PPP 分别由哪一层协议直接提供服务（正确答案：B）

题干与选项：

在 TCP/IP 体系结构中，直接为 ICMP 提供服务的协议是( )

- A. PPP
- B. IP
- C. UDP
- D. TCP

入手点： 一个一个找“直接上一层服务”：ICMP 的报文装在 IP 分组里走，UDP 和 TCP 直接给应用层干活，PPP 直接给网络层干活。

专有名词从零解释：

- 先想“直接提供服务”：A 协议直接用 B 协议的服务，中间不隔着别的协议；比如 ICMP 的报文直接装在 IP 分组里，IP 就是直接给 ICMP 服务的。
- 再想“ICMP”：网际控制报文协议，网络层的协议，报告差错、做查询（比如 ping 用的就是它）。
- 再想“UDP、TCP”：传输层的两个协议；UDP 不可靠、不建连接，TCP 可靠、面向连接；它俩都直接给应用层服务。
- 再想“PPP”：点对点协议，数据链路层的协议，拨号上网、串行链路常用它；它直接给网络层服务。

考查知识点：

- 先想 ICMP：ICMP 报文封装在 IP 分组里传送，IP 直接为 ICMP 服务；所以 ICMP 是由网络层的 IP 直接提供服务的。
- 再想 UDP 和 TCP：它俩是传输层协议，直接给应用层服务；应用层的程序（比如浏览器、邮件客户端）直接用它俩。
- 再想 PPP：PPP 是数据链路层协议，它的上层就是网络层，所以 PPP 直接为网络层服务。
- 再想答案：按这个对应关系，就是 B。
- 拿生活例子验算：ICMP 像贴在 IP 信封里的便条（IP 送信），UDP、TCP 像快递公司的两种服务（直接送给收件人），PPP 像小区门口的门禁（直接给小区里的住户服务）。
- 边界情况：ICMP 虽然是网络层协议，但它用 IP 的服务，不等于它就是 IP；层次和“谁给谁服务”是两回事。
- 易混对子：不要把 ICMP 当成传输层协议——它虽然也传报文，但它是网络层的；不要把 PPP 当成网络层协议——PPP 是数据链路层的，它是给网络层服务的；不要把 UDP、TCP 的服务对象说成网络层——它俩直接服务的是应用层。

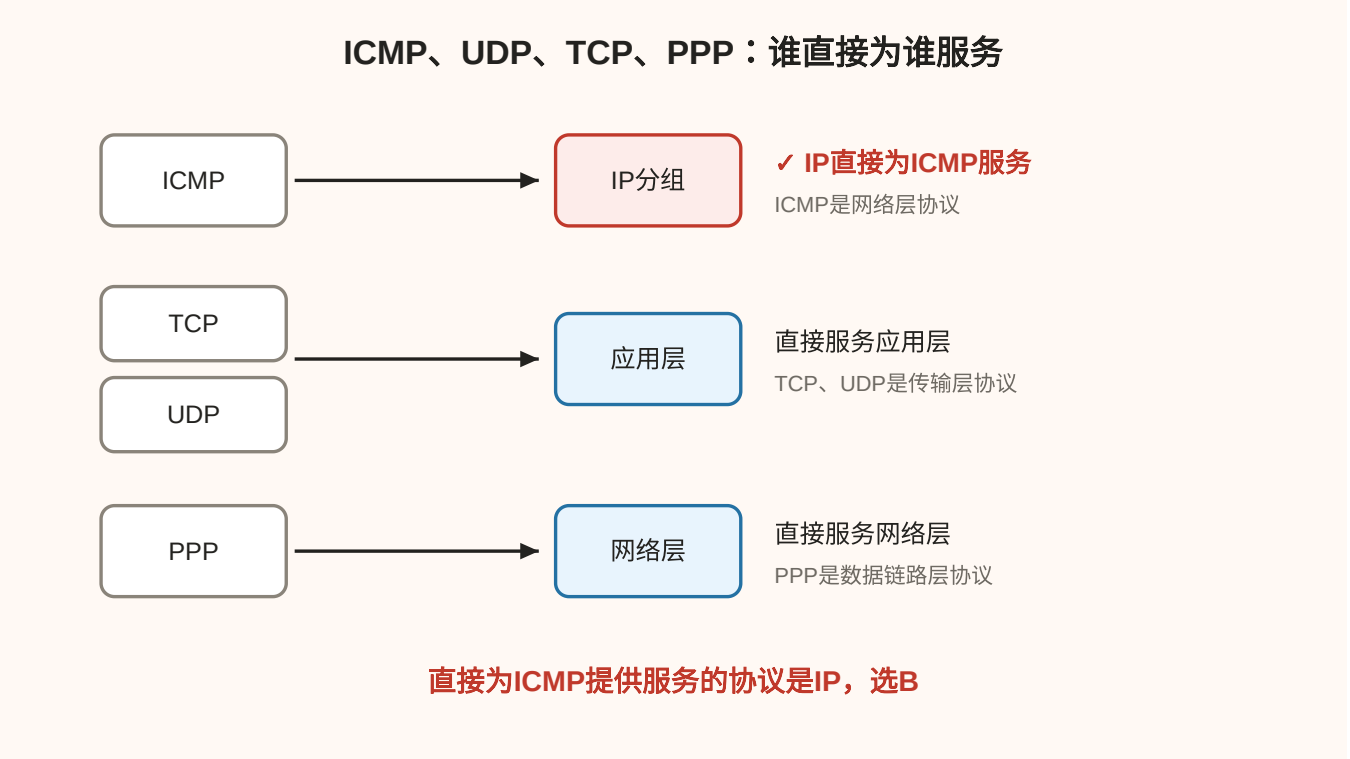
正确选项为什么对： ICMP 由 IP 直接提供服务，UDP、TCP 直接为应用层服务，PPP 直接为网络层服务，对应关系就是 B，选 B。

错误选项为什么错＋举例：

- 把 ICMP 说成由传输层服务：ICMP 的报文装在 IP 分组里，IP 是网络层的。教训：看报文装在哪种分组里，就知道谁直接服务它。
- 把 PPP 说成网络层协议：PPP 是数据链路层的，它是给网络层服务的，不是网络层本身。教训：服务者和被服务者不能弄反。
- 把 UDP、TCP 的服务对象说成网络层：它俩是传输层，直接服务应用层。教训：传输层上面就是应用层。

笔记本一句话： 第 33 题＋谁给谁服务：ICMP 用 IP 的服务，UDP、TCP 服务应用层，PPP 服务网络层。

【图像】



四层协议栈服务关系图：ICMP报文装入IP分组、TCP与UDP服务应用层、PPP服务网络层

【定位】 计算机网络 · 第4章 网络层 · ICMP由哪一层协议直接提供服务

【易混对比】

- "ICMP"和"传输层协议"的区别：ICMP的报文直接装在IP分组里传送，IP是网络层的，所以ICMP是网络层协议；"也传报文"不等于"归传输层管"。
- "PPP"和"网络层协议"的区别：PPP是数据链路层协议，它直接给网络层提供服务，是服务网络层的一方，不是网络层本身；服务者和被服务者不能弄反。
- "UDP、TCP的服务对象"和"网络层"的区别：UDP、TCP是传输层协议，它俩的上一层就是应用层，直接服务的是应用层，不服务网络层。

ICMP的报文装在IP分组里走，直接为ICMP提供服务的是IP，选B。

【坑位】

先想"报文装在哪种分组里"，别被"ICMP也传报文"骗成传输层。

选A（PPP）：PPP是数据链路层的协议，它服务的是网络层，和ICMP没有直接服务关系。选C或D（UDP、TCP）：它俩是传输层协议，服务对象是应用层，层次在ICMP之上，不可能反过来给ICMP提供服务。注意"ICMP是网络层协议"和"ICMP用IP的服务"不矛盾：层次归属看协议定义，"谁给谁服务"看报文装在哪种分组里。

【类比】

像一家快递公司的分工：IP是运货的卡车，ICMP是贴在货箱上的便条——便条跟着卡车走，是卡车直接把便条捎过去的；UDP、TCP是两种快递服务（普通件和加急件），直接把包裹交到收件人手里，服务的是收件人；PPP是小区门口的门禁，直接给小区里的住户开门放行。术语贴回去：便条=ICMP报文、卡车=IP分组、快递服务=传输层协议、收件人=应用层、门禁=PPP、住户=网络层。

【口诀】 ICMP搭IP的车，TCP、UDP送应用，PPP给网络层开门；问谁直接服务ICMP，IP当仁不让选B。

【考频】 「ICMP由哪层协议提供服务」在 2009–2022 年解析中出现 3 次 · 中频

第 34 题 · 物理层中“规定各条物理线路工作过程和时序关系”的特性名称（正确答案：C）

题干与选项：

在物理层接口特性中，用于描述完成每种功能的事件发生顺序的是( )

- A. 机械特性
- B. 功能特性
- C. 过程特性
- D. 电气特性

入手点： 四个特性一个一个对：形状尺寸、干什么用、电压电平、工作过程和时序——最后这个就是规程特性。

专有名词从零解释：

- 先想“机械特性”：接口长什么样——形状、尺寸、引脚有多少、线怎么接。
- 再想“功能特性”：每条线是干什么用的——哪条传数据、哪条传控制信号、哪条接地。
- 再想“电气特性”：信号长什么样——电压范围是多少、什么电平算 0、什么电平算 1、传输速率多快。
- 再想“规程特性”：各条线怎么配合干活——工作过程是什么、先干什么后干什么、时序关系是怎样的（比如先握手再传数据）。

考查知识点：

- 先想题目给的描述：“规定各条物理线路工作过程和时序关系”——关键词是“过程”和“时序”，这正是规程特性的定义，就是 C。
- 再想其余三个为什么不对：机械特性说的是形状尺寸，不说过程；功能特性说的是每条线干什么用，不说时序；电气特性说的是电压电平，不说过程。
- 拿生活例子验算：开会时，会议室多大、桌子什么样是机械特性；谁发言、谁记录是功能特性；话筒音量多大是电气特性；先主持人开场、再轮流发言、最后总结，这个流程和顺序就是规程特性。
- 边界情况：四个特性缺一不可，描述接口时四个都要说；但题目只问“过程和时序”对应哪一个，答案只有一个。
- 易混对子：不要把“功能特性”和“规程特性”混在一起——功能是“干什么”，规程是“按什么顺序干”；不要把“电气特性”当成规程——电压电平和过程时序是两回事。

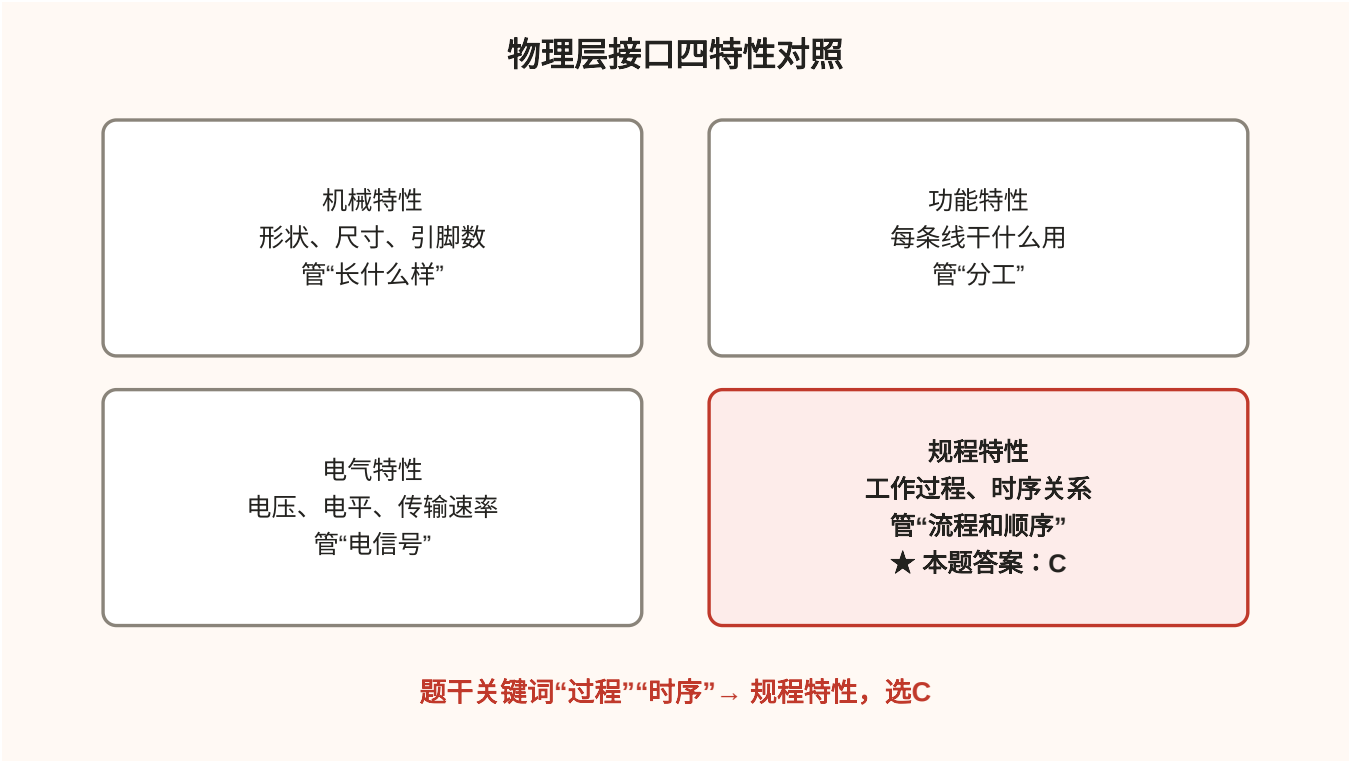
正确选项为什么对： “工作过程和时序关系”就是规程特性的定义，就是 C，选 C。

错误选项为什么错＋举例：

- A（机械特性）：机械特性说的是形状、尺寸、引脚数，不说过程。教训：形状和流程是两回事。
- B（功能特性）：功能特性说的是每条线干什么用，不说时序。教训：“干什么”和“按什么顺序干”要分开。
- D（电气特性）：电气特性说的是电压范围、电平、速率，不说过程。教训：电平高低和工作顺序是两回事。

笔记本一句话： 第 34 题＋物理层四特性：机械管形状，功能管用途，电气管电平，规程管过程和时序。

【图像】



物理层四特性对照卡：机械管形状、功能管用途、电气管电平、规程管工作过程和时序，规程特性高亮

【定位】 计算机网络 · 第2章 物理层 · 物理层接口的四种特性

【易混对比】

- "功能特性"和"规程特性"的区别：功能特性回答"每条线干什么用"（分工）；规程特性回答"按什么顺序、什么时序配合干活"（流程和时序）。"干什么"和"按什么顺序干"是两回事。
- "机械特性"和"规程特性"的区别：机械特性管形状、尺寸、引脚数这些看得见的物理形态，不涉及工作过程和时序。
- "电气特性"和"规程特性"的区别：电气特性管电压范围、电平、传输速率这些电信号参数，不涉及工作过程和时序。

"规定各条物理线路工作过程和时序关系"就是规程特性的定义，选C。

【坑位】

题干出现"过程""时序"两个字就锁定规程特性，别被电气特性里的"速率"带跑。

选A（机械特性）：机械特性只说接口的形状、尺寸、引脚数，不说过程。选B（功能特性）：功能特性只说每条线干什么用，不说时序——"干什么"和"按什么顺序干"要分开。选D（电气特性）：电气特性说的是电压、电平、速率；速率是信号有多快，时序是事件谁先谁后，两回事。记忆抓手：机械看外形，功能看分工，电气看电信号，规程看流程和时序。

【类比】

像组织一场会议：会议室多大、桌椅什么样是机械特性；谁发言、谁记录是功能特性；话筒音量开多大是电气特性；先主持人开场、再轮流发言、最后总结——这个流程和先后顺序就是规程特性。术语贴回去：会议流程=工作过程、发言顺序=时序关系、四张卡片=物理层四特性。

【口诀】 机械管外形，功能管分工，电气管电平，规程管过程和时序。

【考频】 「物理层接口特性」在 2009–2022 年解析中出现 2 次 · 低频

第 35 题 · 以太网提供的是何种服务（正确答案：A）

题干与选项：

以太网的 MAC 协议提供的是( )

- A. 无连接的不可靠的服务
- B. 无连接的可靠的服务
- C. 有连接的不可靠的服务
- D. 有连接的可靠的服务

入手点： 先看以太网干活的方式：无连接、不编号、不要求确认；再想这种方式对应哪种服务：不管对方收到没有，就是不可靠服务。

专有名词从零解释：

- 先想“无连接”：发数据之前不先建立连接，拿起来就发。
- 再想“不编号”：帧没有序号，接收方不知道顺序、也不知道丢了哪一帧。
- 再想“不要求确认”：发完不管对方收到没有，不等回信、不重发。
- 再想“不可靠服务”：也叫尽最大努力交付——尽量送到，但不保证一定送到、不保证顺序、不保证不重不丢；差错要靠高层去纠。

考查知识点：

- 先想以太网的方式：无连接、不编号、不要求确认，三条全占。
- 再想这是什么服务：三条合起来就是不可靠服务（尽最大努力交付），就是 A。
- 再想差错谁来纠：以太网不管差错，差错由高层（比如 TCP）去发现、去重传、去纠。
- 拿生活例子验算：以太网像寄平信——写好地址就投进邮筒，不管对方收到没有、也不管顺序；挂号信（可靠服务）要对方签收、丢了重寄，那是 TCP 干的事，不是以太网干的事。
- 边界情况：局域网里出错率本来就低，平信大概率能到，但“大概率能到”不等于“保证能到”；服务性质看的是保证不保证，不是概率高不高。
- 易混对子：不要把 TCP 的可靠安到以太网头上——可靠是 TCP 的活，以太网只管尽最大努力；不要以为“局域网出错少”就是“可靠服务”——可靠服务的意思是“保证”，不是“很少出错”。

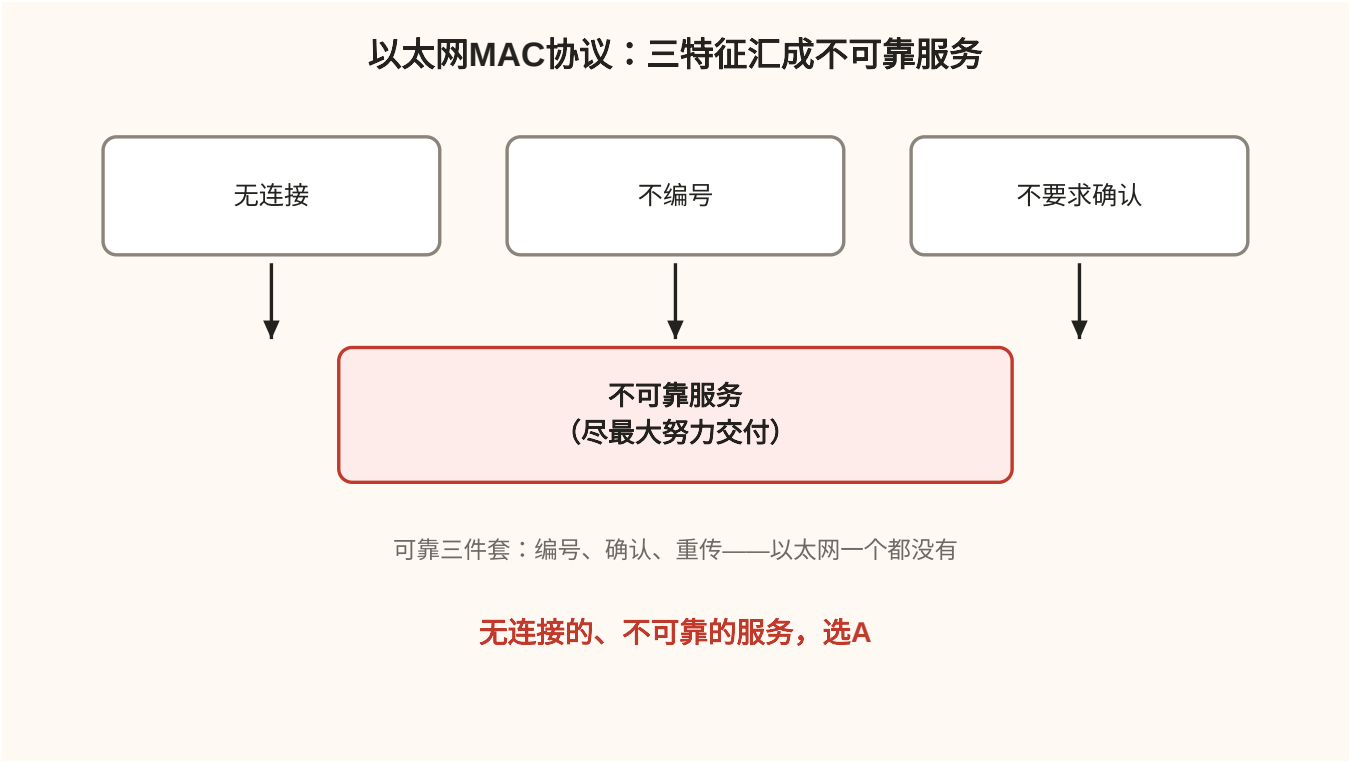
正确选项为什么对： 以太网无连接、不编号、不要求确认，就是不可靠服务（尽最大努力交付），就是 A，选 A。

错误选项为什么错＋举例：

- 说成可靠服务：可靠服务要编号、要确认、要重传，以太网三条都没有。教训：可靠的三个要素一个一个对，缺一个就不是可靠。
- 说成面向连接服务：以太网发之前不建连接，是无连接的。教训：面向连接要先握手建连接，以太网没有这一步。
- 说成有确认的服务：以太网发完不等回信。教训：不要求确认，就没有确认机制。

笔记本一句话： 第 35 题＋以太网服务：无连接、不编号、不要求确认，就是不可靠服务（尽最大努力交付）。

【图像】



以太网MAC服务三特征图：无连接、不编号、不要求确认，汇成不可靠服务即尽最大努力交付

【定位】 计算机网络 · 第3章 数据链路层 · 以太网MAC协议提供的服务类型

【易混对比】

- "无连接"和"面向连接"的区别：无连接是拿起帧就发，事先不建连接；面向连接要先握手建立连接，发完再释放。
- "不可靠服务"和"可靠服务"的区别：可靠服务要编号、确认、重传三件套，缺一不可；以太网三条全没有，就是不可靠服务（尽最大努力交付）。
- "以太网"和"TCP"的区别：可靠是TCP的活——TCP编号、确认、重传全做；以太网只管把帧尽力送出去，差错留给高层去纠。

以太网无连接、不编号、不要求确认，就是不可靠服务（尽最大努力交付），选A。

【坑位】

看到"局域网出错少"别手滑选可靠——可靠看的是保不保证，不是概率高不高。

误选B或D（可靠服务）：可靠服务的三个要素是编号、确认、重传，以太网一个都没有；以太网有CRC校验不假，但那是"检错后丢弃"，不是"纠错重传"，检错不等于可靠。误选C（面向连接）：以太网发帧之前不建连接，是无连接的。记住：局域网误码率低是事实，但"大概率能到"不等于"保证能到"，服务性质看保证不保证。

【类比】

以太网像寄平信：写好地址投进邮筒就不管了，不管对方收到没有、也不管顺序；可靠服务像寄挂号信：要对方签收，丢了重寄——那是TCP干的事。术语贴回去：平信=无连接不可靠服务、挂号信=面向连接的可靠服务、邮筒=以太网、签收和重寄=确认和重传。

【口诀】 无连接，不编号，不确认；平信一封尽力送，不可靠，选A准。

【考频】 「以太网MAC协议服务类型」在 2009–2022 年解析中出现 11 次 · 高频

第 36 题 · 在一帧发送到收到确认的时间内最多可发多少帧、至少需几位编号（正确答案：B）

题干与选项：

两台主机之间的数据链路层采用后退 N 帧协议（GBN）传输数据，数据传输速率为 16kbps，单向传播时延为 270ms，数据帧长度范围是 128~512 字节，接收方总是以与数据帧等长的帧进行确认。为使信道利用率达到最高，帧序号的比特数至少为( )

- A. 5
- B. 4
- C. 3
- D. 2

入手点： 先算发一帧要多久，再算一来回要多久，用一来回除以发一帧的时间，得到一来回能发的帧数；编号位数要能区分这些帧，不能少。

专有名词从零解释：

- 先想“发送时延”：把一帧的每一位都推到信道上要花的时间，等于帧长除以信道速率。
- 再想“传播时延”：信号在信道里跑过去要花的时间，本题单程是 270ms。
- 再想“帧编号”：给每帧编的号；接收方靠编号认出“这是新帧还是重发的旧帧”；编号个数必须大于一来回内能发的帧数，不然新帧和旧帧的号就撞了。

考查知识点：

- 先想发一帧的时间：帧长 128 字节＝128×8＝1024 位，信道速率 16×10^3 位每秒，发送时延＝1024÷16000＝64ms。
- 再想一来回的时间：发一帧 64ms，信号单程跑 270ms，来回就是 270×2＝540ms，再加确认帧本身的发送时间 64ms，总共 64＋540＋64＝668ms。
- 再想一来回能发多少帧：668÷64＝10.4 帧，也就是说，一个来回的时间里最多能发 10.4 帧。
- 再想编号至少几位：编号个数要大于 10.4，至少要 11 个号；3 位只有 8 个号，不够；4 位有 16 个号，够了，所以至少 4 位，就是 B。
- 拿生活例子验算：寄一封信要 64 分钟写好，路上单程 270 分钟，对方回信也要 64 分钟写；一来回 668 分钟里，你能写 10.4 封信；要给信编号不撞号，至少要 11 个号，4 位二进制（16 个）才够。
- 边界情况：确认帧和数据帧一样长（都是 128 字节），所以确认帧的发送时间也是 64ms，不能漏；如果确认帧不占发送时间，一来回就是 604ms，结果就不一样了。
- 易混对子：不要漏了确认帧的发送时间——一来回是“发数据帧＋来回传播＋发确认帧”，三段都要算；不要把 128 字节当成 128 位——字节要乘 8 才是位数；不要算成 3 位——3 位只有 8 个号，小于 10.4，编号会撞。

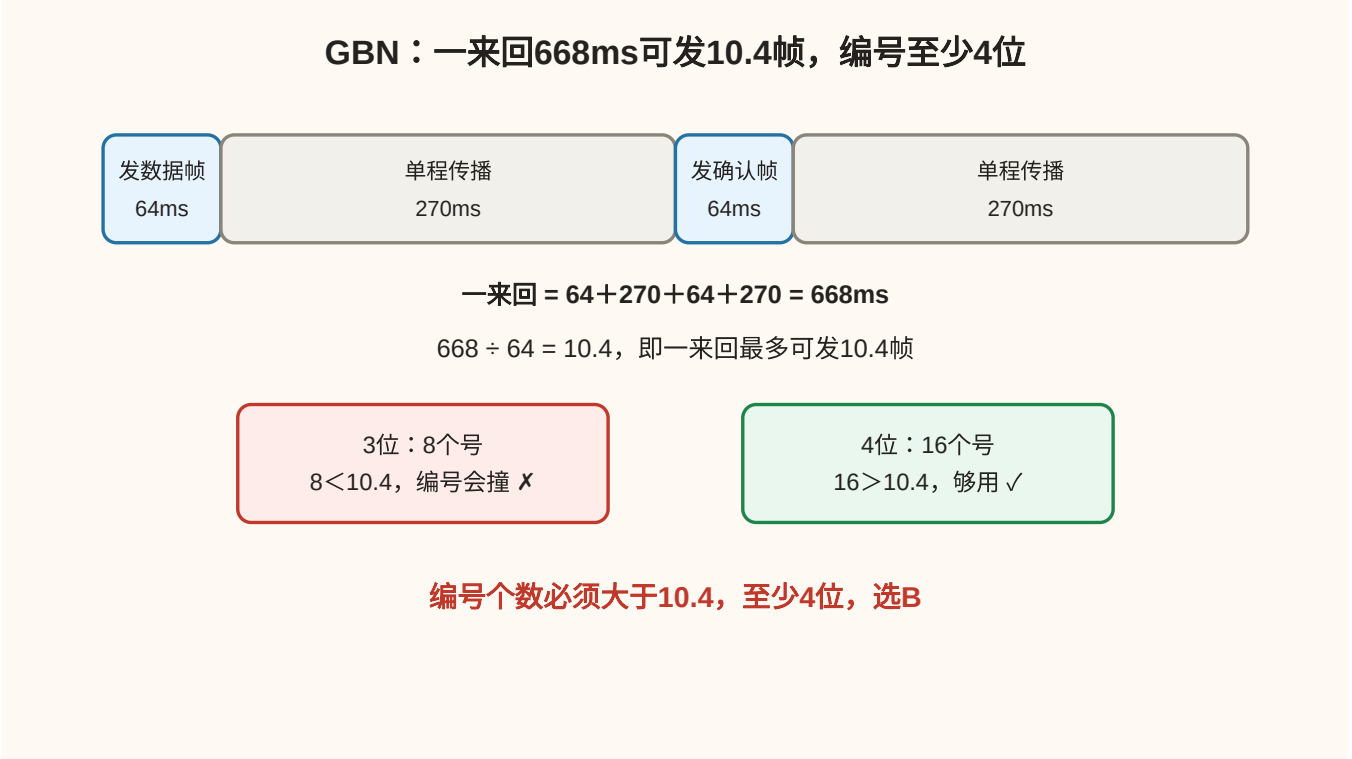
正确选项为什么对： 发一帧 64ms，一来回 668ms，能发 10.4 帧，编号至少 4 位，就是 B，选 B。

错误选项为什么错＋举例：

- 算成 3 位：3 位只有 8 个号，小于 10.4，新帧和旧帧的号会撞。教训：编号个数必须大于一来回能发的帧数。
- 漏了确认帧的发送时间：只算 64＋540＝604ms，得到 9.4 帧，结论跟着错。教训：一来回有三段，确认帧的发送时间也要算。
- 128 字节当成 128 位：发送时延算成 8ms，后面全错。教训：字节乘 8 才是位数。

笔记本一句话： 第 36 题＋编号位数：发一帧 64ms，一来回 668ms，能发 10.4 帧，至少 4 位编号。

【图像】



GBN一来回时间轴：发送数据帧64ms、单程传播270ms、发送确认帧64ms、单程传播270ms，总668ms可发10.4帧

【定位】 计算机网络 · 第3章 数据链路层 · GBN协议帧序号的最少比特数

【易混对比】

- "发送时延"和"传播时延"的区别：发送时延是把帧的每一位推上信道花的时间，等于帧长除以速率；传播时延是信号在信道里跑的时间。一个看帧多长，一个看路多远。
- "3位编号"和"4位编号"的区别：3位只有8个号，小于一来回能发的10.4帧，新帧和旧帧的号会撞车；4位有16个号，才够区分。
- "确认帧的发送时间"和"零"的区别：确认帧和数据帧等长，也要64ms才能推上信道；一来回有三段，漏了这一段总数就错。

发一帧64ms，一来回668ms，能发10.4帧，编号至少4位，选B。

【坑位】

一来回是三段相加，确认帧64ms的发送时间最容易漏。

漏算确认帧发送时间：只算64＋540＝604ms，得到9.4帧，后面全错——确认帧和数据帧等长（题干已给），发送时间也是64ms。把128字节当128位：发送时延算成8ms，后面全错——字节要乘8才是位数。算成3位：3位只有8个号，小于10.4，编号会撞——编号个数必须大于一来回能发的帧数。

【套路】

- 第一步，算发送时延：128字节＝1024位，1024÷16000＝64ms。
- 第二步，算一来回时间：发数据帧64ms＋来回传播270×2＝540ms＋发确认帧64ms＝668ms。
- 第三步，算帧数定位数：668÷64＝10.4帧；编号个数要大于10.4，3位只有8个不够，4位有16个够了，选B。

三段加总得一来回，除以发一帧的时间，编号个数要大于这个商。

【类比】

像寄挂号信：写好一封信要64分钟，路上单程270分钟，对方写回执也要64分钟；一来回668分钟里你能写10.4封信。要给每封信编号不撞号，至少要11个号——3位二进制只有8个号，不够用；4位有16个号，才够。术语贴回去：写信=发送时延、路上时间=传播时延、回执=确认帧、信的编号=帧序号。

【口诀】 一帧六十四毫秒推上道，来回六六八分三段到；十点四帧八号不够撞，四位十六号才够用。

【考频】 「GBN帧序号比特数」在 2009–2022 年解析中出现 10 次 · 高频

第 37 题 · IP 路由器功能若干陈述的正误组合判断（正确答案：C）

题干与选项：

下列关于 IP 路由器功能的描述中，正确的是( )

Ⅰ.运行路由协议，设备路由表

Ⅱ.检测到拥塞时，合理丢弃 IP 分组

Ⅲ.对收到的 IP 分组头进行差错校验，确保传输的 IP 分组不丢失

Ⅳ.根据收到的 IP 分组的目的 IP 地址，将其转发到合适的输出线路上

- A. 仅Ⅲ、Ⅳ
- B. 仅Ⅰ、Ⅱ、Ⅲ
- C. 仅Ⅰ、Ⅱ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

入手点： 一条一条验：路由协议和路由表是不是路由器的活、转发是不是按目的 IP 地址、拥塞时能不能丢包、首部差错检验包不包括“保证不丢失”。

专有名词从零解释：

- 先想“路由器”：连接不同网络的设备，收到 IP 分组后，按目的 IP 地址查路由表，决定从哪个口转发出去。
- 再想“路由协议”：路由器之间互相通告“我能到哪些网络”的协议；靠它，每台路由器才能建起自己的路由表。
- 再想“路由表”：路由器手里的“地图”，写着“去某个网络，从哪个口走”。
- 再想“源点抑制”：路由器拥塞时给源点发的一种 ICMP 报文，意思是“你发慢点，我快装不下了”。

考查知识点：

- 先想陈述Ⅰ（对）：运行路由协议、设置路由表——路由器要靠路由协议学到各网络的去向，才能建起路由表，这是路由器的本职。
- 再想陈述Ⅳ（对）：按收到的 IP 分组的目的 IP 地址，把分组转发到合适的输出线路上——这就是路由器转发分组的核心动作。
- 再想陈述Ⅱ（对）：拥塞时合理丢弃分组，并且给源点发源点抑制 ICMP 报文，让源点慢点发，缓解拥塞。
- 再想陈述Ⅲ（错）：路由器确实做首部差错检验、丢弃首部有差错的报文，但这不等于“保证不丢失”——路由器在拥塞、队列满的时候本来就会丢包，IP 层本来就不保证可靠；“检验首部”对，不等于“保证不丢失”对。
- 再想组合：Ⅰ、Ⅱ、Ⅳ正确，就是 C。
- 拿生活例子验算：邮局分拣员要会看地图（路由表）、按收件地址分拣（转发）、包裹太多时可以退掉一些并告诉寄件人慢点寄（拥塞丢弃加源点抑制）；但分拣员验出破损包裹扔掉，不等于他保证每个包裹都不丢。
- 边界情况：路由器只验首部，不验整个分组的数据部分；数据部分的差错由高层去管。
- 易混对子：不要把“丢弃有差错首部”理解成“保证不丢失”——丢弃本身就是一种丢失；不要把源点抑制当成“不丢包的办法”——源点抑制是丢包之后告诉源点慢点发，不是代替丢包。

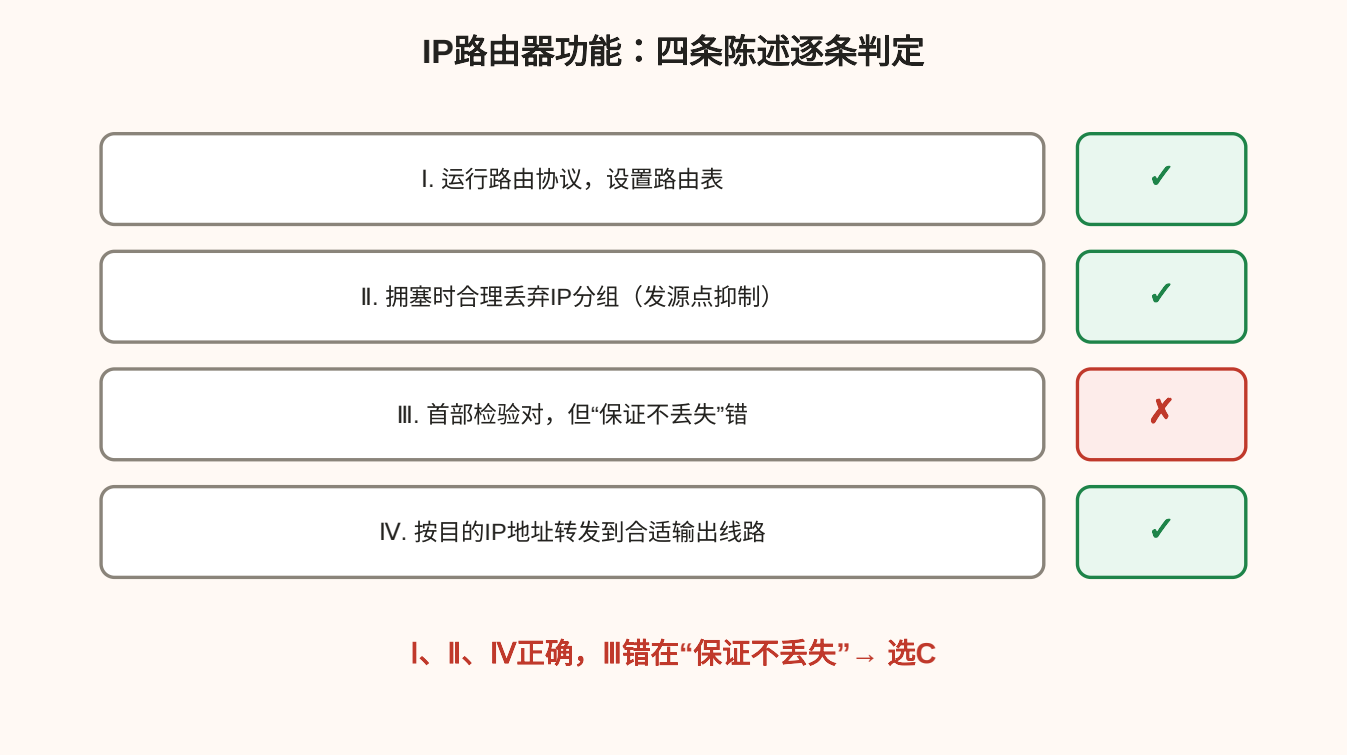
正确选项为什么对： Ⅰ、Ⅱ、Ⅳ正确，Ⅲ错在“保证不丢失”，组合是Ⅰ、Ⅱ、Ⅳ，就是 C，选 C。

错误选项为什么错＋举例：

- 含III的组合：III把“首部检验”偷换成了“保证不丢失”，IP 层本来就不保证可靠。教训：检验和保证是两回事。
- 漏了 I 的组合：没有路由协议和路由表，路由器就不知道往哪转。教训：路由表是转发的前提。
- 漏了 II 的组合：拥塞时丢弃加源点抑制是路由器缓解拥塞的正当做法。教训：拥塞时路由器可以丢包，不是只能硬扛。
- 漏了IV的组合：按目的 IP 地址转发是路由器的核心动作，不能漏。教训：转发就是路由器的本职。

笔记本一句话： 第 37 题＋路由器功能：运行路由协议建表、按目的 IP 转发、拥塞丢弃加源点抑制；首部检验不等于保证不丢失。

【图像】



路由器四条功能陈述判定表： Ⅰ 建表打勾、 Ⅱ 拥塞丢弃打勾、Ⅲ保证不丢失打叉、Ⅳ按目的IP转发打勾，组合为C

【定位】 计算机网络 · 第4章 网络层 · IP路由器的功能

【易混对比】

- "首部差错检验"和"保证不丢失"的区别：检验只是发现首部错了就丢弃，丢弃本身就是一种丢失；IP层本来就不保证可靠，"会检验"和"保证不丢"是两回事。
- "拥塞时丢弃"和"硬扛"的区别：路由器拥塞时合理丢包、并给源点发抑制报文让它慢点发，是正当的缓解做法，不是失职。
- "运行路由协议"和"只会转发"的区别：没有路由协议学来的路由表，路由器根本不知道往哪转；建表是转发的前提，不是多余动作。

Ⅰ、Ⅱ、Ⅳ正确，Ⅲ错在"保证不丢失"，组合是 Ⅰ、Ⅱ、Ⅳ，选C。

【坑位】

陈述III前半句是对的、后半句偷换了概念，半对半错整条判错。

III说"对收到的IP分组头进行差错校验"——这半句是对的，路由器确实只验首部；但后半句"确保传输的IP分组不丢失"错了：验出差错的直接丢弃是常态，拥塞队列满了也会丢，IP层从不做可靠保证。凡是含III的选项（A、B、D）全错。 I 不能漏：没有路由协议和路由表，转发无从谈起。 II 不能漏：拥塞丢弃加源点抑制是路由器的正当操作。IV是核心动作：按目的IP地址查表转发，漏了它等于路由器不会干本职工作。

【类比】

路由器像邮局的分拣员：要会看地图（运行路由协议、设置路由表），按收件地址把包裹分到对应的车（按目的IP转发），包裹爆仓时可以退掉一部分并打电话让寄件人慢点寄（拥塞丢弃加源点抑制）；但分拣员验出破损包裹扔掉，不等于他保证每个包裹都不丢。术语贴回去：地图=路由表、收件地址=目的IP地址、爆仓退件=拥塞丢弃、打电话=源点抑制ICMP报文。

【口诀】 建表转发加抑制，拥塞丢包是本分；首部检验会丢弃，保证不丢是陷阱。

【考频】 「IP路由器功能」在 2009–2022 年解析中出现 14 次 · 高频

第 38 题 · 在数据链路层上传送数据时最终要用的地址及实现该地址解析的协议（正确答案：A）

题干与选项：

ARP 协议的功能是( )

- A. 根据 IP 地址查询 MAC 地址
- B. 根据 MAC 地址查询 IP 地址
- C. 根据域名查询 IP 地址
- D. 根据 IP 地址查询域名

入手点： 先分清两层各用什么地址：网络层用 IP 地址，数据链路层用硬件地址；再想换地址的协议：IP 地址换成硬件地址，靠的就是 ARP。

专有名词从零解释：

- 先想“硬件地址”：也叫 MAC 地址，烧在网卡里的地址，数据链路层认的就是它；帧头里写的就是收发双方的硬件地址。
- 再想“IP 地址”：网络层的地址，逻辑上的地址；分组在网络层跑的时候认它，到了数据链路层要换成硬件地址才能真正发出去。
- 再想“ARP”：地址解析协议，干的就是“已知 IP 地址、问硬件地址”这件事；一台主机想给某个 IP 发帧，先用 ARP 问一句“这个 IP 的硬件地址是多少”，问到了再发。

考查知识点：

- 先想最终用的地址：数据链路层传数据时，帧头里写的是硬件地址（MAC 地址），不是 IP 地址，就是 A 的前半部分。
- 再想解析协议：把 IP 地址解析成硬件地址的协议是 ARP，就是 A 的后半部分。
- 再想为什么要换：IP 地址是逻辑地址，网卡不认；网卡只认硬件地址，所以发帧之前必须先换。
- 拿生活例子验算：IP 地址像人名（逻辑上的称呼），硬件地址像工位号（真正送信要找的座位）；送信之前要先查花名册（ARP），把人名换成工位号。
- 边界情况：同一局域网里直接 ARP 就能问到；跨网时先把帧发给默认网关，再由网关往下传，每一段链路都要重新换硬件地址。
- 易混对子：不要把 ARP 和 DNS 混在一起——DNS 是域名换 IP 地址，ARP 是 IP 地址换硬件地址，方向和层次都不一样；不要以为链路层用 IP 地址——IP 地址是网络层的，链路层只认硬件地址。

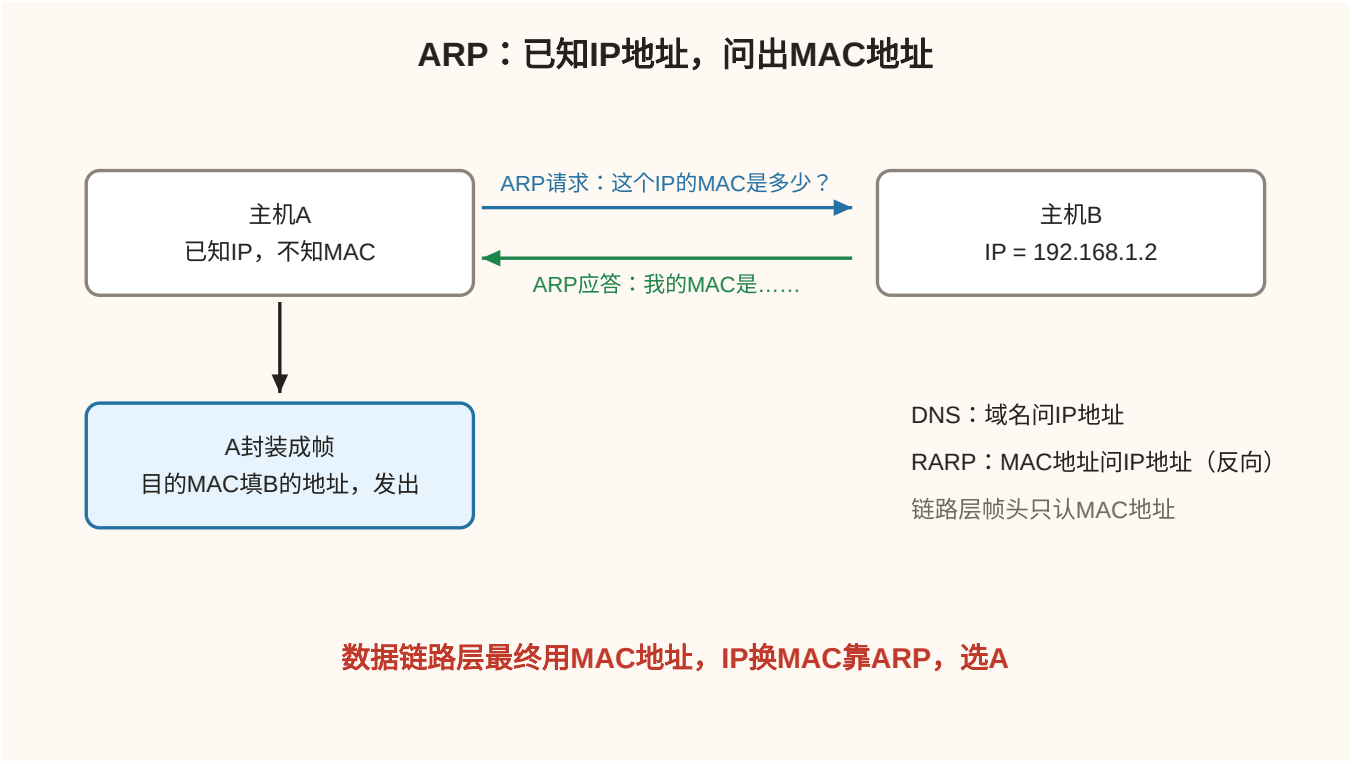
正确选项为什么对： 数据链路层最终用硬件地址，IP 地址换成硬件地址靠 ARP，就是 A，选 A。

错误选项为什么错＋举例：

- 说链路层用 IP 地址：IP 地址是网络层的，链路层帧头里写的是硬件地址。教训：层次不同，地址不同。
- 说解析协议是 DNS：DNS 是域名换 IP 地址，不是 IP 地址换硬件地址。教训：DNS 和 ARP 换的东西不一样。
- 说不用解析、直接发：网卡不认 IP 地址，不换成硬件地址发不出去。教训：发帧之前必须先换地址。

笔记本一句话： 第 38 题＋链路层地址：最终用硬件地址，IP 换硬件靠 ARP（DNS 是域名换 IP，别混）。

【图像】



ARP地址解析流程图：已知IP地址发ARP请求问硬件地址，收到应答后把IP换成MAC再封装成帧发出

【定位】 计算机网络 · 第4章 网络层 · ARP协议的功能

【易混对比】

- "ARP"和"DNS"的区别：ARP是"已知IP地址、问硬件地址"（网络层到数据链路层）；DNS是"已知域名、问IP地址"（应用层到网络层）。换的东西不一样，方向也不一样。
- "ARP"和"RARP"的区别：RARP正好反过来——"已知硬件地址、问IP地址"；选项B说的就是RARP的活，不是ARP。
- "硬件地址"和"IP地址"的区别：硬件地址烧在网卡里，帧头里写它；IP地址是逻辑地址，分组头里写它。网卡不认IP地址，发帧之前必须先换。

数据链路层最终用硬件地址，IP地址换成硬件地址靠ARP，选A。

【坑位】

看到"查询"先定方向：谁问谁，问完换成什么。

选B：那是RARP（硬件地址问IP地址），方向反了。选C或D：那是DNS的活（域名和IP地址互查），层次都不对。还有一种错法是"链路层直接用IP地址"：IP地址是网络层的，网卡只认硬件地址，不换就发不出去。记住链条：应用层认域名，网络层认IP，数据链路层认MAC；域名→IP靠DNS，IP→MAC靠ARP。

【类比】

IP地址像人名（逻辑上的称呼），硬件地址像工位号（送信真正要找的座位）；要送信，先查花名册把人名换成工位号——花名册就是ARP。反过来拿着工位号查人名，那是另一本册子（RARP）；拿着人名查电话号码，那是114查号台（DNS）。术语贴回去：人名=IP地址、工位号=MAC地址、花名册=ARP表、查花名册=ARP请求与应答。

【口诀】 链路层认MAC不认IP，IP换MAC靠ARP；反向查IP找RARP，域名查IP是DNS。

【考频】 「ARP协议功能」在 2009–2022 年解析中出现 8 次 · 中频

第 39 题 · 已知子网掩码与 IP 地址时求广播地址（正确答案：D）

题干与选项：

某主机的 IP 地址为 180.80.77.55，子网掩码为 255.255.252.0。若该主机向其所在子网发

送广播分组，则目的地址可以是( )

- A. 180.80.76.0
- B. 180.80.76.255
- C. 180.80.77.255
- D. 180.80.79.255

入手点： 先看掩码分出几位是子网号、几位是主机号，再把主机号全部置 1，子网号原样不动，就是广播地址。

专有名词从零解释：

- 先想“子网掩码”：一串 1 后面跟一串 0；1 的部分对应子网号，0 的部分对应主机号。
- 再想“子网号”：子网的编号；同一子网里的主机，子网号都一样。
- 再想“主机号”：子网里某台主机的编号；同一子网里每台主机都不一样。
- 再想“广播地址”：子网里专门用来“一喊全子网都收到”的地址；它的主机号全是 1。

考查知识点：

- 先想掩码分界：掩码第 3 字节是 11111100，前 22 位是子网号（16 位加第 3 字节的前 6 位），后 10 位是主机号（第 3 字节的后 2 位加第 4 字节的 8 位）。
- 再想 IP 的第 3 字节：IP 第 3 字节是 01001101；前 6 位 010011 是子网号，保持不动；后 2 位是主机号的一部分，置 1 变成 11。
- 再想拼起来：第 3 字节变成 01001111＝79，第 4 字节主机号全置 1＝255，前两个字节 180、80 不动，广播地址就是 180.80.79.255，就是 D。
- 拿生活例子验算：一个班 22 位是班级号、10 位是座位号；广播就是“班级号不变、座位号全喊到”，座位号全置 1。
- 边界情况：主机号全是 0 的是网络地址（代表这个子网本身），不是广播地址；主机号全是 1 的才是广播地址。
- 易混对子：不要只把第 4 字节置 1、第 3 字节不动——第 3 字节里也有 2 位主机号，也要置 1；不要把主机号全置 0——那是网络地址，不是广播地址；不要把掩码位数数错——第 3 字节 11111100 是 6 位子网号、2 位主机号。

正确选项为什么对： 子网号 010011 不动，主机号 10 位全置 1，第 3 字节变 79、第 4 字节变 255，广播地址是 180.80.79.255，就是 D，选 D。

错误选项为什么错＋举例：

- 主机号全置 0：那是网络地址，不是广播地址。教训：广播是主机号全 1，网络是主机号全 0。
- 只把第 4 字节置 1：第 3 字节里还有 2 位主机号，也要置 1。教训：主机号有 10 位，分布在第 3 字节后 2 位和第 4 字节。
- 掩码位数数错：11111100 是 6 位子网号，不是 8 位。教训：掩码要一位一位数，1 是子网号、0 是主机号。

笔记本一句话： 第 39 题＋广播地址：子网号不动，主机号全置 1，180.80.79.255。

【图像】



广播地址计算分步图：掩码255.255.252.0定出斜22分界，第三字节01001101前6位子网号不动后2位置1得79，第四字节全置1得255，拼成180.80.79.255

【定位】 计算机网络 · 第4章 网络层 · 已知IP和子网掩码求广播地址

【易混对比】

- "广播地址"和"网络地址"的区别：广播地址是主机号全1（一喊全子网都收到）；网络地址是主机号全0（代表子网本身）。全1和全0，一字之差，意思完全不同。
- "252"和"255"的区别：252＝11111100，是6位子网号加2位主机号，不是整字节；别把第三字节当成8位子网号。
- "只把第4字节置1"和"主机号全置1"的区别：主机号一共10位，分布在第3字节后2位和第4字节8位，两处都要置1，漏了第3字节那2位就错。

子网号010011不动，10位主机号全置1，第三字节变79、第四字节变255，广播地址是180.80.79.255，选D。

【坑位】

252不是255，第三字节里还藏着2位主机号，置1时别漏。

只把第4字节置1、第三字节原样不动：得到180.80.77.255（选项C）——错在第3字节后2位主机号没置1。主机号全置0：得到的是网络地址，不是广播地址。掩码位数数错：把11111100数成8位子网号，后面分界全错——掩码要一位一位数，1是子网号、0是主机号。

【套路】

- 第一步，数掩码定分界：255.255.252.0是前22位1、后10位0，即/22；前22位是子网号，后10位是主机号。
- 第二步，处理第3字节：77＝01001101；前6位010011是子网号，原样不动；后2位是主机号，置1得01001111＝79。
- 第三步，拼出广播地址：第4字节8位主机号全置1得255，前两字节180、80不动，得180.80.79.255，选D。

掩码定分界，子网号原样不动，主机号全置1。

【类比】

像一个班的座位表：前22位是班级号，后10位是座位号；广播就是"班级号不变、把所有座位号都喊一遍"——座位号全置1。只喊第4字节那8个座位、漏了第3字节里的2个座位，就不是全班广播了。术语贴回去：班级号=子网号、座位号=主机号、全班点名=广播地址、班级牌=网络地址（座位号全0）。

【口诀】 掩码斜二十二，主机十位要记住；子网号，原样留，主机号，全置一；七十九点二五五，广播地址就是它。

【考频】 「子网广播地址计算」在 2009–2022 年解析中出现 13 次 · 高频

第 40 题 · SMTP 与 POP3 在邮件传送中分别采用“推”还是“拉”的方式（正确答案：D）

题干与选项：

若用户 1 与用户 2 之间发送和接收电子邮件的过程如下图所示，则图中①、②、③阶段

分别使用的应用层协议可以是( )

- A. SMTP、SMTP、SMTP
- B. POP3、SMTP、POP3
- C. POP3、SMTP、SMTP
- D. SMTP、SMTP、POP3

> 注：邮件收发过程示意图以原卷为准

入手点： 先想推和拉的区别：推是发送方主动送过去，拉是接收方主动去取；再看 SMTP 和 POP3 各是谁主动。

专有名词从零解释：

- 先想“推”：发送方主动把东西送到接收方手里，不用接收方来要。
- 再想“拉”：接收方主动发请求、把东西取回来，不用发送方送上门。
- 再想“SMTP”：发邮件用的协议，用户代理把邮件交给邮件服务器、服务器之间互相传邮件，用的都是它。
- 再想“POP3”：收邮件用的协议，用户想看信时，客户端向邮件服务器发请求、把信取回来。

考查知识点：

- 先想 SMTP：用户代理把邮件推给自己的邮件服务器，邮件服务器之间也是互相推，发送方主动送过去，就是“推”。
- 再想 POP3：用户想读信时，客户端向邮件服务器发请求，把信拉回来，接收方主动去取，就是“拉”。
- 再想答案：SMTP 是推，POP3 是拉，就是 D。
- 拿生活例子验算：SMTP 像寄快递——寄件人主动把包裹送到快递站；POP3 像去快递柜取件——收件人自己去取，不取就一直在柜子里。
- 边界情况：POP3 取完信，服务器上的信一般就删了（也可以设置保留）；SMTP 推完之后，信就存在接收方的服务器上，等着被拉。
- 易混对子：不要把 SMTP 和 POP3 的方向说反——SMTP 是发邮件的推，POP3 是收邮件的拉；不要以为收邮件也是服务器主动推——POP3 一定要客户端先发请求，服务器不会主动把信推到客户端。

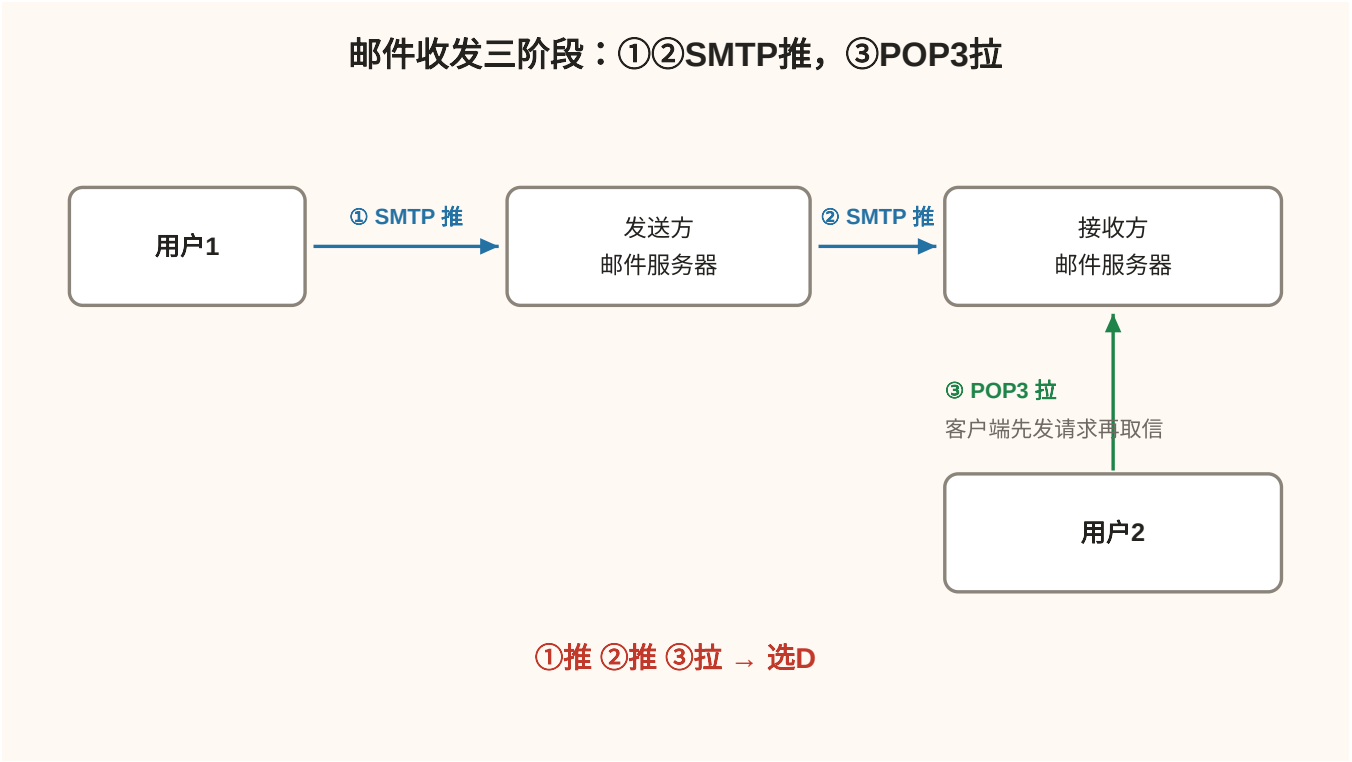
正确选项为什么对： SMTP 发送方主动推，POP3 接收方主动拉，推和拉的对应就是 D，选 D。

错误选项为什么错＋举例：

- 说 SMTP 是拉：SMTP 是发邮件的，发送方主动推。教训：发邮件是寄出去，不是取回来。
- 说 POP3 是推：POP3 一定要客户端先发请求取信，服务器不会主动推。教训：收邮件是取回来，不是等着送上门。
- 说两个都是推或者都是拉：一个推一个拉，方向正好反过来。教训：SMTP 管发、POP3 管收，方向不一样。

笔记本一句话： 第 40 题＋SMTP 推、POP3 拉：SMTP 发送方主动推，POP3 接收方主动取。

【图像】



邮件收发三阶段协议图：用户1用SMTP把信推给发送方服务器①，服务器之间用SMTP互相推送②，用户2用POP3向接收方服务器发请求把信拉回来③

【定位】 计算机网络 · 第6章 应用层 · 邮件收发各阶段的应用层协议

【易混对比】

- "推"和"拉"的区别：推是发送方主动把信送过去；拉是接收方主动发请求、把信取回来。主动的一方不一样。
- "SMTP"和"POP3"的区别：SMTP管发邮件（用户推给服务器、服务器之间互相推）；POP3管收邮件（客户端发请求从服务器拉回来）。分工和方向都不同。
- "阶段②"和"阶段③"的区别：②是邮件服务器之间传送，用SMTP推；③是用户从自己的服务器取信，用POP3拉。别把服务器之间的传送也填成POP3。

①SMTP推、②SMTP推、③POP3拉，对应D。

【坑位】

POP3只出现在"用户从自己服务器取信"的最后一段，服务器之间传送一律SMTP。

选A（三个全SMTP）：③是用户取信，服务器不会主动把信推到用户手里，必须是POP3拉。选B或C（含POP3在①或②）：POP3只管"收"，发邮件和服务器之间传邮件都是SMTP的活。记住分工口诀：发信、传信靠SMTP推，取信靠POP3拉；看到"用户向服务器要信"，就是拉，就是POP3。

【类比】

SMTP像寄快递：你把包裹送到快递站（用户推给服务器），快递站之间互相转运（服务器之间推）；POP3像去快递柜取件：包裹到了你家附近的柜子，但得你自己去取，不取就一直在柜子里——快递站不会把包裹送到你手上。术语贴回去：寄包裹=SMTP推、快递站=邮件服务器、去柜子取件=POP3拉。

【口诀】 发信推送SMTP，服务器之间也推它；取信要拉POP3，一推一推再一拉，选D没差。

【考频】 「SMTP；POP3」在 2009–2022 年解析中出现 6 次 · 中频

附录 · 答案速查表

- 1–8：B、A、A、B、C、C、C、A
- 9–16：D、A、D、D、B、D、D、A
- 17–24：C、C、C、D、D、B、C、B
- 25–32：B、A、D、A、B、C、A、B
- 33–40：B、C、A、B、C、A、D、D